

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA



PROYECTO DE BASE DE DATOS

Aeropuerto Internacional Jorge Chávez

ALUMNOS:

Benjamin Toro Leddihn – 202510596

Thiago Frias Pinto – 202510638

Jobeth Alexander Leon Pantaleon – 202510035

CURSO:

Base de Datos I

Lima, Perú

Índice

1. Requisitos	4
1.1. Introducción	4
1.2. Descripción general del problema/organización/empresa	4
1.3. Necesidad/usos de la base de datos	4
1.4. ¿Cómo resuelve el problema hoy?	4
1.5. Descripción detallada del sistema	5
1.5.1. Objetos de información actuales	5
1.5.2. Características y funcionalidades esperadas	5
1.5.3. Tipos de usuarios existentes/necesarios	5
1.5.4. Tipos de consulta, actualizaciones	6
1.5.5. Tamaño estimado de la base de datos	6
1.6. Objetivos del proyecto	7
1.7. Referencias del proyecto	7
1.8. Eventualidades	8
1.8.1. Problemas que pudieran encontrarse en el proyecto	8
1.8.2. Límites y alcances del proyecto	9
2. Diseño conceptual	9
2.1. Reglas semánticas	9
2.2. Modelo Entidad-Relación	10
2.3.1. Entidad Persona	10
2.3.2. Entidad Pasajero	10
2.3.3. Entidad Categoría Migratoria	11
2.3.4. Entidad Ticket	11
2.3.5. Entidad Equipaje	12
2.3.6. Entidad Vuelo	12
2.3.7. Entidad Aeronave	12
2.3.9. Entidad Aerolínea	12
2.3.10. Entidad Recurso	13
2.3.11. Entidad Manga	13

2.3.12. Entidad Radar	13
2.3.13. Entidad Incidencia	13
2.3.14. Entidad Personal (Operativo y Tripulación)	14
3. Modelo Relacional	14
3.1. Modelo Relacional	14
3.2. Especificaciones de Transformación	15
3.2.1. Entidades	15
3.2.2. Entidades Débiles	16
3.2.3. Entidades superclase/ subclases	16
3.2.4. Relaciones Binarias	17
3.2.5. Relaciones Ternarias	17
3.3. Diccionario de Datos	17
4. Implementación de la Base de Datos	22
4.1. Creación de la Base de Datos	22
4.2. Carga de Datos	28
4.3. Simulación de Datos	31
4.3.1. Escenario 1 000 registros	32
4.3.2. Escenario 10 000 registros	35
4.3.3. Escenario 100 000 registros	39
4.3.4. Escenario 1 000 000 registros	43
4.4. Generación de vistas de usuario	46
5. Experimentación y evaluación de rendimiento	48
5.1. Consultas SQL para el experimento	48
5.1.1. Descripción del tipo de consultas seleccionadas	48
5.1.2. Implementación de consultas en SQL	49
5.2. Metodología del experimento	50
5.3. Optimización de consultas	51
5.3.1. Índices para la Consulta 1	51
5.3.2. Índices para la Consulta 2	52
5.3.3. Índices para la Consulta 3	52

5.3.4. Identificación de índices implementados por PostgreSQL	52
5.4. Plataforma de Pruebas	53
5.4. Medición de tiempos	54
5.4.1. Ejecución de consulta 1	54
5.4.2. Ejecución de consulta 2	56
5.4.3. Ejecución de consulta 3	58
5.4.4. Ejecución de consulta 1	61
5.4.5. Ejecución de consulta 2	63
5.4.6. Ejecución de consulta 3	64
5.4.7. Ejecución de consulta 1	68
5.4.8. Ejecución de consulta 2	70
5.4.9. Ejecución de consulta 3	72
5.4.10. Ejecución de consulta 1	75
5.4.11. Ejecución de consulta 2	78
5.4.12. Ejecución de consulta 3	79
5.5. Medición de Tiempos	83
5.5.1. Sin Índices	83
5.5.2. Con Índices	84
5.6. Resultados	85
5.6.1. Consulta 1	85
5.6.2. Consulta 2	86
5.6.3. Consulta 3	87
5.7. Análisis y Discusión	87
6. Stored Procedures	89
6.1. Registro de un nuevo pasajero	89
6.2. Emisión de un ticket con su check-in	90
6.3. Registro de una incidencia sobre un recurso	91
7. Conclusiones	92
8. Anexos	93

1. Requisitos

1.1. Introducción

El presente proyecto detalla el diseño y la implementación de una arquitectura de base de datos relacional para el Aeropuerto Internacional Jorge Chávez (LIM), bajo la gestión de Lima Airport Partners (LAP). En el actual contexto de transición hacia su nueva infraestructura, el terminal enfrenta desafíos críticos de saturación y fallas sistémicas. El sistema propuesto en PostgreSQL busca centralizar la información operativa, permitiendo una gestión eficiente de recursos limitados y garantizando la integridad de los datos en un entorno que exige alta disponibilidad y tiempos de respuesta mínimos ante contingencias.

1.2. Descripción general del problema/organización/empresa

LAP opera el terminal aéreo más importante del país, el cual atraviesa una crisis de implementación tras su expansión en 2025. El problema central es la desconexión operativa: el nuevo terminal funciona con infraestructura incompleta (mangas de embarque inoperativas), fallas recurrentes en sistemas de radar y un colapso logístico en sus accesos terrestres (Av. Morales Duárez). Esta desorganización afecta directamente la competitividad del Jorge Chávez como hub regional, provocando retrasos masivos y una gestión ineficiente de las operaciones aeronáuticas.

1.3. Necesidad/usos de la base de datos

La base de datos es una herramienta estratégica de gestión de crisis diseñada para:

- Optimización de Infraestructura: Administrar dinámicamente la asignación de mangas operativas para reducir tiempos de espera en pista.
- Control de Contingencias: Registrar y monitorear fallas técnicas (inundaciones, radares) vinculándose a vuelos afectados.
- Trazabilidad de Ingresos: Gestionar el impacto de la nueva tarifa TUUA en pasajeros de conexión para predecir la fuga de rutas internacionales hacia otros terminales de la región.

1.4. ¿Cómo resuelve el problema hoy?

1.4.1. ¿Cómo se almacenan/procesan los datos hoy?

Actualmente, el aeropuerto opera con silos de información. LAP (infraestructura), Corpac (navegación) y las aerolíneas poseen sistemas independientes que no se sincronizan en tiempo real. Gran parte de los reportes de fallas técnicas y saturación vial aún se procesan en hojas de cálculo y sistemas legados desconectados de los planes de vuelo.

1.4.2. Flujo de datos

El flujo actual es reactivo y fragmentado. Ante una falla de radar o falta de suministro, la información viaja por canales de radio o mensajería interna, generando una latencia crítica. La reprogramación de

vuelos ocurre sin una visión unificada de los recursos disponibles, lo que agrava el colapso del terminal durante horas punta.

1.5. Descripción detallada del sistema

1.5.1. Objetos de información actuales

- Recursos Críticos: Entidades físicas limitadas que determinan la capacidad operativa del terminal. Incluye las Mangas (Puentes de Embarque) con su estado de disponibilidad y clase técnica, y los Radares (sujetos a fallas técnicas), ambos modelados como especializaciones de un Recurso de infraestructura.
- Operaciones: Componentes logísticos del flujo aéreo. Comprende los Vuelos (con itinerario, estado y clasificación nacional/internacional), las Tripulaciones (personal con licencias específicas) y las Aeronaves (configuradas por modelo, capacidad y clase de envergadura).
- Transacciones: Registros de flujo económico. Incluye los Tickets de abordaje, el procesamiento de Equipaje y los Cobros de Tarifas (TUUA), diferenciando las categorías migratorias nacional, internacional y de tránsito.
- Aerolíneas: Empresas concesionarias que operan en el aeropuerto. Se almacena su información fiscal (RUC), nombre comercial y la alianza global a la que pertenecen.
- Pasajeros: Usuarios finales del sistema. Se registra su información personal, documentos de identidad y su categoría migratoria, fundamental para la aplicación de la tarifa de conexión.
- Equipaje: Entidad ligada al pasajero y al vuelo. Se controla mediante un identificador único (Tag ID), su peso y la cantidad de bultos asociados.

1.5.2. Características y funcionalidades esperadas

- Gestión de Estados en Tiempo Real: Seguimiento constante de la operatividad de cada manga para una asignación inteligente de slots de parqueo.
- Módulo de Contingencias: Sistema de registro de fallas técnicas anclado a códigos de vuelo específicos para análisis de responsabilidad y seguros.
- Automatización de Tarifas: Aplicación del cobro TUUA para pasajeros en tránsito, discriminando por tipo de conexión y destino final.

1.5.3. Tipos de usuarios existentes/necesarios

- Administrador de Sistema (DBA): Responsable de la integridad referencial, backups y la optimización de índices para el manejo de un millón de registros.
- Agente de Operaciones LAP: Encargado de la asignación de recursos físicos y el reporte inmediato de fallas de infraestructura.

- Personal de Aerolínea: Usuarios operativos para check-in, pesaje de equipaje y gestión de boarding passes.
- Autoridad Migratoria y Seguridad: Acceso a consultas de lectura sobre manifiestos de pasajeros y flujos de conexión internacional.

1.5.4. Tipos de consulta, actualizaciones

Las principales consultas las realizarán los distintos actores del ecosistema aeroportuario (administradores de LAP, personal de aerolíneas y autoridades). Algunos ejemplos de consultas de nivel de complejidad aceptable para la gestión de la crisis actual son:

- ¿Cuál es el recurso de infraestructura (manga o radar) que presenta la mayor cantidad de fallas técnicas en la última semana? (Ataca el problema de fallas técnicas).
- ¿Cuál es el tiempo promedio de retraso de los vuelos internacionales? (Ataca la saturación operativa).
- ¿Cuál es la aerolínea con mayor cantidad de incidencias por falta de suministro de combustible en el terminal? (Ataca las fallas de planificación).
- ¿Cuál es la recaudación estimada de la tarifa TUUA según cada categoría migratoria? (Ataca el problema de trazabilidad de ingresos y competitividad).
- ¿Qué porcentaje de los vuelos programados en "hora punta"(06–09 h y 18–21 h) terminó retrasado? (Ataca el problema de saturación en horarios críticos). Asimismo, se consideran distintos tipos de actualizaciones para mantener la integridad del sistema ante la volatilidad de la operación:
- Actualizaciones automáticas: Corresponden a los cambios dinámicos en la data operativa ya existente, como la actualización del estado de un vuelo (de "Programado.^a Retrasado"), el registro de la hora real de salida o la fluctuación del peso registrado en el equipaje.
- Actualizaciones manuales: De las cuales se encargará un usuario administrador (LAP) o personal autorizado de aerolínea, como la creación de una nueva incidencia técnica (ej. reporte de inundación), la habilitación de una nueva manga de embarque tras su construcción o la modificación de las tarifas TUUA por decreto institucional. Estos cambios deberán realizarse de forma coherente con las restricciones de integridad y las reglas de negocio establecidas en la base de datos.

1.5.5. Tamaño estimado de la base de datos

La cantidad de registros por tabla será considerablemente distinta debido a la naturaleza operativa del terminal. La tabla con mayor volumen de data es la de Incidencia, la cual, debido a la crisis de infraestructura reportada (fallas de radar, inundaciones, saturación), ascenderá a un millón de registros para fines de experimentación. Se calculan decenas de miles de registros para Pasajeros, cientos de Recursos y Aerolíneas, y miles de Vuelos mensuales. Cada tabla está diseñada para escalar según la demanda operativa hasta el año 2026.

Nombre de la Tabla	Longitud de los Atributos	Longitud del Registro
Persona	4+41+41+10+1	97
Pasajero	4+21+21+1	47
Personal	4+15+10	29
CategoriaMigratoria	4+31+8	43
Recurso (Manga/Radar)	4+21+31+1	57
Aeronave	4+10+21+4	39
Asiento	4+5+15	24
Vuelo	8+11+8+8+4+4+11+12	93
Ticket	8+10+8+4+4+4	38
Incidencia	8+10+101+4+4	127
Aerolinea	4+11+41+21	77
Equipaje	8+21+8+4+4	45
Utiliza	8+4	12
Afecto	8+4	12
Retrasa	8+8	16
OperaTripulacion	4+8	12

Figura 1: Estimación de la Base de Datos

1.6. Objetivos del proyecto

- Formular una base de datos en base a la operatividad del Aeropuerto Internacional Jorge Chávez de manera que se puedan realizar estudios y simulaciones de gestión logística en base a la data.
- Optimizar los tiempos de respuesta de la base de datos de manera que sea capaz de soportar consultas complejas (como el impacto de la tarifa TUUA o saturación de pistas) a pesar de su enorme cantidad de datos.

1.7. Referencias del proyecto

Parte de la base de datos que utilizaremos se obtuvo adaptando información proveniente del portal de Lima Airport Partners (LAP) y reportes de la Corpac, los cuales se pueden hallar ingresando a sus webs oficiales. Asimismo, se consultaron fuentes periodísticas y técnicas sobre la nueva terminal para mapear los problemas de infraestructura actuales.

1.8. Eventualidades

1.8.1. Problemas que pudieran encontrarse en el proyecto

Nombre de la Tabla	Tamaño en Bytes	Datos Estimados	Tamaño Total en Bytes
Persona	97	1,005,000	97,485,000
Pasajero	47	1,000,000	47,000,000
Personal	29	5,000	145,000
CategoriaMigratoria	43	3	129
Recurso	57	500	28,500
Aeronave	39	1,000	39,000
Asiento	24	200,000	4,800,000
Vuelo	93	100,000	9,300,000
Ticket	38	1,000,000	38,000,000
Incidencia	127	1,000,000	127,000,000
Aerolinea	77	100	7,700
Equipaje	45	1,000,000	45,000,000
Utiliza	12	200,000	2,400,000
Afecto	12	800,000	9,600,000
Retrasa	16	800,000	12,800,000
OperaTripulacion	12	400,000	4,800,000
Total			398,405,329

Figura 2: Estimación de la Base de Datos

- El tamaño de la base de datos podría ser un problema, pues dificulta la escalabilidad de los datos, especialmente al manejar un millón de registros de incidencias y tickets.
- Este proyecto está tomando en cuenta únicamente las características actuales y los problemas de la inauguración del nuevo terminal, pero debe considerarse que el flujo de pasajeros crece anualmente y la infraestructura se completará progresivamente hasta 2026. Mantener toda la data actualizada y coherente frente a estos cambios físicos y normativos puede significar una complicación.
- Asimismo, este proyecto podría ser vulnerable frente a corrupción o pérdida imprevista de datos. Dado que se utilizará información real de vuelos y recursos para optimizar la toma de decisiones, información perdida significa menor exactitud en los resultados de gestión de crisis.
- Finalmente, dado que el aeropuerto recibe actualizaciones en sus protocolos de seguridad y tarifas paulatinamente, el contenido dentro de las tablas puede cambiar considerablemente a lo largo del tiempo. En consecuencia, es posible que el modelo de base de datos usado resulte limitado para futuras expansiones del terminal (como una tercera pista).

1.8.2. Límites y alcances del proyecto

Alcances: Este proyecto busca un alcance operativo total para el terminal Jorge Chávez, ya que el público objetivo serán los administradores de LAP, aerolíneas y autoridades interesadas en una herramienta de optimización de recursos y mitigación de retrasos.

Límites: A pesar del impacto en vuelos internacionales, en primera instancia, la herramienta solo contará con versiones en español e inglés, ya que actualmente no se cuenta con los diccionarios técnicos aeronáuticos oficiales para otros idiomas de la región.

2. Diseño conceptual

2.1. Reglas semánticas

En el diseño del modelo E-R se deben tomar en cuenta las siguientes reglas semánticas que rigen la compleja operativa del terminal:

- Un Pasajero tiene nombre, apellido y fecha de nacimiento heredados de Persona, y se identifica internamente por su `id_persona`. Adicionalmente, posee una clave alternativa única conformada por la combinación de un Tipo de Documento (DNI, Pasaporte o Carné de Extranjería) y su respectivo Número de Documento. Cada pasajero podrá poseer varios Tickets, identificados por un `id`, con precio, fecha de emisión y estado de boarding.
- Cada pasajero pertenece necesariamente a una única Categoría Migratoria. Las categorías se identifican por un `id`, tienen un nombre (Nacional, Internacional, Tránsito) y una tasa de tarifa TUUA asociada.
- Los Vuelos se identifican por un `id` y un número de vuelo comercial. Cada vuelo registra su fecha y hora de salida programada, su fecha y hora real (cuando ya ha despegado), su estado operativo (Programado, Embarcando, Despegado, Aterrizado, Retrasado o Cancelado) y su tipo (Nacional o Internacional). Cada vuelo debe tener asignada obligatoriamente una Aeronave y al menos un Recurso de infraestructura (Manga o Radar).
- Cada Ticket puede tener asociado un único registro de Check-in, que representa el momento en que el pasajero realiza su facturación para el vuelo. El check-in registra la fecha y hora en que se efectúa, el mostrador (counter) de atención y si el pasajero registró equipaje. Un check-in no puede existir sin el ticket al que pertenece.
- Cada Vuelo es operado comercialmente por una única Aerolínea, y una aerolínea opera múltiples vuelos.
- Cada Vuelo es atendido por uno o más tripulantes, y un mismo tripulante puede ser asignado a varios vuelos a lo largo del tiempo (relación de muchos a muchos).
- Las Aeronaves se identifican con una placa única; tienen modelo, fabricante, capacidad de pasajeros y una clase de envergadura (código OACI A-F). Además, poseen un conjunto de Asientos.

- Los Asientos se identifican por el código de asiento (ej. 12A) y por la placa del avión al que pertenecen.
- El personal del aeropuerto se identifica por un id de persona. Existen empleados de tipo Tripulación, que poseen obligatoriamente un número de licencia de vuelo, y empleados de tipo Personal Operativo, que se identifican por su área operativa asignada.
- Los Recursos de infraestructura se identifican por un id único y se registran mediante su nombre técnico de locación dentro del terminal. Existen dos tipos: las Mangas, que registran su estado de acople (Libre, Ocupado, Mantenimiento), su longitud máxima de extensión y su clase técnica máxima admitida; y los Radares, que cuentan con un rango de alcance en millas náuticas y su frecuencia de operación.
- Las Incidencias (fallas técnicas, inundaciones o falta de combustible) se identifican por un id único, una gravedad, una descripción y un tipo categórico de incidencia (Falla_Radar, Inundacion, Falta_Combustible, Saturacion_Vial, Manga_Inoperativa u Otro). Por su naturaleza, cada incidencia debe estar vinculada a uno o más recursos afectados o a uno o más vuelos retrasados.
- El Equipaje se identifica por un Tag ID único. Cada equipaje pertenece a un solo pasajero y está vinculado a un vuelo específico. Posee un peso.
- Existe una restricción de integridad física: una aeronave cuya clase de envergadura supere la clase máxima admitida por una manga no puede ser asignada a dicha manga (por ejemplo, una aeronave Clase F"no puede acoplarse a una manga de Clase C").
- Cada Aerolínea se identifica con su RUC, tiene un nombre comercial y pertenece únicamente a una de las tres alianzas globales: 'Star Alliance', 'SkyTeam' u 'Oneworld'.

2.2. Modelo Entidad-Relación

[Link a Lucid del modelo de entidad relación](#)

2.3.1. Entidad Persona

Especificaciones.

Superclase que centraliza los datos de identidad de todos los individuos. Su llave primaria es id_persona y almacena el nombre, apellido y fecha de nacimiento.

Consideraciones.

Se utiliza una jerarquía de especialización (IsA) para optimizar la estructura, separando Pasajeros de Personal y manteniendo roles lógicos distintos sin duplicar los datos básicos de identidad.

2.3.2. Entidad Pasajero

Especificaciones.

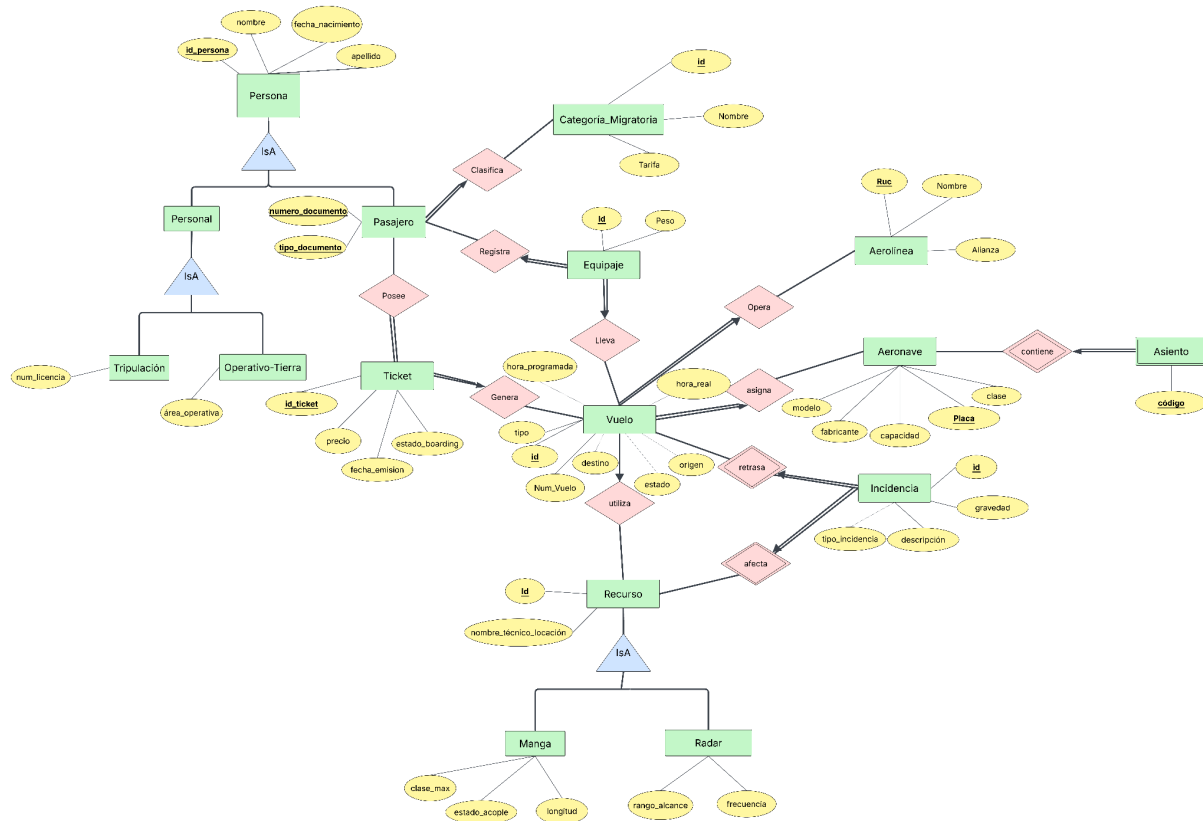


Figura 3: Elemento gráfico extraído de la página 9 del documento original.

Subclase de Persona. Se identifica de forma única mediante la combinación de tipo_documento y numero_documento.

Consideraciones.

Aunque hereda el ID, la combinación de documentos actúa como una llave alternativa necesaria para procesos migratorios. Se ignora la nacionalidad detallada ya que el análisis se centra en su categoría migratoria.

2.3.3. Entidad Categoría Migratoria

Especificaciones.

Define el estatus del pasajero. La llave primaria es un id y contiene el nombre de la categoría y la tarifa asociada.

Consideraciones.

Se modela de forma independiente para permitir que la tasa TUUA sea un atributo dinámico que cambie por temporada sin afectar los registros históricos de los pasajeros.

2.3.4. Entidad Ticket

Especificaciones.

Documento de viaje con un id_ticket como llave primaria. Almacena el precio, fecha de emisión y estado del boarding.

Consideraciones Es la entidad que vincula económicamente al Pasajero con el Vuelo. El estado del boarding es crítico para disparar la validación del equipaje.

2.3.5. Entidad Equipaje

Especificaciones.

Se identifica por un id (Tag ID) como llave primaria. Almacena el peso del equipaje.

Consideraciones.

Es una entidad asociativa con dos dependencias 1:N: depende de un pasajero (dueño) y de un vuelo (trayecto). No puede existir equipaje sin un dueño y un trayecto confirmado.

2.3.6. Entidad Vuelo

Especificaciones.

Entidad central identificada por un id y un num_vuelo.

Consideraciones.

Posee participación total con Aeronave y Recurso. En la lógica del terminal, un vuelo es una unidad operativa que no puede existir .^{en} el papel"si no tiene un avión y una locación (manga o radar) asignada.

2.3.7. Entidad Aeronave

Especificaciones.

Identificada por su placa (matrícula). Almacena modelo, fabricante, capacidad y clase de envergadura (clase).

Consideraciones.

La clase de envergadura es un atributo técnico fundamental para la restricción de integridad física con las mangas, que impide asignar una aeronave a una manga cuya clase máxima admitida sea inferior.

Especificaciones.

Entidad débil de Aeronave. Su llave es el código del asiento y el ID de la aeronave.

Consideraciones.

Es una entidad débil por existencia; si una aeronave se retira de la flota, sus asientos dejan de existir en el sistema de reservas automáticamente.

2.3.9. Entidad Aerolínea

Especificaciones.

Identificada por el RUC. Almacena el nombre y la alianza global.

Consideraciones.

La alianza se restringe a tres valores fijos (Star Alliance, SkyTeam, Oneworld) mediante un tipo enumerado, agrupando a las aerolíneas operadoras bajo un mismo estándar comercial.

2.3.10. Entidad Recurso**Especificaciones.**

Superclase de infraestructura identificada por un id y su nombre_técnico_locación.

Consideraciones.

Centraliza la infraestructura del terminal. Se utiliza para que las incidencias puedan reportarse de manera uniforme sobre cualquier punto físico del aeropuerto.

2.3.11. Entidad Manga**Especificaciones.**

Especialización de Recurso. Almacena el estado_acople, la longitud y la clase máxima de aeronave admitida (clase_max).

Consideraciones.

Atributos específicos que solo aplican a este tipo de recurso. Se ignora el material de construcción por no ser relevante para la asignación de vuelos.

2.3.12. Entidad Radar**Especificaciones.**

Especialización de Recurso. Almacena el rango_alcance y la frecuencia.

Consideraciones.

El radar requiere atributos de radiofrecuencia (rango de alcance y frecuencia) que no comparten otros recursos de infraestructura.

2.3.13. Entidad Incidencia**Especificaciones.**

Identificada por un id como llave primaria. Almacena la gravedad (severidad), la descripción, el tipo de incidencia (Falla_Radar, Inundacion, Falta_Combustible, Saturacion_Vial, Manga_Inoperativa u Otro) y las fechas de reporte y cierre.

Consideraciones.

Es una entidad dependiente de un Recurso o un Vuelo. La severidad permite priorizar el mantenimiento operativo, filtrando desde fallos leves hasta crisis críticas.

2.3.14. Entidad Personal (Operativo y Tripulación)

Especificaciones.

Subclase de Persona. Se divide en Tripulación (con num_licencia) y Operativo-Tierra (con área_operativa).

Consideraciones.

Se modelan por separado para gestionar los permisos de acceso a zonas restringidas y la asignación de roles técnicos en el vuelo.

3. Modelo Relacional

3.1. Modelo Relacional

```

1  Persona (id_persona: INT, nombre: VARCHAR(50), apellido: VARCHAR(50),
2  fecha_nacimiento: DATE)
3
4  Pasajero (Persona.id_persona: INT, tipo_documento: VARCHAR(20),
5  numero_documento: VARCHAR(15), CM.id: INT)
6
7  Categoria_Migratoria (id: INT, nombre: VARCHAR(30), tarifa: DECIMAL(10,2))
8
9  Personal (Persona.id_persona: INT)
10
11 Tripulacion (Persona.id_persona: INT, num_licencia: VARCHAR(20))
12
13 Operativo-Tierra (Persona.id_persona: INT, area_operativa: VARCHAR(30))
14
15 Equipaje (id: VARCHAR(20), peso: FLOAT, Pasajero.id_persona: INT, Vuelo.id: INT)
16
17 Ticket (id_ticket: INT, precio: float, fecha_emisión: DATE, estado_boarding:
18 VARCHAR(20), Vuelo.id: INT, Persona.id_persona: INT)
19
20 Vuelo (id: INT, num_vuelo: VARCHAR(10), hora_programada: TIMESTAMPTZ,
21 hora_real: TIMESTAMPTZ, estado: estado_vuelo_t, tipo: tipo_vuelo_t, origen:
22 VARCHAR(4), destino: VARCHAR(4), Aeronave.placa: VARCHAR(10),
23 Aerolinea.ruc: VARCHAR(11))
24
25
26 Aeronave (Placa: VARCHAR(10), modelo: VARCHAR(50), fabricante: VARCHAR(50),
    ↪ capacidad: INT, clase: VARCHAR(10))
27
28 Asiento (código: VARCHAR(5), A.Placa: VARCHAR(10))

```

```

29
30 Aerolinea (Ruc: VARCHAR(11), nombre: VARCHAR(50), alianza: VARCHAR(20))
31
32 Recurso (id: INT, nombre_técnico_locación: VARCHAR(100))
33
34 Manga (Recurso.id: INT, estado_acople: VARCHAR(20), longitud: float, clase_max:
    ↪ VARCHAR(10))
35
36 Radar (Recurso.id: INT, rango_alcance: INT, frecuencia: VARCHAR(20))
37
38 Incidencia (id: INT, gravedad: VARCHAR(15), descripción: TEXT, tipo_incidencia:
    ↪ ENUM('Falla_Radar', 'Inundacion', 'Falta_Combustible', 'Saturacion_Vial', '
    ↪ Manga_Inoperativa', 'Otro') )
39
40 Utiliza (Vuelo.id: INT, Recurso.id: INT)
41
42 Afecto (Incidencia.id: INT, Recurso.id: INT)
43
44 Retrasa (Incidencia.id: INT, Vuelo.id: INT)
45
46 OperaTripulacion (Tripulacion.id_persona: INT, Vuelo.id: INT)
47
48 CheckIn (Ticket.id_ticket: BIGINT, fecha_hora: TIMESTAMPTZ, counter: VARCHAR(10),
    ↪ con_equipaje: BOOLEAN)

```

3.2. Especificaciones de Transformación

3.2.1. Entidades

Las entidades fuertes identificadas en el modelo E-R (Categoria_Migratoria, Vuelo, Aeronave, Aerolinea, Ticket) se transformaron a relaciones independientes en el modelo lógico, conservando los atributos atómicos definidos durante el análisis conceptual. Para cada relación se eligió una clave primaria estable, mínima e inmutable, garantizando que ningún atributo descriptivo pueda alterar su valor a lo largo del ciclo de vida del registro. En las entidades operacionales del aeropuerto (Vuelo, Ticket, Categoria_Migratoria, Recurso) se priorizó el uso de identificadores sustitutos numéricos (INT) sobre claves naturales con semántica externa, dado que estos últimos suelen estar sujetos a regulaciones cambiantes (por ejemplo, los formatos OACI/IATA de codificación de vuelos o las matrículas aeronáuticas). Por el contrario, en Aerolinea se conserva el RUC (VARCHAR(11)) y en Aeronave la Placa de matrícula (VARCHAR(10)) como claves naturales, puesto que ambos son identificadores formales, únicos y de uso obligatorio definidos por entes regulatorios (SUNAT y la Dirección General de Aeronáutica Civil, respectivamente). Todos los atributos descriptivos se tipificaron con dominios PostgreSQL nativos (INT, VARCHAR(n), DATE, DECIMAL, FLOAT, TEXT) buscando minimizar el espacio de almacenamiento y reforzar la integridad de dominio. Las restricciones NOT NULL se aplican sobre los atributos críticos para la operación aeroportuaria (claves primarias, claves foráneas referenciales y datos imprescindibles para procesos migratorios y de facturación TUUA).

3.2.2. Entidades Débiles

En el modelo conceptual se identificaron tres entidades débiles: Asiento, Incidencia y CheckIn. Las tres dependen, en términos de identidad o existencia, de una entidad fuerte y por tanto su transformación al modelo relacional requiere propagar la clave primaria de su entidad identificadora.

La entidad Asiento es débil por identidad respecto de Aeronave: el código de asiento (por ejemplo, 12A) no es único globalmente, sino únicamente dentro de cada aeronave. Por esta razón, su clave primaria en el modelo lógico es la combinación (Aeronave.Placa, código). Adicionalmente, se establece una restricción ON DELETE CASCADE sobre la clave foránea Aeronave.Placa, garantizando que la baja de una aeronave de la flota elimine automáticamente todos los asientos asociados, preservando la consistencia referencial del sistema de reservas.

La entidad Incidencia es débil por existencia: ninguna incidencia puede registrarse en el sistema sin estar asociada a un recurso afectado (Afecto) o a un vuelo retrasado (Retrasa). Su clave primaria propia (id) actúa como sustituto identificador, pero su existencia en la

base de datos está condicionada por las relaciones Afecto y Retrasa que materializan la dependencia existencial.

La entidad CheckIn es débil por existencia respecto de Ticket: un registro de check-in no puede existir sin el ticket al que está asociado. En su transformación al modelo relacional, la clave primaria de la entidad fuerte (Ticket.id_ticket) se propaga como clave primaria y, simultáneamente, como clave foránea de la tabla CheckIn, garantizando la relación 1:1 (un ticket admite a lo sumo un check-in). Sobre dicha clave foránea se establece la política ON DELETE CASCADE, de modo que al eliminarse un ticket se elimine automáticamente su registro de check-in asociado, preservando la integridad referencial.

3.2.3. Entidades superclase/ subclases

El modelo E-R contempla tres jerarquías de especialización (IsA): Persona (superclase) → Pasajero / Personal (subclases); Personal (superclase) → Tripulacion / Operativo-Tierra (subclases); y Recurso (superclase) → Manga / Radar (subclases). Para implementar estas jerarquías se adoptó la estrategia denominada Tabla por Subclase (Class Table Inheritance o CTI). Bajo este enfoque, cada subclase se materializa como una tabla independiente cuya clave primaria es simultáneamente clave foránea hacia la tabla de la superclase. Por ejemplo, la tabla Pasajero almacena exclusivamente los atributos específicos del rol pasajero (tipo_documento, numero_documento, categoria_migratoria) y referencia mediante id_persona los atributos comunes residentes en Persona (nombre, apellido, fecha_nacimiento). De forma análoga, Tripulacion y Operativo-Tierra heredan de Personal, que a su vez hereda de Persona. Las razones técnicas que justifican esta decisión sobre alternativas como Single Table Inheritance o Concrete Table Inheritance son: (a) eliminación de la redundancia, ya que los atributos comunes se almacenan una única vez; (b) preservación de la integridad referencial, dado que un mismo individuo no puede registrarse simultáneamente como Pasajero y Tripulación con id_persona distintos; (c) ausencia de columnas NULL masivas que existirían si todas las subclases compartieran una sola tabla; y (d) normalización plena hasta 3FN. La cobertura de las jerarquías es parcial y disjunta: parcial, porque una Persona registrada no necesariamente es Pasajero o Personal; disjunta, porque una misma persona no puede pertenecer simultáneamente a más de una subclase dentro del mismo nivel jerárquico.

3.2.4. Relaciones Binarias

Las relaciones 1:N implementadas son: (i) Aerolínea — Vuelo, donde Vuelo recibe la clave foránea Aerolínea.Ruc, pues una aerolínea opera múltiples vuelos pero cada vuelo pertenece a una sola aerolínea; (ii) Aeronave — Vuelo, donde Vuelo recibe la clave foránea Aeronave.Placa con participación total del lado Vuelo (todo vuelo requiere una aeronave); (iii) Vuelo — Ticket, donde Ticket propaga Vuelo.id; (iv) Vuelo — Equipaje, propagando Vuelo.id en la tabla Equipaje; (v) Categoría_Migratoria — Pasajero, donde Pasajero recibe CM.id como FK; (vi) Aeronave — Asiento, ya descrita en la sección de entidades débiles; y (vii) Pasajero — Ticket, donde Ticket propaga id_persona, pues un pasajero posee varios tickets pero cada ticket pertenece a un único pasajero.

Las relaciones M:N se materializaron como tablas intermedias de unión: (i) Utiliza (Vuelo.id, Recurso.id), que modela que un vuelo emplea múltiples recursos —mangas, radares— y un mismo recurso puede ser utilizado por múltiples vuelos a lo largo del día; (ii) Retrasa (Incidencia.id, Vuelo.id), que registra qué vuelos se ven demorados por qué incidencias, considerando que una sola incidencia (por ejemplo, la falla de un radar) puede impactar a decenas de vuelos simultáneos; (iii) Afecto (Incidencia.id, Recurso.id), que vincula incidencias con recursos averiados; y (iv) OperaTripulacion (Tripulacion.id_persona, Vuelo.id), que registra qué tripulantes operan cada vuelo. En todos los casos la clave primaria de la tabla intermedia es la unión de las claves foráneas, asegurando la unicidad de cada combinación.

3.2.5. Relaciones Ternarias

El modelo no contiene relaciones ternarias puras, es decir, aquellas en que la asociación simultánea de tres entidades aporta información que no puede descomponerse en relaciones binarias sin pérdida semántica. El caso que inicialmente sugiere una estructura ternaria —el vínculo entre Pasajero, Vuelo y Equipaje— se resuelve correctamente como una entidad asociativa con dos dependencias binarias 1:N: cada maleta (Equipaje) pertenece a un único pasajero y se transporta en un único vuelo, mientras que un pasajero puede tener varias maletas y un vuelo agrupa las maletas de muchos pasajeros.

Equipaje posee un identificador propio (id VARCHAR(20), el Tag ID físico generado por el sistema BHS — Baggage Handling System) que actúa como clave primaria, y propaga dos claves foráneas: id_persona hacia Pasajero e id_vuelo hacia Vuelo. Se eligió el Tag ID como clave primaria —en lugar de una clave compuesta de las foráneas— por su valor operativo: es el código de barras impreso físicamente sobre la etiqueta adherida a la maleta y permite trazabilidad inmediata en la faja de equipajes.

Para asegurar la integridad referencial, sobre ambas claves foráneas se establecen restricciones ON DELETE RESTRICT, impidiendo eliminar registros de Pasajero o Vuelo mientras existan equipajes asociados. Esta política previene huérfanos en escenarios de cancelación, exigiendo que el proceso operativo primero reasigne o libere las maletas antes de eliminar el vuelo o el pasajero.

3.3. Diccionario de Datos

A continuación se presenta el diccionario de datos de las veinte (20) relaciones que conforman el modelo de la base de datos del Aeropuerto Internacional Jorge Chávez. Para cada tabla se especifica el nombre del campo, su tipo de dato PostgreSQL nativo, si forma parte de la clave primaria (PK), si es clave

foránea (FK), y una descripción del atributo.

Persona				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
id_persona	INT	X		Identificador interno del individuo dentro del sistema del aeropuerto
nombre	VARCHAR(50)			Nombres de pila del individuo
apellido	VARCHAR(50)			Apellidos paterno y materno del individuo
fecha_nacimiento	DATE			Fecha de nacimiento del individuo

Tabla 1: Tabla Persona

Pasajero				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
Persona.id_persona	INT	X	X	ID del pasajero heredado de la entidad Persona
tipo_documento	VARCHAR(20)			Tipo de documento de identidad (DNI, Pasaporte o Carné de Extranjería). Parte de la clave alternativa única
numero_documento	VARCHAR(15)			Número del documento de identidad. Parte de la clave alternativa única
CategoriaMigratoria.id	INT		X	Categoría migratoria del pasajero que determina la tarifa TUUA

Tabla 2: Tabla Pasajero

CategoriaMigratoria				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
id	INT	X		Identificador único de la categoría migratoria
nombre	VARCHAR(30)			Nombre de la categoría: Nacional, Internacional o Tránsito
tarifa	DECIMAL(10,2)			Tarifa TUUA en soles aplicable a la categoría

Tabla 3: Tabla CategoriaMigratoria

Personal				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
Persona.id_persona	INT	X	X	ID del personal heredado de la entidad Persona

Tabla 4: Tabla Personal

Tripulacion				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
Personal.id_persona	INT	X	X	ID del tripulante heredado de la entidad Personal
num_licencia	VARCHAR(20)			Número de licencia de vuelo emitida por la DGAC (UNIQUE)

Tabla 5: Tabla Tripulacion

OperativoTierra				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
Personal.id_persona	INT	X	X	ID del operativo de tierra heredado de la entidad Personal
area_operativa	VARCHAR(30)			Área operativa asignada (rampa, equipajes, seguridad, etc.)

Tabla 6: Tabla OperativoTierra

Equipaje				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
id	VARCHAR(20)	X		Tag ID impreso sobre la etiqueta de la maleta por el sistema BHS
peso	FLOAT			Peso del equipaje registrado en kilogramos
Pasajero.id_persona	INT		X	ID del pasajero propietario del equipaje
Vuelo.id	INT		X	ID del vuelo en el que será transportado el equipaje

Tabla 7: Tabla Equipaje

Ticket				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
id_ticket	INT	X		Número que identifica al ticket
precio	FLOAT			Precio del ticket en soles
fecha_emision	DATE			Fecha de emisión del ticket
estado_boarding	VARCHAR(20)			Estado del boarding: Emitido, Check-in, Embarcado, No-show o Cancelado
Vuelo.id	INT		X	ID del vuelo al que pertenece el ticket
Pasajero.id_persona	INT		X	ID del pasajero propietario del ticket

Tabla 8: Tabla Ticket

Vuelo				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
id	INT	X		Identificador interno del vuelo
num_vuelo	VARCHAR(10)			Código comercial IATA/OACI del vuelo (ej. LA2477)
hora_programada	TIMESTAMPZ			Fecha y hora de salida programada del vuelo
hora_real	TIMESTAMPZ			Fecha y hora real de salida; NULL mientras no haya despegado
estado	VARCHAR(15)			Estado operativo: Programado, Embarcando, Despegado, Aterrizado, Retrasado o Cancelado
tipo	VARCHAR(15)			Clasificación del vuelo: Nacional o Internacional
origen	VARCHAR(4)			Código IATA del aeropuerto de origen
destino	VARCHAR(4)			Código IATA del aeropuerto de destino
Aeronave.placa	VARCHAR(10)		X	Matrícula de la aeronave asignada al vuelo
Aerolinea.ruc	VARCHAR(11)		X	RUC de la aerolínea operadora del vuelo

Tabla 9: Tabla Vuelo

Aeronave				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
placa	VARCHAR(10)	X		Matrícula aeronáutica oficial asignada por la DGAC
modelo	VARCHAR(50)			Modelo de la aeronave (ej. Boeing 787-9, Airbus A320neo)
fabricante	VARCHAR(50)			Fabricante de la aeronave (Boeing, Airbus, Embraer, ATR)
capacidad	INT			Capacidad total de pasajeros de la aeronave
clase	VARCHAR(10)			Clase de envergadura OACI de la aeronave (A-F)

Tabla 10: Tabla Aeronave

Asiento				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
codigo	VARCHAR(5)	X		Código del asiento dentro de la aeronave (ej. 12A, 34F)
Aeronave.placa	VARCHAR(10)	X	X	Matrícula de la aeronave a la que pertenece el asiento

Tabla 11: Tabla Asiento

Aerolínea				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
ruc	VARCHAR(11)	X		RUC asignado por SUNAT a la aerolínea
nombre	VARCHAR(50)			Razón comercial de la aerolínea
alianza	VARCHAR(20)			Alianza global: Star Alliance, SkyTeam u Oneworld

Tabla 12: Tabla Aerolínea

Recurso				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
id	INT	X		Identificador único del recurso de infraestructura
nombre_tecnico_locacion	VARCHAR(100)			Nombre técnico de la locación dentro del terminal

Tabla 13: Tabla Recurso

Manga				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
Recurso.id	INT	X	X	ID de la manga heredado de la entidad Recurso
estado_acople	VARCHAR(20)			Estado operativo: Libre, Ocupado o Mantenimiento
longitud	FLOAT			Longitud máxima de extensión de la manga en metros
clase_max	VARCHAR(10)			Clase máxima de aeronave (OACI A-F) admitida por la manga

Tabla 14: Tabla Manga

Radar				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
Recurso.id	INT	X	X	ID del radar heredado de la entidad Recurso
rango_alcance	INT			Rango de alcance del radar en millas náuticas
frecuencia	VARCHAR(20)			Frecuencia de operación del radar (banda S, banda L, etc.)

Tabla 15: Tabla Radar

Incidencia				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
id	INT	X		Identificador único de la incidencia
gravedad	VARCHAR(15)			Severidad de la incidencia: Leve, Moderada, Alta o Crítica
descripcion	TEXT			Descripción narrativa del evento reportado
tipo_incidencia	VARCHAR(20)			Tipo de incidencia: Falla_Radar, Inundacion, Falta_Combustible, Saturacion_Vial, Manga_Inoperativa u Otro
fecha_reporte	TIMESTAMP TZ			Fecha y hora en que se reportó la incidencia
fecha_cierre	TIMESTAMP TZ			Fecha y hora de cierre de la incidencia; NULL si permanece abierta

Tabla 16: Tabla Incidencia

Utiliza				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
Vuelo.id	INT	X	X	ID del vuelo que utiliza el recurso
Recurso.id	INT	X	X	ID del recurso asignado al vuelo

Tabla 17: Tabla Utiliza

Retrasa				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
Incidencia.id	INT	X	X	ID de la incidencia que ocasiona el retraso
Vuelo.id	INT	X	X	ID del vuelo afectado por la demora

Tabla 18: Tabla Retrasa

Afecto				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
Incidencia.id	INT	X	X	ID de la incidencia reportada sobre el recurso
Recurso.id	INT	X	X	ID del recurso de infraestructura afectado

Tabla 19: Tabla Afecto

OperaTripulacion				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
Tripulacion.id_persona	INT	X	X	ID del tripulante que opera el vuelo
Vuelo.id	INT	X	X	ID del vuelo operado por el tripulante

Tabla 20: Tabla OperaTripulacion

CheckIn				
Nombre Campo	Tipo de Dato	PK	FK	Descripción
Ticket.id_ticket	BIGINT	X	X	ID del ticket al que pertenece el check-in (heredado de Ticket)
fecha_hora	TIMESTAMPTZ			Fecha y hora en que el pasajero realizó el check-in
counter	VARCHAR(10)			Mostrador de facturación donde se efectuó el check-in
con equipaje	BOOLEAN			Indica si el pasajero registró equipaje en el check-in

Tabla 21: Tabla CheckIn

4. Implementación de la Base de Datos

4.1. Creación de la Base de Datos

```

1  -- =====
2  -- PASO 1 - ESTRUCTURA COMPLETA (4.1)
3  -- =====
4  DROP SCHEMA public CASCADE;
5  CREATE SCHEMA public;
6  -- TIPOS ENUMERADOS
7  CREATE TYPE tipo_doc_t AS ENUM ('DNI', 'Pasaporte', 'Carnet de Extranjeria');
8  CREATE TYPE alianza_t AS ENUM ('Star Alliance', 'SkyTeam', 'Oneworld', 'Ninguna');
9
10 CREATE TYPE estado_acople_t AS ENUM ('Libre', 'Ocupado', 'Mantenimiento');
11 CREATE TYPE estado_boarding_t AS ENUM ('Emitido', 'Check-in', 'Embarcado', 'No-
    ↳ show', 'Cancelado');
12 CREATE TYPE estado_vuelo_t AS ENUM ('Programado', 'Embarcando', 'Despegado', '
    ↳ Aterrizado',
13 'Retrasado', 'Cancelado');
14 CREATE TYPE tipo_vuelo_t AS ENUM ('Nacional', 'Internacional');
15 CREATE TYPE gravedad_t AS ENUM ('Leve', 'Moderada', 'Alta', 'Critica');
16
17 CREATE TYPE tipo_incidencia_t AS ENUM
18 ('Falla_Radar', 'Inundacion', 'Falta_Combustible', 'Saturacion_Vial', '
    ↳ Manga_Inoperativa', 'Otro');
19
20 CREATE TYPE clase_oaci_t AS ENUM ('A', 'B', 'C', 'D', 'E', 'F');
21
22

```

```
23
24
25  -- TABLAS
26
27  CREATE TABLE Persona (
28
29  id_persona SERIAL, nombre VARCHAR(50) NOT NULL,
30
31  apellido VARCHAR(50) NOT NULL, fecha_nacimiento DATE NOT NULL );
32
33  CREATE TABLE CategoriaMigratoria (
34
35  id SERIAL, nombre VARCHAR(30) NOT NULL, tarifa DECIMAL(10,2) NOT NULL );
36
37  CREATE TABLE Aerolinea (
38
39  ruc VARCHAR(11), nombre VARCHAR(50) NOT NULL, alianza alianza_t NOT NULL );
40
41  CREATE TABLE Aeronave (
42
43  placa VARCHAR(11), modelo VARCHAR(50) NOT NULL, fabricante VARCHAR(50) NOT NULL,
44
45  capacidad INT NOT NULL, clase clase_oaci_t NOT NULL );
46
47  CREATE TABLE Recurso (
48
49  id SERIAL, nombre_tecnico_locacion VARCHAR(100) NOT NULL );
50
51  CREATE TABLE Pasajero (
52
53  id_persona INT, tipo_documento tipo_doc_t NOT NULL,
54
55  numero_documento VARCHAR(15) NOT NULL, id_categoria INT NOT NULL );
56
57  CREATE TABLE Personal ( id_persona INT );
58
59  CREATE TABLE Tripulacion ( id_persona INT, num_licencia VARCHAR(20) NOT NULL );
60
61  CREATE TABLE OperativoTierra ( id_persona INT, area_operativa VARCHAR(30) NOT
    ↪ NULL );
62
63  CREATE TABLE Manga (
64
65  id_recurso INT, estado_acople estado_acople_t NOT NULL DEFAULT 'Libre',
66
67  longitud NUMERIC(6,2) NOT NULL, clase_max clase_oaci_t NOT NULL );
68
69  CREATE TABLE Radar (
70
```



```

71 id_recurso INT, rango_alcance INT NOT NULL, frecuencia VARCHAR(20) NOT NULL );
72
73 CREATE TABLE CheckIn (
74
75 id_ticket BIGINT,
76
77 fecha_hora TIMESTAMPTZ NOT NULL DEFAULT NOW(),
78
79 counter VARCHAR(10),
80
81 con equipaje BOOLEAN NOT NULL DEFAULT FALSE );
82
83 CREATE TABLE Vuelo (
84
85 id BIGSERIAL, num_vuelo VARCHAR(10) NOT NULL,
86
87 hora_programada TIMESTAMPTZ NOT NULL, hora_real TIMESTAMPTZ,
88
89 estado estado_vuelo_t NOT NULL DEFAULT 'Programado', tipo tipo_vuelo_t NOT NULL,
90
91 origen VARCHAR(4), destino VARCHAR(4),
92
93 placa_aeronave VARCHAR(11) NOT NULL, ruc_aerolinea VARCHAR(11) NOT NULL );
94
95 CREATE TABLE Asiento ( codigo VARCHAR(5) NOT NULL, placa_aeronave VARCHAR(11) NOT
    ↪ NULL );
96
97 CREATE TABLE Ticket (
98
99 id_ticket BIGSERIAL, precio NUMERIC(10,2) NOT NULL, fecha_emision DATE NOT NULL,
100
101 estado_boarding estado_boarding_t NOT NULL DEFAULT 'Emitido',
102
103 id_vuelo BIGINT NOT NULL, id_persona INT NOT NULL );
104
105 CREATE TABLE Equipaje (
106
107 id VARCHAR(20), peso NUMERIC(6,2) NOT NULL, id_persona INT NOT NULL, id_vuelo
    ↪ BIGINT NOT NULL );
108
109 CREATE TABLE Incidencia (
110
111 id BIGSERIAL, gravedad gravedad_t NOT NULL, descripcion TEXT NOT NULL,
112
113 tipo_incidencia tipo_incidencia_t NOT NULL,
114
115 fecha_reporte TIMESTAMPTZ NOT NULL DEFAULT NOW(), fecha_cierre TIMESTAMPTZ );
116
117 CREATE TABLE Utiliza ( id_vuelo BIGINT NOT NULL, id_recurso INT NOT NULL );

```

```

118
119 CREATE TABLE Retrasa ( id_incidencia BIGINT NOT NULL, id_vuelo BIGINT NOT NULL );
120
121 CREATE TABLE Afecto ( id_incidencia BIGINT NOT NULL, id_recurso INT NOT NULL );
122
123 CREATE TABLE OperaTripulacion ( id_persona INT NOT NULL, id_vuelo BIGINT NOT NULL
    ↪ );
124
125
126
127
128 -- RESTRICCIONES (PK, FK, CHECK)
129
130 ALTER TABLE Persona ADD CONSTRAINT persona_pk PRIMARY KEY (id_persona);
131
132 ALTER TABLE CheckIn ADD CONSTRAINT checkin_pk PRIMARY KEY (id_ticket);
133
134
135 ALTER TABLE CheckIn ADD CONSTRAINT checkin_ticket_fk FOREIGN KEY (id_ticket)
    ↪ REFERENCES
136 Ticket(id_ticket) ON DELETE CASCADE ON UPDATE CASCADE;
137
138 ALTER TABLE CategoriaMigratoria ADD CONSTRAINT categoria_pk PRIMARY KEY (id);
139
140 ALTER TABLE CategoriaMigratoria ADD CONSTRAINT tarifa_no_neg CHECK (tarifa >= 0);
141
142 ALTER TABLE Aerolinea ADD CONSTRAINT aerolinea_pk PRIMARY KEY (ruc);
143
144 ALTER TABLE Aerolinea ADD CONSTRAINT ruc_formato CHECK (ruc ~ '^[0-9]{11}$');
145
146 ALTER TABLE Aeronave ADD CONSTRAINT aeronave_pk PRIMARY KEY (placa);
147
148 ALTER TABLE Aeronave ADD CONSTRAINT capacidad_pos CHECK (capacidad > 0);
149
150 ALTER TABLE Recurso ADD CONSTRAINT recurso_pk PRIMARY KEY (id);
151
152 ALTER TABLE Pasajero ADD CONSTRAINT pasajero_pk PRIMARY KEY (id_persona);
153
154 ALTER TABLE Pasajero ADD CONSTRAINT pasajero_doc_uk UNIQUE (tipo_documento,
155 numero_documento);
156
157 ALTER TABLE Pasajero ADD CONSTRAINT pasajero_persona_fk FOREIGN KEY (id_persona)
158 REFERENCES Persona(id_persona) ON DELETE CASCADE ON UPDATE CASCADE;
159
160 ALTER TABLE Pasajero ADD CONSTRAINT pasajero_categoria_fk FOREIGN KEY (
    ↪ id_categoria)
161 REFERENCES CategoriaMigratoria(id) ON DELETE RESTRICT ON UPDATE CASCADE;
162
163 ALTER TABLE Personal ADD CONSTRAINT personal_pk PRIMARY KEY (id_persona);

```

```
164
165 ALTER TABLE Personal ADD CONSTRAINT personal_persona_fk FOREIGN KEY (id_persona)
166 REFERENCES Persona(id_persona) ON DELETE CASCADE ON UPDATE CASCADE;
167
168 ALTER TABLE Tripulacion ADD CONSTRAINT tripulacion_pk PRIMARY KEY (id_persona);
169
170 ALTER TABLE Tripulacion ADD CONSTRAINT tripulacion_lic_uk UNIQUE (num_licencia);
171
172 ALTER TABLE Tripulacion ADD CONSTRAINT tripulacion_personal_fk FOREIGN KEY (
    ↳ id_persona)
173 REFERENCES Personal(id_persona) ON DELETE CASCADE ON UPDATE CASCADE;
174
175 ALTER TABLE OperativoTierra ADD CONSTRAINT operativo_pk PRIMARY KEY (id_persona);
176
177 ALTER TABLE OperativoTierra ADD CONSTRAINT operativo_personal_fk FOREIGN KEY (
    ↳ id_persona)
178 REFERENCES Personal(id_persona) ON DELETE CASCADE ON UPDATE CASCADE;
179
180 ALTER TABLE Manga ADD CONSTRAINT manga_pk PRIMARY KEY (id_recurso);
181
182 ALTER TABLE Manga ADD CONSTRAINT longitud_pos CHECK (longitud > 0);
183
184 ALTER TABLE Manga ADD CONSTRAINT manga_recurso_fk FOREIGN KEY (id_recurso)
    ↳ REFERENCES
185 Recurso(id) ON DELETE CASCADE ON UPDATE CASCADE;
186
187 ALTER TABLE Radar ADD CONSTRAINT radar_pk PRIMARY KEY (id_recurso);
188
189 ALTER TABLE Radar ADD CONSTRAINT rango_pos CHECK (rango_alcance > 0);
190
191 ALTER TABLE Radar ADD CONSTRAINT radar_recurso_fk FOREIGN KEY (id_recurso)
    ↳ REFERENCES
192 Recurso(id) ON DELETE CASCADE ON UPDATE CASCADE;
193
194
195 ALTER TABLE Vuelo ADD CONSTRAINT vuelo_pk PRIMARY KEY (id);
196
197 ALTER TABLE Vuelo ADD CONSTRAINT vuelo_horas_ok CHECK (hora_real IS NULL OR
    ↳ hora_real >=
198 hora_programada);
199
200 ALTER TABLE Vuelo ADD CONSTRAINT vuelo_aeronave_fk FOREIGN KEY (placa_aeronave)
201 REFERENCES Aeronave(placa) ON DELETE RESTRICT ON UPDATE CASCADE;
202
203 ALTER TABLE Vuelo ADD CONSTRAINT vuelo_aerolinea_fk FOREIGN KEY (ruc_aerolinea)
    ↳ REFERENCES
204 Aerolinea(ruc) ON DELETE RESTRICT ON UPDATE CASCADE;
205
```

```
206 ALTER TABLE Asiento ADD CONSTRAINT asiento_pk PRIMARY KEY (codigo, placa_aeronave
    ↪ );
207
208 ALTER TABLE Asiento ADD CONSTRAINT asiento_aeronave_fk FOREIGN KEY (
    ↪ placa_aeronave)
209 REFERENCES Aeronave(placa) ON DELETE CASCADE ON UPDATE CASCADE;
210
211 ALTER TABLE Ticket ADD CONSTRAINT ticket_pk PRIMARY KEY (id_ticket);
212
213 ALTER TABLE Ticket ADD CONSTRAINT precio_pos CHECK (precio >= 0);
214
215 ALTER TABLE Ticket ADD CONSTRAINT ticket_vuelo_fk FOREIGN KEY (id_vuelo)
    ↪ REFERENCES Vuelo(id)
216 ON DELETE RESTRICT ON UPDATE CASCADE;
217
218 ALTER TABLE Ticket ADD CONSTRAINT ticket_pasajero_fk FOREIGN KEY (id_persona)
    ↪ REFERENCES
219 Pasajero(id_persona) ON DELETE RESTRICT ON UPDATE CASCADE;
220
221 ALTER TABLE Equipaje ADD CONSTRAINT equipaje_pk PRIMARY KEY (id);
222
223 ALTER TABLE Equipaje ADD CONSTRAINT peso_pos CHECK (peso > 0);
224
225 ALTER TABLE Equipaje ADD CONSTRAINT equipaje_pasajero_fk FOREIGN KEY (id_persona)
226 REFERENCES Pasajero(id_persona) ON DELETE RESTRICT ON UPDATE CASCADE;
227
228 ALTER TABLE Equipaje ADD CONSTRAINT equipaje_vuelo_fk FOREIGN KEY (id_vuelo)
    ↪ REFERENCES
229 Vuelo(id) ON DELETE RESTRICT ON UPDATE CASCADE;
230
231 ALTER TABLE Incidencia ADD CONSTRAINT incidencia_pk PRIMARY KEY (id);
232
233 ALTER TABLE Incidencia ADD CONSTRAINT cierre_post CHECK (fecha_cierre IS NULL OR
    ↪ fecha_cierre >
234 fecha_reporte);
235
236 ALTER TABLE Utiliza ADD CONSTRAINT utiliza_pk PRIMARY KEY (id_vuelo, id_recurso);
237
238 ALTER TABLE Utiliza ADD CONSTRAINT utiliza_vuelo_fk FOREIGN KEY (id_vuelo)
    ↪ REFERENCES
239 Vuelo(id) ON DELETE CASCADE ON UPDATE CASCADE;
240
241 ALTER TABLE Utiliza ADD CONSTRAINT utiliza_recurso_fk FOREIGN KEY (id_recurso)
    ↪ REFERENCES
242 Recurso(id) ON DELETE RESTRICT ON UPDATE CASCADE;
243
244 ALTER TABLE Retrasa ADD CONSTRAINT retrasa_pk PRIMARY KEY (id_incidencia,
    ↪ id_vuelo);
245
```

```

246 ALTER TABLE Retrasa ADD CONSTRAINT retrasa_incidencia_fk FOREIGN KEY (
    ↪ id_incidencia)
247 REFERENCES Incidencia(id) ON DELETE CASCADE ON UPDATE CASCADE;
248
249 ALTER TABLE Retrasa ADD CONSTRAINT retrasa_vuelo_fk FOREIGN KEY (id_vuelo)
    ↪ REFERENCES
250 Vuelo(id) ON DELETE CASCADE ON UPDATE CASCADE;
251
252 ALTER TABLE Afecto ADD CONSTRAINT afecto_pk PRIMARY KEY (id_incidencia,
    ↪ id_recurso);
253
254
255 ALTER TABLE Afecto ADD CONSTRAINT afecto_incidencia_fk FOREIGN KEY (id_incidencia
    ↪ )
256 REFERENCES Incidencia(id) ON DELETE CASCADE ON UPDATE CASCADE;
257
258 ALTER TABLE Afecto ADD CONSTRAINT afecto_recurso_fk FOREIGN KEY (id_recurso)
    ↪ REFERENCES
259 Recurso(id) ON DELETE RESTRICT ON UPDATE CASCADE;
260
261 ALTER TABLE OperaTripulacion ADD CONSTRAINT opera_pk PRIMARY KEY (id_persona,
    ↪ id_vuelo);
262
263 ALTER TABLE OperaTripulacion ADD CONSTRAINT opera_trip_fk FOREIGN KEY (id_persona
    ↪ )
264 REFERENCES Tripulacion(id_persona) ON DELETE RESTRICT ON UPDATE CASCADE;
265
266 ALTER TABLE OperaTripulacion ADD CONSTRAINT opera_vuelo_fk FOREIGN KEY (id_vuelo)
267 REFERENCES Vuelo(id) ON DELETE CASCADE ON UPDATE CASCADE;

```

4.2. Carga de Datos

Dado que la información operativa real del Aeropuerto Internacional Jorge Chávez es de carácter confidencial y no es de acceso público, la base de datos se pobló combinando datos reales para los catálogos y datos sintéticos para la información masiva y sensible (personas, documentos, tickets, equipaje e incidencias), conforme a lo permitido por el enunciado del proyecto.

Las tablas de catálogo se cargaron con información real verificable, adaptada de fuentes públicas (portal de Lima Airport Partners, registros de la terminal 2025–2026 y el anexo de aerolíneas y destinos del aeropuerto):

- **CategoriaMigratoria:** las tres categorías oficiales (Nacional, Internacional y Tránsito) con sus tarifas TUUA correspondientes.
- **Aerolinea:** las 24 aerolíneas que operan actualmente en el terminal (LATAM, Sky Airline, JetSMART, Avianca, Copa, Iberia, KLM, Air France, LEVEL, entre otras), cada una con su alianza global real (Star Alliance, SkyTeam, Oneworld) o sin alianza cuando corresponde. Los números de RUC son

sintéticos por ser información tributaria reservada.

- Aeronave: una flota construida a partir de los modelos reales que operan en LIM (Airbus A320neo, A350-900, Boeing 787-9, 777-300ER, Embraer E190, ATR 72-600, etc.), cada uno con su capacidad y su clase de envergadura OACI real.
- Recurso (Manga / Radar): la infraestructura física del terminal, modelada según las cantidades aproximadas de puentes de embarque y radares operativos. La carga de estos catálogos se realizó mediante sentencias INSERT en SQL ejecutadas en PostgreSQL. Para volúmenes mayores de datos de catálogo se contempla el uso del comando COPY desde archivos CSV.

```
1  -- CATEGORÍAS MIGRATORIAS
2  INSERT INTO CategoriaMigratoria (nombre, tarifa) VALUES
3  ('Nacional', 12.50),
4  ('Internacional', 38.45),
5  ('Transito', 18.00);
6
7
8  INSERT INTO Aerolinea (ruc, nombre, alianza) VALUES
9
10 ('20100000001', 'LATAM Airlines Peru', 'Ninguna'),
11
12 ('20100000002', 'Sky Airline Peru', 'Ninguna'),
13
14 ('20100000003', 'JetSMART', 'Ninguna'),
15
16 ('20100000004', 'Star Peru', 'Ninguna'),
17
18 ('20100000005', 'Avianca', 'Star Alliance'),
19
20 ('20100000006', 'Copa Airlines', 'Star Alliance'),
21
22 ('20100000007', 'Air Canada', 'Star Alliance'),
23
24 ('20100000008', 'United Airlines', 'Star Alliance'),
25
26 ('20100000009', 'Aeromexico', 'SkyTeam'),
27
28 ('20100000010', 'Air France', 'SkyTeam'),
29
30 ('20100000011', 'KLM Royal Dutch Airlines', 'SkyTeam'),
31
32 ('20100000012', 'Delta Air Lines', 'SkyTeam'),
33
34 ('20100000013', 'Aerolineas Argentinas', 'SkyTeam'),
35
36 ('20100000014', 'Iberia', 'Oneworld'),
37
38 ('20100000015', 'American Airlines', 'Oneworld'),
```

```

39
40 ('20100000016', 'British Airways', 'Oneworld'),
41
42 ('20100000017', 'Air Europa', 'SkyTeam'),
43
44 ('20100000018', 'Plus Ultra Lineas Aereas', 'Ninguna'),
45
46 ('20100000019', 'Arajet', 'Ninguna'),
47
48 ('20100000020', 'Wingo', 'Ninguna'),
49
50 ('20100000021', 'LEVEL', 'Ninguna'),
51
52 ('20100000022', 'GOL Linhas Aereas', 'Ninguna'),
53
54 ('20100000023', 'Boliviana de Aviacion', 'Ninguna'),
55
56 ('20100000024', 'Atsa Airlines', 'Ninguna');
57
58
59
60 -- AERONAVES (modelos reales) - 500
61
62
63 INSERT INTO Aeronave (placa, modelo, fabricante, capacidad, clase)
64
65 SELECT
66
67 'OB-' || LPAD(g::text, 4, '0') || (ARRAY['', 'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'J'])
    ↪ [1 + (g % 10)],
68
69 m.modelo, m.fabricante, m.capacidad, m.clase::clase_oaci_t
70
71 FROM generate_series(1, 500) AS g
72
73 CROSS JOIN LATERAL (
74
75 SELECT * FROM (VALUES
76
77 ('Airbus A320neo', 'Airbus', 188, 'C'), ('Airbus A321neo', 'Airbus', 220, 'C'),
78
79 ('Airbus A319', 'Airbus', 144, 'C'), ('Boeing 737-800', 'Boeing', 189, 'C'),
80
81 ('Boeing 787-9', 'Boeing', 296, 'E'), ('Boeing 777-300ER', 'Boeing', 396, 'E'),
82
83 ('Airbus A330-200', 'Airbus', 247, 'E'), ('Airbus A350-900', 'Airbus', 348, 'F'),
84
85 ('Embraer E190', 'Embraer', 100, 'C'), ('ATR 72-600', 'ATR', 70, 'B')
86

```

```

87 ) AS t(modelo,fabricante,capacidad,clase)
88
89 OFFSET (g % 10) LIMIT 1
90
91 ) AS m;
92
93
94
95 -- RECURSOS (200) + Manga/Radar
96
97 INSERT INTO Recurso (nombre_tecnico_locacion)
98
99 SELECT 'LOC-' || LPAD(g::text, 4, '0') FROM generate_series(1, 200) AS g;
100
101
102
103 INSERT INTO Manga (id_recurso, estado_acople, longitud, clase_max)
104
105 SELECT id, (ARRAY['Libre','Ocupado','Mantenimiento'])[1+(id%3)::estado_acople_t,
106
107 20 + (id % 25), (ARRAY['C','D','E','F'])[1+(id%4)::clase_oaci_t
108
109 FROM Recurso WHERE id % 2 = 0;
110
111
112
113 INSERT INTO Radar (id_recurso, rango_alcance, frecuencia)
114
115 SELECT id, 60 + (id % 200), (ARRAY['Banda S','Banda L','Banda X'])[1+(id%3)]
116
117
118 FROM Recurso WHERE id % 2 = 1;

```

4.3. Simulación de Datos

La información masiva y de naturaleza confidencial se generó de forma sintética respetando todas las restricciones de integridad del modelo (claves primarias, foráneas y CHECK). Se emplearon dos técnicas complementarias:

1. Generación mediante generate_series de PostgreSQL: se utilizó esta función para producir grandes volúmenes de tuplas de manera programática y reproducible, respetando el orden de dependencias entre tablas (primero las entidades fuertes, luego las subclases y finalmente las relaciones). Los valores descriptivos se obtuvieron combinando arreglos de valores válidos con operaciones aritméticas sobre el contador, garantizando distribución y unicidad donde es requerida.
2. Mock Data de DBEaver: de forma complementaria se empleó la funcionalidad Mock Data de DBEaver para enriquecer ciertos campos de texto, aprovechando que respeta las restricciones de

clave foránea definidas. El orden de poblamiento fue: Persona → (Pasajero / Personal → Tripulación / Operativo-Tierra) → Vuelo → Ticket y Equipaje → Incidencia → tablas de unión (Utiliza, Retrasa, Afecto, OperaTripulacion). La tabla Incidencia se definió como la de mayor volumen, escalando hasta 1 000 000 de registros para la experimentación, coherente con la narrativa de crisis de infraestructura del terminal. Tras cada carga se ejecutó ANALYZE para actualizar las estadísticas del planificador de consultas. La experimentación se realizó sobre cuatro escenarios de volumen (1 000, 10 000, 100 000 y 1 000 000 de registros en la tabla protagonista), generando un dump independiente para cada uno. A continuación se presenta el script de generación para cada escenario; los catálogos de la sección 4.2 se mantienen idénticos en todos.

4.3.1. Escenario 1 000 registros

```

1  -- PERSONA (2,000)
2
3  INSERT INTO Persona (nombre, apellido, fecha_nacimiento)
4
5  SELECT
6  (ARRAY['Juan','Maria','Carlos','Lucia','Pedro','Ana','Jose','Rosa','Luis','Carmen
   ↪ '])[1+(g%10)],
7
8
9  (ARRAY['Quispe','Garcia','Flores','Rojas','Torres','Vargas','Diaz','Mamani','Cruz
   ↪ ','Leon'])[1+(g%10
10 )],
11
12 DATE '1960-01-01' + ((g*7) % 20000)
13
14 FROM generate_series(1, 2000) AS g;
15
16
17 -- PASAJERO (1,500) y PERSONAL (500)
18
19 INSERT INTO Pasajero (id_persona, tipo_documento, numero_documento, id_categoria)
20
21 SELECT id_persona, (ARRAY['DNI','Pasaporte','Carnet de
22 Extranjeria'])[1+(id_persona%3)::tipo_doc_t,
23
24 LPAD(id_persona::text, 12, '0'), 1 + (id_persona % 3)
25
26 FROM Persona WHERE id_persona <= 1500;
27
28 INSERT INTO Personal (id_persona)
29
30 SELECT id_persona FROM Persona WHERE id_persona BETWEEN 1501 AND 2000;
31
32 INSERT INTO Tripulacion (id_persona, num_licencia)
33

```

```

34 SELECT id_persona, 'LIC-' || LPAD(id_persona::text, 8, '0') FROM Personal WHERE
    ⇨ id_persona
35 % 2 = 0;
36
37 INSERT INTO OperativoTierra (id_persona, area_operativa)
38
39 SELECT id_persona,
40 (ARRAY['Rampa','Equipajes','Seguridad','Migraciones','Mantenimiento'])[1+(
    ⇨ id_persona%5)]
41
42 FROM Personal WHERE id_persona % 2 = 1;
43
44
45
46 -- VUELO (200)
47
48 INSERT INTO Vuelo (num_vuelo, hora_programada, hora_real, estado, tipo, origen,
    ⇨ destino,
49 placa_aeronave, ruc_aerolinea)
50
51 SELECT (ARRAY['LA','H2','JA','2I','AV','CM'])[1+(g%6)] || (1000 + (g%9000)),
52
53 TIMESTAMPTZ '2026-01-01 00:00:00-05' + (g%180)*INTERVAL '1 day' + (g%24)*INTERVAL
    ⇨ '1
54 hour',
55
56 CASE WHEN g%10<7 THEN TIMESTAMPTZ '2026-01-01 00:00:00-05' + (g%180)*INTERVAL
57 '1 day'
58
59 + (g%24)*INTERVAL '1 hour' + (g%120)*INTERVAL '1 minute' ELSE NULL END,
60
61
62 (ARRAY['Aterrizado','Despegado','Programado','Retrasado','Cancelado'])[1+(g%5)]::
    ⇨ estado_vuelo_t,
63
64 CASE WHEN g%2=0 THEN 'Nacional' ELSE 'Internacional' END::tipo_vuelo_t, 'LIM',
65
66 CASE WHEN g%2=0 THEN
67 (ARRAY['CUZ','AQP','IQT','TPP','PCL','TRU','JUL','PIU','PEM','TBP'])[1+(g%10)]
68
69
70 ELSE (ARRAY['BOG','SCL','MIA','GRU','MAD','MEX','PTY','JFK','BCN','EZE'])[1+(g
    ⇨ %10)]
71 END,
72
73 (SELECT placa FROM Aeronave OFFSET (g % 500) LIMIT 1),
74
75 (SELECT ruc FROM Aerolinea OFFSET (g % 24) LIMIT 1)
76

```

```

77 FROM generate_series(1, 200) AS g;
78
79
80
81 -- TICKET (1,000) y EQUIPAJE (800)
82
83 INSERT INTO Ticket (precio, fecha_emision, estado_boarding, id_vuelo, id_persona)
84
85 SELECT 50 + (g%1500), DATE '2026-01-01' + (g%180),
86
87
88 (ARRAY['Emitido', 'Check-in', 'Embarcado', 'No-show', 'Cancelado'])[1+(g%5)]::
    ↪ estado_boarding_t,
89
90 1 + (g%200), 1 + (g%1500)
91
92 FROM generate_series(1, 1000) AS g;
93
94 INSERT INTO Equipaje (id, peso, id_persona, id_vuelo)
95
96 SELECT 'TAG-' || LPAD(g::text,10,'0'), 5 + (g%28), 1 + (g%1500), 1 + (g%200)
97
98 FROM generate_series(1, 800) AS g;
99
100
101
102 -- CHECK-IN (700)
103
104 INSERT INTO CheckIn (id_ticket, fecha_hora, counter, con equipaje)
105
106 SELECT g, NOW() - ((g%180)*INTERVAL '1 day') - ((g%12)*INTERVAL '1 hour'),
107
108 'C' || LPAD((1 + (g%40))::text, 2, '0'), (g%2 = 0)
109
110 FROM generate_series(1, 700) AS g;
111
112
113
114 -- INCIDENCIA (1,000)
115
116 INSERT INTO Incidencia (gravedad, descripcion, tipo_incidencia, fecha_reporte,
    ↪ fecha_cierre)
117
118 SELECT (ARRAY['Leve', 'Moderada', 'Alta', 'Critica'])[1+(g%4)]::gravedad_t,
119
120 'Incidencia sintetica #' || g,
121
122 (ARRAY['Falla_Radar', 'Inundacion', 'Falta_Combustible', 'Saturacion_Vial',
    ↪ Manga_Inoperativa', 'Otro'])[1+(g%6)]::tipo_incidencia_t,

```

```

123
124 NOW() - ((g%365)*INTERVAL '1 day') - ((g%24)*INTERVAL '1 hour'),
125
126 CASE WHEN g%3=0 THEN NOW() - ((g%365)*INTERVAL '1 day') + INTERVAL '2 hour'
127 ELSE NULL END
128
129 FROM generate_series(1, 1000) AS g;
130
131
132
133 -- TABLAS DE UNIÓN
134
135 INSERT INTO Utiliza (id_vuelo, id_recurso)
136
137 SELECT v, 1 + ((v*k) % 200) FROM generate_series(1,200) AS v, generate_series
    ↪ (1,2) AS k
138 ON CONFLICT DO NOTHING;
139
140 INSERT INTO Afecto (id_incidencia, id_recurso)
141
142 SELECT id, 1 + (id % 200) FROM Incidencia WHERE id % 2 = 0 ON CONFLICT DO NOTHING
    ↪ ;
143
144 INSERT INTO Retrasa (id_incidencia, id_vuelo)
145
146 SELECT id, 1 + (id % 200) FROM Incidencia WHERE id % 2 = 1 ON CONFLICT DO NOTHING
    ↪ ;
147
148 INSERT INTO OperaTripulacion (id_persona, id_vuelo)
149
150 SELECT t.id_persona, v FROM generate_series(1,200) AS v
151
152 CROSS JOIN LATERAL (SELECT id_persona FROM Tripulacion ORDER BY (id_persona+v)
153 LIMIT 2) AS t ON CONFLICT DO NOTHING;
154
155
156
157 ANALYZE;

```

4.3.2. Escenario 10 000 registros

```

1 -- PERSONA (20,000)
2
3 INSERT INTO Persona (nombre, apellido, fecha_nacimiento)
4
5 SELECT

```

```

6  (ARRAY['Juan','Maria','Carlos','Lucia','Pedro','Ana','Jose','Rosa','Luis','Carmen
    ↪ '])[1+(g%10)],
7
8
9  (ARRAY['Quispe','Garcia','Flores','Rojas','Torres','Vargas','Diaz','Mamani','Cruz
    ↪ ','Leon'])[1+(g%10
10 )],
11
12 DATE '1960-01-01' + ((g*7) % 20000)
13
14 FROM generate_series(1, 20000) AS g;
15
16
17
18 -- PASAJERO (15,000) y PERSONAL (5,000)
19
20 INSERT INTO Pasajero (id_persona, tipo_documento, numero_documento, id_categoria)
21
22 SELECT id_persona, (ARRAY['DNI','Pasaporte','Carnet de
23 Extranjeria'])[1+(id_persona%3)]::tipo_doc_t,
24
25 LPAD(id_persona::text, 12, '0'), 1 + (id_persona % 3)
26
27 FROM Persona WHERE id_persona <= 15000;
28
29 INSERT INTO Personal (id_persona)
30
31 SELECT id_persona FROM Persona WHERE id_persona BETWEEN 15001 AND 20000;
32
33 INSERT INTO Tripulacion (id_persona, num_licencia)
34
35 SELECT id_persona, 'LIC-' || LPAD(id_persona::text, 8, '0') FROM Personal WHERE
    ↪ id_persona
36 % 2 = 0;
37
38 INSERT INTO OperativoTierra (id_persona, area_operativa)
39
40 SELECT id_persona,
41 (ARRAY['Rampa','Equipajes','Seguridad','Migraciones','Mantenimiento'])[1+(
    ↪ id_persona%5)]
42
43 FROM Personal WHERE id_persona % 2 = 1;
44
45
46
47 -- VUELO (2,000)
48
49 INSERT INTO Vuelo (num_vuelo, hora_programada, hora_real, estado, tipo, origen,
    ↪ destino,

```

```

50  placa_aeronave, ruc_aerolinea)
51
52  SELECT (ARRAY['LA','H2','JA','2I','AV','CM'])[1+(g%6)] || (1000 + (g%9000)),
53
54  TIMESTAMPTZ '2026-01-01 00:00:00-05' + (g%180)*INTERVAL '1 day' + (g%24)*INTERVAL
    ↪ '1
55  hour',
56
57  CASE WHEN g%10<7 THEN TIMESTAMPTZ '2026-01-01 00:00:00-05' + (g%180)*INTERVAL
58  '1 day'
59
60  + (g%24)*INTERVAL '1 hour' + (g%120)*INTERVAL '1 minute' ELSE NULL END,
61
62  (ARRAY['Aterrizado','Despegado','Programado','Retrasado','Cancelado'])[1+(g%5)]::
    ↪ estado_vuelo_t,
63
64  CASE WHEN g%2=0 THEN 'Nacional' ELSE 'Internacional' END::tipo_vuelo_t, 'LIM',
65
66  CASE WHEN g%2=0 THEN
67  (ARRAY['CUZ','AQP','IQT','TPP','PCL','TRU','JUL','PIU','PEM','TBP'])[1+(g%10)]
68
69  ELSE (ARRAY['BOG','SCL','MIA','GRU','MAD','MEX','PTY','JFK','BCN','EZE'])[1+(g
    ↪ %10)]
70  END,
71
72  (SELECT placa FROM Aeronave OFFSET (g % 500) LIMIT 1),
73
74  (SELECT ruc FROM Aerolinea OFFSET (g % 24) LIMIT 1)
75
76  FROM generate_series(1, 2000) AS g;
77
78
79
80  -- TICKET (10,000) y EQUIPAJE (8,000)
81
82  INSERT INTO Ticket (precio, fecha_emision, estado_boarding, id_vuelo, id_persona)
83
84  SELECT 50 + (g%1500), DATE '2026-01-01' + (g%180),
85
86
87  (ARRAY['Emitido','Check-in','Embarcado','No-show','Cancelado'])[1+(g%5)]::
    ↪ estado_boarding_t,
88
89  1 + (g%2000), 1 + (g%15000)
90
91  FROM generate_series(1, 10000) AS g;
92
93  INSERT INTO Equipaje (id, peso, id_persona, id_vuelo)
94

```

```

95  SELECT 'TAG-' || LPAD(g::text,10,'0'), 5 + (g%28), 1 + (g%15000), 1 + (g%2000)
96
97  FROM generate_series(1, 8000) AS g;
98
99
100
101  -- CHECK-IN (7,000)
102
103  INSERT INTO CheckIn (id_ticket, fecha_hora, counter, con equipaje)
104
105  SELECT g, NOW() - ((g%180)*INTERVAL '1 day') - ((g%12)*INTERVAL '1 hour'),
106
107  'C' || LPAD((1 + (g%40))::text, 2, '0'), (g%2 = 0)
108
109  FROM generate_series(1, 7000) AS g;
110
111
112  -- INCIDENCIA (10,000)
113
114  INSERT INTO Incidencia (gravedad, descripcion, tipo_incidencia, fecha_reporte,
    ↪ fecha_cierre)
115
116  SELECT (ARRAY['Leve','Moderada','Alta','Critica'])[1+(g%4)]::gravedad_t,
117
118  'Incidencia sintetica #' || g,
119
120
121  (ARRAY['Falla_Radar','Inundacion','Falta_Combustible','Saturacion_Vial','
    ↪ Manga_Inoperativa','Otro'])[1+(g%6)]::tipo_incidencia_t,
122
123  NOW() - ((g%365)*INTERVAL '1 day') - ((g%24)*INTERVAL '1 hour'),
124
125  CASE WHEN g%3=0 THEN NOW() - ((g%365)*INTERVAL '1 day') + INTERVAL '2 hour'
126  ELSE NULL END
127
128  FROM generate_series(1, 10000) AS g;
129
130
131
132  -- TABLAS DE UNIÓN
133
134  INSERT INTO Utiliza (id_vuelo, id_recurso)
135
136  SELECT v, 1 + ((v*k) % 200) FROM generate_series(1,2000) AS v, generate_series
    ↪ (1,2) AS k
137  ON CONFLICT DO NOTHING;
138
139  INSERT INTO Afecto (id_incidencia, id_recurso)
140

```

```

141 SELECT id, 1 + (id % 200) FROM Incidencia WHERE id % 2 = 0 ON CONFLICT DO NOTHING
    ↪ ;
142
143 INSERT INTO Retrasa (id_incidencia, id_vuelo)
144
145 SELECT id, 1 + (id % 2000) FROM Incidencia WHERE id % 2 = 1 ON CONFLICT DO
    ↪ NOTHING;
146
147 INSERT INTO OperaTripulacion (id_persona, id_vuelo)
148
149 SELECT t.id_persona, v FROM generate_series(1,2000) AS v
150
151 CROSS JOIN LATERAL (SELECT id_persona FROM Tripulacion ORDER BY (id_persona+v)
152 LIMIT 2) AS t ON CONFLICT DO NOTHING;
153
154 ANALYZE;

```

4.3.3. Escenario 100 000 registros

```

1  -- PERSONA (200,000)
2
3  INSERT INTO Persona (nombre, apellido, fecha_nacimiento)
4
5
6  SELECT
7  (ARRAY['Juan','Maria','Carlos','Lucia','Pedro','Ana','Jose','Rosa','Luis','Carmen
    ↪ '])[1+(g%10)],
8
9
10 (ARRAY['Quispe','Garcia','Flores','Rojas','Torres','Vargas','Diaz','Mamani','Cruz
    ↪ ','Leon'])[1+(g%10
11 )],
12
13 DATE '1960-01-01' + ((g*7) % 20000)
14
15 FROM generate_series(1, 200000) AS g;
16
17
18
19 -- PASAJERO (150,000) y PERSONAL (50,000)
20
21 INSERT INTO Pasajero (id_persona, tipo_documento, numero_documento, id_categoria)
22
23 SELECT id_persona, (ARRAY['DNI','Pasaporte','Carnet de
24 Extranjeria'])[1+(id_persona%3)::tipo_doc_t,
25
26 LPAD(id_persona::text, 12, '0'), 1 + (id_persona % 3)

```



```

27
28 FROM Persona WHERE id_persona <= 150000;
29
30 INSERT INTO Personal (id_persona)
31
32 SELECT id_persona FROM Persona WHERE id_persona BETWEEN 150001 AND 200000;
33
34 INSERT INTO Tripulacion (id_persona, num_licencia)
35
36 SELECT id_persona, 'LIC-' || LPAD(id_persona::text, 8, '0') FROM Personal WHERE
    ↪ id_persona
37 % 2 = 0;
38
39 INSERT INTO OperativoTierra (id_persona, area_operativa)
40
41 SELECT id_persona,
42 (ARRAY['Rampa', 'Equipajes', 'Seguridad', 'Migraciones', 'Mantenimiento'])[1+(
    ↪ id_persona%5)]
43
44 FROM Personal WHERE id_persona % 2 = 1;
45
46
47
48 -- VUELO (20,000)
49
50 INSERT INTO Vuelo (num_vuelo, hora_programada, hora_real, estado, tipo, origen,
    ↪ destino,
51 placa_aeronave, ruc_aerolinea)
52
53 SELECT (ARRAY['LA', 'H2', 'JA', '2I', 'AV', 'CM'])[1+(g%6)] || (1000 + (g%9000)),
54
55 TIMESTAMPTZ '2026-01-01 00:00:00-05' + (g%180)*INTERVAL '1 day' + (g%24)*INTERVAL
    ↪ '1
56 hour',
57
58
59 CASE WHEN g%10<7 THEN TIMESTAMPTZ '2026-01-01 00:00:00-05' + (g%180)*INTERVAL
60 '1 day'
61
62 + (g%24)*INTERVAL '1 hour' + (g%120)*INTERVAL '1 minute' ELSE NULL END,
63
64
65 (ARRAY['Aterrizado', 'Despegado', 'Programado', 'Retrasado', 'Cancelado'])[1+(g%5)]::
    ↪ estado_vuelo_t,
66
67 CASE WHEN g%2=0 THEN 'Nacional' ELSE 'Internacional' END::tipo_vuelo_t, 'LIM',
68
69 CASE WHEN g%2=0 THEN
70 (ARRAY['CUZ', 'AQP', 'IQT', 'TPP', 'PCL', 'TRU', 'JUL', 'PIU', 'PEM', 'TBP'])[1+(g%10)]

```

```

71
72 ELSE (ARRAY['BOG','SCL','MIA','GRU','MAD','MEX','PTY','JFK','BCN','EZE'])[1+(g
    ↪ %10)]
73 END,
74
75 (SELECT placa FROM Aeronave OFFSET (g % 500) LIMIT 1),
76
77 (SELECT ruc FROM Aerolinea OFFSET (g % 24) LIMIT 1)
78
79 FROM generate_series(1, 20000) AS g;
80
81
82
83 -- TICKET (100,000) y EQUIPAJE (80,000)
84
85 INSERT INTO Ticket (precio, fecha_emision, estado_boarding, id_vuelo, id_persona)
86
87 SELECT 50 + (g%1500), DATE '2026-01-01' + (g%180),
88
89
90 (ARRAY['Emitido','Check-in','Embarcado','No-show','Cancelado'])[1+(g%5)]::
    ↪ estado_boarding_t,
91
92 1 + (g%20000), 1 + (g%150000)
93
94 FROM generate_series(1, 100000) AS g;
95
96 INSERT INTO Equipaje (id, peso, id_persona, id_vuelo)
97
98 SELECT 'TAG-' || LPAD(g::text,10,'0'), 5 + (g%28), 1 + (g%150000), 1 + (g%20000)
99
100 FROM generate_series(1, 80000) AS g;
101
102
103
104 -- CHECK-IN (70,000)
105
106 INSERT INTO CheckIn (id_ticket, fecha_hora, counter, con equipaje)
107
108 SELECT g, NOW() - ((g%180)*INTERVAL '1 day') - ((g%12)*INTERVAL '1 hour'),
109
110 'C' || LPAD((1 + (g%40))::text, 2, '0'), (g%2 = 0)
111
112 FROM generate_series(1, 70000) AS g;
113
114
115
116 -- INCIDENCIA (100,000)
117

```

```

118 INSERT INTO Incidencia (gravedad, descripcion, tipo_incidencia, fecha_reporte,
    ↪ fecha_cierre)
119
120 SELECT (ARRAY['Leve','Moderada','Alta','Critica'])[1+(g%4)::gravedad_t,
121
122 'Incidencia sintetica #' || g,
123
124
125 (ARRAY['Falla_Radar','Inundacion','Falta_Combustible','Saturacion_Vial','
    ↪ Manga_Inoperativa','Otro'])[1+(g%6)::tipo_incidencia_t,
126
127 NOW() - ((g%365)*INTERVAL '1 day') - ((g%24)*INTERVAL '1 hour'),
128
129 CASE WHEN g%3=0 THEN NOW() - ((g%365)*INTERVAL '1 day') + INTERVAL '2 hour'
130 ELSE NULL END
131
132 FROM generate_series(1, 100000) AS g;
133
134
135
136 -- TABLAS DE UNIÓN
137
138 INSERT INTO Utiliza (id_vuelo, id_recurso)
139
140 SELECT v, 1 + ((v*k) % 200) FROM generate_series(1,20000) AS v, generate_series
    ↪ (1,2) AS k
141 ON CONFLICT DO NOTHING;
142
143 INSERT INTO Afecto (id_incidencia, id_recurso)
144
145 SELECT id, 1 + (id % 200) FROM Incidencia WHERE id % 2 = 0 ON CONFLICT DO NOTHING
    ↪ ;
146
147 INSERT INTO Retrasa (id_incidencia, id_vuelo)
148
149 SELECT id, 1 + (id % 20000) FROM Incidencia WHERE id % 2 = 1 ON CONFLICT DO
    ↪ NOTHING;
150
151 INSERT INTO OperaTripulacion (id_persona, id_vuelo)
152
153 SELECT t.id_persona, v FROM generate_series(1,20000) AS v
154
155 CROSS JOIN LATERAL (SELECT id_persona FROM Tripulacion ORDER BY (id_persona+v)
156 LIMIT 2) AS t ON CONFLICT DO NOTHING;
157
158
159 ANALYZE;

```

4.3.4. Escenario 1 000 000 registros

```

1  -- PERSONA (2,000,000)
2
3  INSERT INTO Persona (nombre, apellido, fecha_nacimiento)
4
5  SELECT
6  (ARRAY['Juan','Maria','Carlos','Lucia','Pedro','Ana','Jose','Rosa','Luis','Carmen
   ↪ '])[1+(g%10)],
7
8
9  (ARRAY['Quispe','Garcia','Flores','Rojas','Torres','Vargas','Diaz','Mamani','Cruz
   ↪ ','Leon'])[1+(g%10
10 )],
11
12 DATE '1960-01-01' + ((g*7) % 20000)
13
14 FROM generate_series(1, 2000000) AS g;
15
16
17
18 -- PASAJERO (1,500,000) y PERSONAL (500,000)
19
20 INSERT INTO Pasajero (id_persona, tipo_documento, numero_documento, id_categoria)
21
22 SELECT id_persona, (ARRAY['DNI','Pasaporte','Carnet de
23 Extranjeria'])[1+(id_persona%3)::tipo_doc_t,
24
25 LPAD(id_persona::text, 12, '0'), 1 + (id_persona % 3)
26
27 FROM Persona WHERE id_persona <= 1500000;
28
29 INSERT INTO Personal (id_persona)
30
31 SELECT id_persona FROM Persona WHERE id_persona BETWEEN 1500001 AND 2000000;
32
33 INSERT INTO Tripulacion (id_persona, num_licencia)
34
35 SELECT id_persona, 'LIC-' || LPAD(id_persona::text, 8, '0') FROM Personal WHERE
   ↪ id_persona
36 % 2 = 0;
37
38 INSERT INTO OperativoTierra (id_persona, area_operativa)
39
40 SELECT id_persona,
41 (ARRAY['Rampa','Equipajes','Seguridad','Migraciones','Mantenimiento'])[1+(
   ↪ id_persona%5)]
42
43 FROM Personal WHERE id_persona % 2 = 1;

```

```

44
45
46
47 -- VUELO (100,000)
48
49
50 INSERT INTO Vuelo (num_vuelo, hora_programada, hora_real, estado, tipo, origen,
    ↪ destino,
51 placa_aeronave, ruc_aerolinea)
52
53 SELECT (ARRAY['LA','H2','JA','2I','AV','CM'])[1+(g%6)] || (1000 + (g%9000)),
54
55 TIMESTAMPTZ '2026-01-01 00:00:00-05' + (g%180)*INTERVAL '1 day' + (g%24)*INTERVAL
    ↪ '1
56 hour',
57
58 CASE WHEN g%10<7 THEN TIMESTAMPTZ '2026-01-01 00:00:00-05' + (g%180)*INTERVAL
59 '1 day'
60
61 + (g%24)*INTERVAL '1 hour' + (g%120)*INTERVAL '1 minute' ELSE NULL END,
62
63
64 (ARRAY['Aterrizado','Despegado','Programado','Retrasado','Cancelado'])[1+(g%5)]::
    ↪ estado_vuelo_t,
65
66 CASE WHEN g%2=0 THEN 'Nacional' ELSE 'Internacional' END::tipo_vuelo_t, 'LIM',
67
68 CASE WHEN g%2=0 THEN
69 (ARRAY['CUZ','AQP','IQT','TPP','PCL','TRU','JUL','PIU','PEM','TBP'])[1+(g%10)]
70
71 ELSE (ARRAY['BOG','SCL','MIA','GRU','MAD','MEX','PTY','JFK','BCN','EZE'])[1+(g
    ↪ %10)]
72 END,
73
74 (SELECT placa FROM Aeronave OFFSET (g % 500) LIMIT 1),
75
76 (SELECT ruc FROM Aerolinea OFFSET (g % 24) LIMIT 1)
77
78 FROM generate_series(1, 100000) AS g;
79
80
81
82 -- TICKET (1,000,000) y EQUIPAJE (800,000)
83
84 INSERT INTO Ticket (precio, fecha_emision, estado_boarding, id_vuelo, id_persona)
85
86 SELECT 50 + (g%1500), DATE '2026-01-01' + (g%180),
87
88

```

```

89  (ARRAY['Emitido','Check-in','Embarcado','No-show','Cancelado'])[1+(g%5)]::
    ↪ estado_boarding_t,
90
91  1 + (g%100000), 1 + (g%1500000)
92
93  FROM generate_series(1, 1000000) AS g;
94
95  INSERT INTO Equipaje (id, peso, id_persona, id_vuelo)
96
97  SELECT 'TAG-' || LPAD(g::text,10,'0'), 5 + (g%28), 1 + (g%1500000), 1 + (g
    ↪ %100000)
98
99  FROM generate_series(1, 800000) AS g;
100
101
102  -- CHECK-IN (700,000)
103
104  INSERT INTO CheckIn (id_ticket, fecha_hora, counter, con equipaje)
105
106  SELECT g, NOW() - ((g%180)*INTERVAL '1 day') - ((g%12)*INTERVAL '1 hour'),
107
108  'C' || LPAD((1 + (g%40))::text, 2, '0'), (g%2 = 0)
109
110  FROM generate_series(1, 700000) AS g;
111
112
113
114  -- INCIDENCIA (1,000,000) ← protagonista
115
116  INSERT INTO Incidencia (gravedad, descripcion, tipo_incidencia, fecha_reporte,
    ↪ fecha_cierre)
117
118  SELECT (ARRAY['Leve','Moderada','Alta','Critica'])[1+(g%4)]::gravedad_t,
119
120  'Incidencia sintetica #' || g,
121
122
123  (ARRAY['Falla_Radar','Inundacion','Falta_Combustible','Saturacion_Vial','
    ↪ Manga_Inoperativa','Otro'])[1+(g%6)]::tipo_incidencia_t,
124
125  NOW() - ((g%365)*INTERVAL '1 day') - ((g%24)*INTERVAL '1 hour'),
126
127  CASE WHEN g%3=0 THEN NOW() - ((g%365)*INTERVAL '1 day') + INTERVAL '2 hour'
128  ELSE NULL END
129
130  FROM generate_series(1, 1000000) AS g;
131
132
133

```

```

134 -- TABLAS DE UNIÓN
135
136 INSERT INTO Utiliza (id_vuelo, id_recurso)
137
138 SELECT v, 1 + ((v*k) % 200) FROM generate_series(1,100000) AS v, generate_series
    ↪ (1,2) AS k
139 ON CONFLICT DO NOTHING;
140
141 INSERT INTO Afecto (id_incidencia, id_recurso)
142
143 SELECT id, 1 + (id % 200) FROM Incidencia WHERE id % 2 = 0 ON CONFLICT DO NOTHING
    ↪ ;
144
145 INSERT INTO Retrasa (id_incidencia, id_vuelo)
146
147 SELECT id, 1 + (id % 100000) FROM Incidencia WHERE id % 2 = 1 ON CONFLICT DO
    ↪ NOTHING;
148
149 INSERT INTO OperaTripulacion (id_persona, id_vuelo)
150
151
152 SELECT t.id_persona, v FROM generate_series(1,100000) AS v
153
154 CROSS JOIN LATERAL (SELECT id_persona FROM Tripulacion ORDER BY (id_persona+v)
155 LIMIT 2) AS t ON CONFLICT DO NOTHING;
156
157
158
159 ANALYZE;

```

4.4. Generación de vistas de usuario

Con el fin de simplificar el acceso a la información para los distintos tipos de usuario identificados en la sección 1.5.3, se definieron cuatro vistas de usuario. Cada vista encapsula una consulta de uso frecuente, presentando la información ya preparada y combinada de varias tablas, de modo que el usuario final no requiera conocer la estructura interna ni escribir juntas complejas. A continuación se describen las vistas implementadas:

vista_vuelos_operacion — Orientada al Agente de Operaciones LAP. Consolida el panel operativo de vuelos, mostrando para cada vuelo su número, tipo, estado, horarios programado y real, origen, destino, así como la aerolínea operadora y el modelo de aeronave asignado. Facilita el monitoreo en tiempo real de la actividad del terminal.

vista_checkin_pasajeros — Orientada al Personal de Aerolínea. Presenta el estado de facturación de cada pasajero, combinando la información del ticket, los datos del pasajero y su registro de check-in (hora, mostrador y si registró equipaje). Permite gestionar el embarque y verificar qué pasajeros han completado su check-in.

vista_incidentes_recurso — Orientada al Agente de Operaciones LAP. Lista las incidencias actualmente activas (sin fecha de cierre) sobre cada recurso de infraestructura, indicando su tipo, gravedad y fecha de reporte. Sirve para priorizar el mantenimiento de mangas y radares afectados.

vista_pasajeros_migracion — Orientada a la Autoridad Migratoria. Muestra los pasajeros de vuelos internacionales junto con su documento de identidad, categoría migratoria, tarifa TUUA aplicable y destino. Apoya el control migratorio y la trazabilidad de la tarifa de conexión.

El código SQL de creación de las vistas es el siguiente:

```
1  CREATE VIEW vista_vuelos_operacion AS
2
3  SELECT v.id, v.num_vuelo, v.tipo, v.estado,
4
5  v.hora_programada, v.hora_real, v.origen, v.destino,
6
7  al.nombre AS aerolinea, an.modelo AS aeronave
8
9  FROM Vuelo v
10
11 JOIN Aerolinea al ON al.ruc = v.ruc_aerolinea
12
13 JOIN Aeronave an ON an.placa = v.placa_aeronave;
14
15
16 CREATE VIEW vista_checkin_pasajeros AS
17
18 SELECT t.id_ticket, p.id_persona, per.nombre, per.apellido,
19
20 t.estado_boarding, c.fecha_hora AS hora_checkin,
21
22 c.counter, c.con equipaje, t.id_vuelo
23
24 FROM Ticket t
25
26 JOIN Pasajero p ON p.id_persona = t.id_persona
27
28 JOIN Persona per ON per.id_persona = p.id_persona
29
30 LEFT JOIN CheckIn c ON c.id_ticket = t.id_ticket;
31
32
33
34 CREATE VIEW vista_incidentes_recurso AS
35
36 SELECT i.id, i.tipo_incidencia, i.gravedad,
37
38 i.fecha_reporte, i.fecha_cierre,
39
40 r.id AS id_recurso, r.nombre_tecnico_locacion
```



```
41
42 FROM Incidencia i
43
44 JOIN Afecto a ON a.id_incidencia = i.id
45
46 JOIN Recurso r ON r.id = a.id_recurso
47
48 WHERE i.fecha_cierre IS NULL;
49
50
51
52 CREATE VIEW vista_pasajeros_migracion AS
53
54 SELECT per.nombre, per.apellido, p.tipo_documento, p.numero_documento,
55
56 cm.nombre AS categoria, cm.tarifa AS tarifa_tuua,
57
58 v.num_vuelo, v.destino
59
60 FROM Pasajero p
61
62 JOIN Persona per ON per.id_persona = p.id_persona
63
64 JOIN CategoriaMigratoria cm ON cm.id = p.id_categoria
65
66 JOIN Ticket t ON t.id_persona = p.id_persona
67
68
69 JOIN Vuelo v ON v.id = t.id_vuelo
70
71 WHERE v.tipo = 'Internacional';
```

5. Experimentación y evaluación de rendimiento

5.1. Consultas SQL para el experimento

5.1.1. Descripción del tipo de consultas seleccionadas

Para la experimentación se seleccionaron tres consultas de complejidad creciente, todas representativas de las necesidades reales de gestión del terminal y orientadas a los problemas operativos descritos en la sección de requisitos. Cada una combina operaciones de filtrado, junta (join) entre varias tablas y agregación, lo que las hace adecuadas para evaluar el impacto de los índices.

Consulta 1 — Recurso con mayor cantidad de fallas en la última semana.

Identifica el recurso de infraestructura (manga o radar) que ha registrado el mayor número de incidencias en los últimos siete días. Es una consulta de complejidad media que involucra un filtro por rango de

fechas sobre la tabla protagonista (Incidencia), una junta con la tabla de relación Afecto y con Recurso, y una agregación con agrupamiento y ordenamiento. Ataca directamente el problema de fallas técnicas recurrentes del terminal.

Consulta 2 — Tiempo promedio de retraso de los vuelos internacionales.

Calcula el promedio de minutos de retraso de los vuelos internacionales que ya han despegado. Es una consulta de complejidad baja-media centrada en un cálculo temporal ($\text{hora_real} - \text{hora_programada}$) sobre la tabla Vuelo, con un filtro por igualdad sobre el tipo de vuelo y descarte de valores nulos. Ataca el problema de saturación operativa.

Consulta 3 — Aerolínea con mayor cantidad de incidencias por falta de combustible.

Determina qué aerolínea concentra el mayor número de incidencias de tipo "falta de combustible" entre sus vuelos retrasados. Es la consulta de mayor complejidad, ya que encadena una junta de cuatro tablas (Incidencia, Retrasa, Vuelo, Aerolinea), un filtro por igualdad sobre el tipo de incidencia y una agregación con agrupamiento y ordenamiento. Ataca el problema de fallas de planificación logística.

La selección busca cubrir distintos patrones de acceso —filtro por rango de fechas, filtro por igualdad y cálculo temporal—, de modo que la experimentación evidencie el comportamiento de los índices ante operaciones de naturaleza diferente.

5.1.2. Implementación de consultas en SQL

Consulta 1:

```
1 SELECT r.id, r.nombre_tecnico_locacion, COUNT(*) AS total_fallas
```

	id bigint	num_vuelo character varying (10)	tipo tipo_vuelo_t	estado estado_vuelo_t	hora_programada timestamp with time zone	hora_real timestamp with time zone	origen character varying (4)	destino character varying (4)	aerolinea character va
1	1	H21001	Internacional	Despegado	2026-01-02 01:00:00-05	2026-01-02 01:01:00-05	LIM	SCL	Sky Airline P
2	2	JA1002	Nacional	Programado	2026-01-03 02:00:00-05	2026-01-03 02:02:00-05	LIM	IQT	JetSMART
3	3	2I1003	Internacional	Retrasado	2026-01-04 03:00:00-05	2026-01-04 03:03:00-05	LIM	GRU	Star Peru
4	4	AV1004	Nacional	Cancelado	2026-01-05 04:00:00-05	2026-01-05 04:04:00-05	LIM	PCL	Avianca
5	5	CM1005	Internacional	Aterrizado	2026-01-06 05:00:00-05	2026-01-06 05:05:00-05	LIM	MEX	Copa Airline

Figura 4: Resultado de la vista de vuelos de operación.

```
1 FROM Incidencia i
2
3 JOIN Afecto a ON a.id_incidencia = i.id
4
5 JOIN Recurso r ON r.id = a.id_recurso
6
7 WHERE i.fecha_reporte >= NOW() - INTERVAL '7 days'
8
9 GROUP BY r.id, r.nombre_tecnico_locacion
10
11 ORDER BY total_fallas DESC
12
13 LIMIT 1;
```

Consulta 2:

```
1  SELECT ROUND(AVG(EXTRACT(EPOCH FROM (v.hora_real - v.hora_programada)) / 60), 2)
2
3  AS retraso_promedio_min
4
5  FROM Vuelo v
6
7  WHERE v.tipo = 'Internacional'
8
9  AND v.hora_real IS NOT NULL;
```

Consulta 3:

```
1  SELECT al.ruc, al.nombre, COUNT(*) AS incidencias_combustible
2
3  FROM Incidencia i
4
5  JOIN Retrasa rt ON rt.id_incidencia = i.id
6
7  JOIN Vuelo v ON v.id = rt.id_vuelo
8
9  JOIN Aerolinea al ON al.ruc = v.ruc_aerolinea
10
11 WHERE i.tipo_incidencia = 'Falta_Combustible'
12
13 GROUP BY al.ruc, al.nombre
14
15 ORDER BY incidencias_combustible DESC
16
17 LIMIT 1;
```

5.2. Metodología del experimento

Con el fin de evaluar de manera objetiva el impacto de los índices sobre el rendimiento de la base de datos, se diseñó un experimento controlado que ejecuta cada una de las tres consultas seleccionadas sobre cuatro volúmenes de datos distintos (1 000, 10 000, 100 000 y 1 000 000 de registros en la tabla protagonista), en dos condiciones: sin índices y con índices. El procedimiento seguido fue el siguiente:

1. Preparación de los escenarios. Para cada volumen se construyó una instancia independiente de la base de datos a partir del modelo relacional, ejecutando los scripts de creación de estructura (4.1), carga de catálogos (4.2) y generación de datos (4.3). De cada instancia se obtuvo un dump independiente.
2. Limpieza de caché entre iteraciones. Antes de cada medición se ejecutó el comando VACUUM FULL para limpiar la caché de PostgreSQL y reorganizar el almacenamiento físico, garantizando que todas las consultas se ejecuten en igualdad de condiciones y que los tiempos no se vean alterados

por resultados previamente almacenados en memoria.

3. Desactivación de los métodos de acceso optimizados (escenario sin índices). Dado que PostgreSQL crea automáticamente índices sobre las claves primarias y restricciones UNIQUE, no es posible eliminar todos los índices. Por ello, para simular un escenario sin optimización se desactivaron los métodos de acceso que dependen de estructuras auxiliares, mediante: SET enable_mergejoin TO OFF; SET enable_hashjoin TO OFF; SET enable_bitmapscan TO OFF; SET enable_sort TO OFF;
4. Medición con EXPLAIN ANALYZE. Cada consulta se ejecutó precedida del comando EXPLAIN ANALYZE, que devuelve el plan de ejecución real elegido por el planificador junto con el tiempo de ejecución efectivo (Execution Time). Se registró tanto el árbol del plan de ejecución (Query Plan Tree) como el detalle textual del plan.
5. Activación de índices y métodos de acceso (escenario con índices). Posteriormente se reactivaron los métodos de acceso y se crearon los índices específicos para cada consulta (detallados en la sección 5.3), repitiendo la medición bajo las mismas condiciones: SET enable_mergejoin TO ON; SET enable_hashjoin TO ON; SET enable_bitmapscan TO ON; SET enable_sort TO ON;
6. Repeticiones y tratamiento estadístico. Cada consulta, en cada volumen y en cada condición (con y sin índices), se ejecutó cinco veces. A partir de los cinco tiempos obtenidos se calcularon el promedio y la desviación estándar, con el objetivo de reducir el efecto de variaciones puntuales del sistema operativo y obtener mediciones representativas. Todos los tiempos se expresan en milisegundos (ms).
7. Registro y análisis de resultados. Los tiempos promedio se consolidaron en tablas comparativas y se representaron gráficamente (tiempo de ejecución frente a volumen de datos), contrastando la curva sin índices con la curva con índices para cada consulta, lo que permite visualizar el efecto de la indexación a medida que crece el tamaño de la base de datos.

5.3. Optimización de consultas

Para optimizar cada consulta se crearon índices sobre las columnas involucradas en los filtros y las juntas. La elección del tipo de índice depende de la operación: se emplea hash para búsquedas por igualdad (=), ya que ofrece acceso directo muy eficiente para ese caso, y btree para búsquedas por rango (>=, <=, BETWEEN) y para columnas usadas en juntas y ordenamientos, por su capacidad de recorrer valores ordenados.

5.3.1. Índices para la Consulta 1

La Consulta 1 filtra las incidencias por un rango de fechas (fecha_reporte >= NOW() - INTERVAL '7 days') y realiza una junta a través de Afecto.id_recurso. Se crean dos índices:

```
1 CREATE INDEX idx_incidencia_fecha ON Incidencia USING btree (fecha_reporte);
2
3 CREATE INDEX idx_afecto_recurso ON Afecto USING btree (id_recurso);
```

```
4
5 Se utiliza btree en fecha_reporte porque la condición es un filtro por rango,
  ↪ escenario en el que el
6 árbol balanceado permite localizar eficientemente el subconjunto de filas dentro
  ↪ del intervalo. El índice
7 sobre Afecto.id_recurso acelera la junta con la tabla Recurso.
```

5.3.2. Índices para la Consulta 2

La Consulta 2 filtra los vuelos por igualdad sobre el tipo (tipo = 'Internacional') y descarta los que aún no tienen hora real. Se crea un índice compuesto:

```
1 CREATE INDEX idx_vuelo_tipo_hora ON Vuelo USING btree (tipo, hora_real);
2
3 Se emplea un índice btree compuesto sobre (tipo, hora_real) porque permite
  ↪ resolver en una
4 sola estructura tanto el filtro por igualdad del tipo de vuelo como el descarte
  ↪ de los valores nulos de
5 hora_real, evitando recorrer la tabla completa.
```

5.3.3. Índices para la Consulta 3

La Consulta 3 filtra las incidencias por igualdad sobre el tipo (tipo_incidencia = 'Falta_Combustible') y encadena juntas a través de Retrasa.id_vuelo y Vuelo.ruc_aerolinea. Se crean tres índices:

```
1 CREATE INDEX idx_incidencia_tipo ON Incidencia USING hash (tipo_incidencia);
2
3 CREATE INDEX idx_retrasa_vuelo ON Retrasa USING btree (id_vuelo);
4
5 CREATE INDEX idx_vuelo_aerolinea ON Vuelo USING btree (ruc_aerolinea);
6
7 Se utiliza hash en tipo_incidencia porque la condición es una búsqueda por
  ↪ igualdad sobre un
8 valor fijo del enumerado, caso en el que el índice hash ofrece acceso directo.
  ↪ Los índices btree sobre
9 Retrasa.id_vuelo y Vuelo.ruc_aerolinea aceleran las juntas sucesivas entre las
  ↪ cuatro tablas.
```

5.3.4. Identificación de índices implementados por PostgreSQL

Adicionalmente, dado que PostgreSQL crea automáticamente índices sobre las claves primarias y las restricciones UNIQUE, se ejecutó la siguiente consulta para identificar los índices existentes en el esquema antes de añadir los índices propios del experimento:

```
1 SELECT tablename, indexname, indexdef
```

```

2
3 FROM pg_indexes
4
5 WHERE schemaname = 'public'
6
7 ORDER BY tablename;
8
9 Esto permite distinguir entre los índices generados automáticamente por las
    ↪ restricciones del modelo
10 (claves primarias y alternativas) y los índices creados manualmente para
    ↪ optimizar las consultas
11 seleccionadas.

```

5.4. Plataforma de Pruebas

	tablename name	indexname name	indexdef text
1	aerolinea	aerolinea_pk	CREATE UNIQUE INDEX aerolinea_pk ON public.aerolinea USING btree (ruc)
2	aeronave	aeronave_pk	CREATE UNIQUE INDEX aeronave_pk ON public.aeronave USING btree (placa)
3	afecto	afecto_pk	CREATE UNIQUE INDEX afecto_pk ON public.afecto USING btree (id_incidencia, id_recurso)
4	asiento	asiento_pk	CREATE UNIQUE INDEX asiento_pk ON public.asiento USING btree (codigo, placa_aeronave)
5	categoriamigrato...	categoria_pk	CREATE UNIQUE INDEX categoria_pk ON public.categoriamigratoria USING btree (id)
6	checkin	checkin_pk	CREATE UNIQUE INDEX checkin_pk ON public.checkin USING btree (id_ticket)
7	equipaje	equipaje_pk	CREATE UNIQUE INDEX equipaje_pk ON public.equipaje USING btree (id)
8	incidencia	incidencia_pk	CREATE UNIQUE INDEX incidencia_pk ON public.incidencia USING btree (id)
9	manga	manga_pk	CREATE UNIQUE INDEX manga_pk ON public.manga USING btree (id_recurso)
10	operativotierra	operativo_pk	CREATE UNIQUE INDEX operativo_pk ON public.operativotierra USING btree (id_persona)
11	operatripulacion	opera_pk	CREATE UNIQUE INDEX opera_pk ON public.operatripulacion USING btree (id_persona, id_vuelo)
12	pasajero	pasajero_doc_uk	CREATE UNIQUE INDEX pasajero_doc_uk ON public.pasajero USING btree (tipo_documento, numero_docum...
13	pasajero	pasajero_pk	CREATE UNIQUE INDEX pasajero_pk ON public.pasajero USING btree (id_persona)
14	persona	persona_pk	CREATE UNIQUE INDEX persona_pk ON public.persona USING btree (id_persona)
15	personal	personal_pk	CREATE UNIQUE INDEX personal_pk ON public.personal USING btree (id_persona)
16	radar	radar_pk	CREATE UNIQUE INDEX radar_pk ON public.radar USING btree (id_recurso)
17	recurso	recurso_pk	CREATE UNIQUE INDEX recurso_pk ON public.recurso USING btree (id)
18	retrasa	retrasa_pk	CREATE UNIQUE INDEX retrasa_pk ON public.retrasa USING btree (id_incidencia, id_vuelo)
19	ticket	ticket_pk	CREATE UNIQUE INDEX ticket_pk ON public.ticket USING btree (id_ticket)
20	tripulacion	tripulacion_lic_...	CREATE UNIQUE INDEX tripulacion_lic_uk ON public.tripulacion USING btree (num_licencia)
21	tripulacion	tripulacion_pk	CREATE UNIQUE INDEX tripulacion_pk ON public.tripulacion USING btree (id_persona)

Figura 5: Índices creados por PostgreSQL

22	utiliza	utiliza_pk	CREATE UNIQUE INDEX utiliza_pk ON public.utiliza USING btree (id_vuelo, id_recurso)
23	vuelo	vuelo_pk	CREATE UNIQUE INDEX vuelo_pk ON public.vuelo USING btree (id)

Figura 6: Continuación del listado de índices creados por PostgreSQL.

Todas las mediciones de la experimentación se realizaron sobre un mismo equipo, con el fin de garantizar la consistencia y comparabilidad de los tiempos obtenidos. A continuación se detallan las características de hardware y software de la plataforma utilizada:

Sistema Operativo Windows 11 Home (64 bits) Procesador 13th Gen Intel(R) Core(TM) i7-13620H @ 2.40 GHz Memoria RAM 16 GB Almacenamiento Unidad de estado sólido (SSD) Sistema gestor de base de PostgreSQL 18.3 (x86_64-windows, 64-bit) datos Herramienta de pgAdmin 4 administración

Especificaciones de la plataforma de pruebas

Cabe señalar que la capacidad del hardware influye directamente en los tiempos de respuesta medidos; un equipo con mayor memoria, un procesador más veloz o almacenamiento de estado sólido reduce los tiempos absolutos de ejecución. No obstante, las tendencias relativas observadas entre las ejecuciones con y sin índices se mantienen independientemente del equipo, ya que reflejan diferencias algorítmicas en el acceso a los datos.

5.4. Medición de tiempos

EDICIÓN 1K REGISTROS

5.4.1. Ejecución de consulta 1

5.4.1.1. Ejecución sin índices para 1k datos

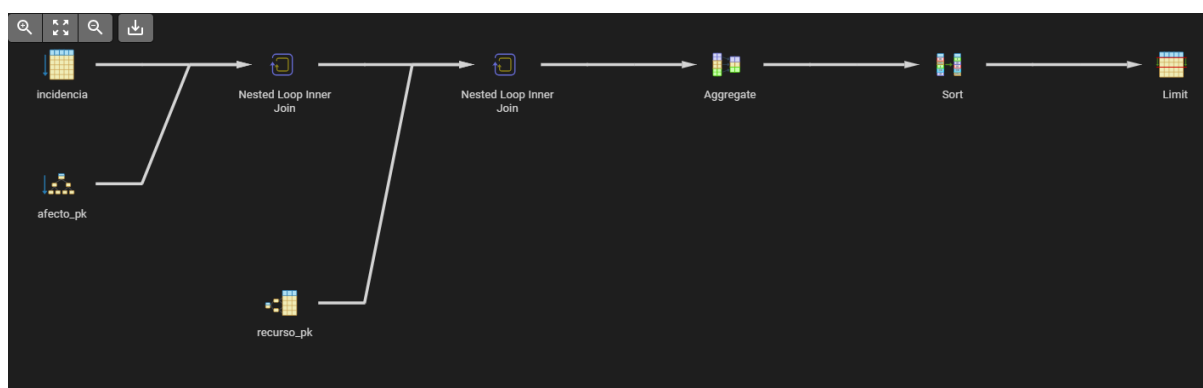


Figura 7: Componente Especificación

5.4.1.2. Ejecución con índices para 1k datos

	QUERY PLAN
	text
1	Limit (cost=64.65..64.65 rows=1 width=21) (actual time=0.512..0.515 rows=1.00 loops=1)
2	Buffers: shared hit=81
3	-> Sort (cost=64.65..64.67 rows=10 width=21) (actual time=0.511..0.513 rows=1.00 loops=1)
4	Disabled: true
5	Sort Key: (count(*)) DESC
6	Sort Method: top-N heapsort Memory: 25kB
7	Buffers: shared hit=81
8	-> HashAggregate (cost=64.50..64.60 rows=10 width=21) (actual time=0.484..0.488 rows=10.00 loops=1)
9	Group Key: r.id
10	Batches: 1 Memory Usage: 32kB
11	Buffers: shared hit=81
12	-> Nested Loop (cost=0.42..64.45 rows=10 width=13) (actual time=0.089..0.466 rows=10.00 loops=1)
13	Buffers: shared hit=81
14	-> Nested Loop (cost=0.27..62.50 rows=10 width=4) (actual time=0.074..0.432 rows=10.00 loops=1)
15	Buffers: shared hit=61
16	-> Seq Scan on incidencia i (cost=0.00..28.50 rows=20 width=8) (actual time=0.034..0.349 rows=20.00 loops=1)
17	Filter: (fecha_reporte >= (now() - '7 days'::interval))
18	Rows Removed by Filter: 980
19	Buffers: shared hit=11
20	-> Index Only Scan using afecto_pk on afecto a (cost=0.27..1.69 rows=1 width=12) (actual time=0.004..0.004 rows=0.50 loop...

Figura 8: Query Plan Tree de la consulta 1 de 1k datos sin índices

21	Index Cond: (id_incidencia = i.id)
22	Heap Fetches: 10
23	Index Searches: 20
24	Buffers: shared hit=50
25	-> Index Scan using recurso_pk on recurso r (cost=0.14..0.19 rows=1 width=13) (actual time=0.003..0.003 rows=1.00 loops=10)
26	Index Cond: (id = a.id_recurso)
27	Index Searches: 10
28	Buffers: shared hit=20
29	Planning Time: 0.410 ms
30	Execution Time: 0.619 ms

Figura 9: Query Plan de la consulta 1 de 1k datos sin índices

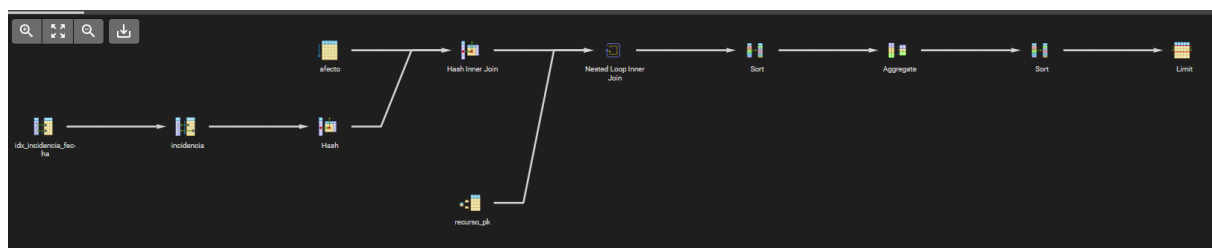


Figura 10: Query Plan Tree de la consulta 1 de 1k datos con índices

	QUERY PLAN text	
1	Limit (cost=27.68..27.69 rows=1 width=21) (actual time=0.168..0.170 rows=1.00 loops=1)	
2	Buffers: shared hit=28	
3	-> Sort (cost=27.68..27.71 rows=10 width=21) (actual time=0.167..0.168 rows=1.00 loops=1)	
4	Sort Key: (count(*)) DESC	
5	Sort Method: top-N heapsort Memory: 25kB	
6	Buffers: shared hit=28	
7	-> GroupAggregate (cost=27.46..27.63 rows=10 width=21) (actual time=0.153..0.159 rows=10.00 loops=1)	
8	Group Key: r.id	
9	Buffers: shared hit=28	
10	-> Sort (cost=27.46..27.48 rows=10 width=13) (actual time=0.148..0.150 rows=10.00 loops=1)	
11	Sort Key: r.id	
12	Sort Method: quicksort Memory: 25kB	
13	Buffers: shared hit=28	
14	-> Nested Loop (cost=16.18..27.29 rows=10 width=13) (actual time=0.065..0.141 rows=10.00 loops=1)	
15	Buffers: shared hit=28	
16	-> Hash Join (cost=16.04..25.35 rows=10 width=4) (actual time=0.061..0.126 rows=10.00 loops=1)	
17	Hash Cond: (a.id_incidencia = i.id)	
18	Buffers: shared hit=8	
19	-> Seq Scan on afecto a (cost=0.00..8.00 rows=500 width=12) (actual time=0.015..0.042 rows=500.00 loops=1)	
20	Buffers: shared hit=3	
21	-> Hash (cost=15.79..15.79 rows=20 width=8) (actual time=0.030..0.030 rows=20.00 loops=1)	

Figura 11: Elemento gráfico extraído de la página 51 del documento original.

5.4.2. Ejecución de consulta 2

5.4.2.1. Ejecución sin índices para 1k datos

22	Buckets: 1024 Batches: 1 Memory Usage: 9kB
23	Buffers: shared hit=5
24	-> Bitmap Heap Scan on incidencia i (cost=4.44..15.79 rows=20 width=8) (actual time=0.021..0.026 rows=20.00 loops=1)
25	Recheck Cond: (fecha_reporte >= (now() - '7 days'::interval))
26	Heap Blocks: exact=3
27	Buffers: shared hit=5
28	-> Bitmap Index Scan on idx_incidencia_fecha (cost=0.00..4.43 rows=20 width=0) (actual time=0.011..0.011 rows=20.00 lo...
29	Index Cond: (fecha_reporte >= (now() - '7 days'::interval))
30	Index Searches: 1
31	Buffers: shared hit=2
32	-> Index Scan using recurso_pk on recurso r (cost=0.14..0.19 rows=1 width=13) (actual time=0.001..0.001 rows=1.00 loops=10)
33	Index Cond: (id = a.id_recurso)
34	Index Searches: 10
35	Buffers: shared hit=20
36	Planning:
37	Buffers: shared hit=237 read=18
38	Planning Time: 10.675 ms
39	Execution Time: 0.229 ms

Figura 12: Query Plan de la consulta 1 de 1k datos con índices

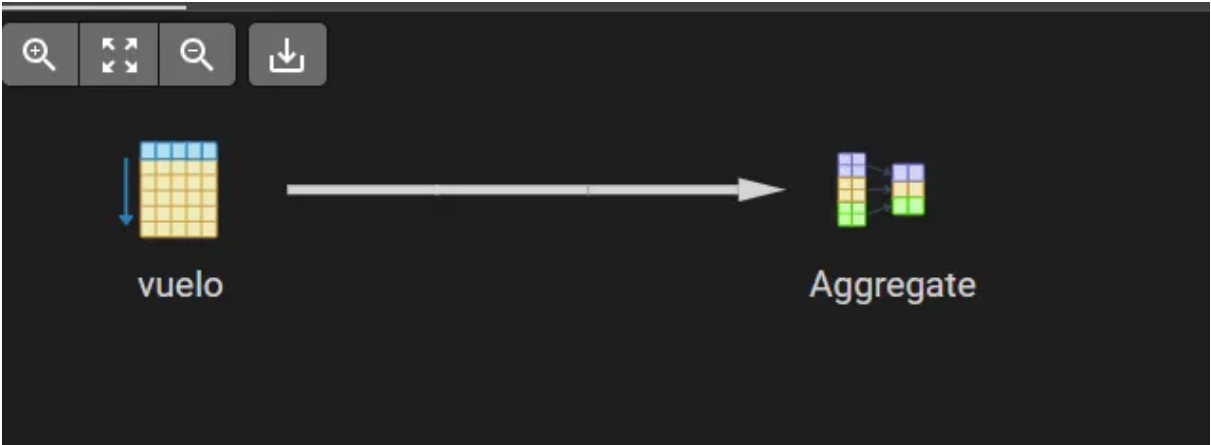


Figura 13: Query Plan Tree de la consulta 2 de 1k datos sin índices

5.4.2.2. Ejecución con índices para 1k datos

	QUERY PLAN	
	text	
1	Aggregate (cost=6.20..6.22 rows=1 width=32) (actual time=0.097..0.098 rows=1.00 loops=1)	
2	Buffers: shared hit=3	
3	-> Seq Scan on vuelo v (cost=0.00..5.50 rows=70 width=16) (actual time=0.017..0.052 rows=60.00 loop...	
4	Filter: ((hora_real IS NOT NULL) AND (tipo = 'Internacional'::tipo_vuelo_t))	
5	Rows Removed by Filter: 140	
6	Buffers: shared hit=3	
7	Planning:	
8	Buffers: shared hit=53 read=5	
9	Planning Time: 2.634 ms	
10	Execution Time: 0.129 ms	

Figura 14: Query Plan de la consulta 2 de 1k datos sin índices



Figura 15: Query Plan Tree de la consulta 2 de 1k datos con índices

5.4.3. Ejecución de consulta 3

5.4.3.1. Ejecución sin índices para 1k datos

	QUERY PLAN
	text
1	Aggregate (cost=6.20..6.22 rows=1 width=32) (actual time=0.069..0.069 rows=1.00 loops=1)
2	Buffers: shared hit=3
3	-> Seq Scan on vuelo v (cost=0.00..5.50 rows=70 width=16) (actual time=0.017..0.038 rows=60.00 loop...
4	Filter: ((hora_real IS NOT NULL) AND (tipo = 'Internacional'::tipo_vuelo_t))
5	Rows Removed by Filter: 140
6	Buffers: shared hit=3
7	Planning:
8	Buffers: shared hit=103 read=11
9	Planning Time: 4.554 ms
10	Execution Time: 0.097 ms

Figura 16: Query Plan de la consulta 2 de 1k datos con índices

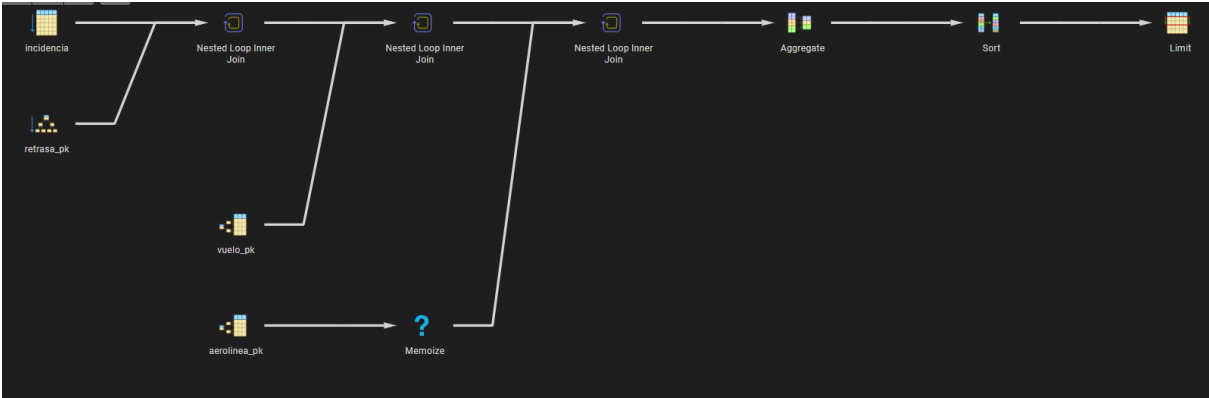


Figura 17: Query Plan Tree de la consulta 3 de 1k datos sin índices

	QUERY PLAN	
	text	🔒
1	Limit (cost=126.66..126.66 rows=1 width=34) (actual time=0.311..0.312 rows=0.00 loops=1)	
2	Buffers: shared hit=342 read=3	
3	-> Sort (cost=126.66..126.72 rows=24 width=34) (actual time=0.310..0.311 rows=0.00 loops=1)	
4	Disabled: true	
5	Sort Key: (count(*)) DESC	
6	Sort Method: quicksort Memory: 25kB	
7	Buffers: shared hit=342 read=3	
8	-> HashAggregate (cost=126.30..126.54 rows=24 width=34) (actual time=0.304..0.306 rows=0.00 loops=1)	
9	Group Key: al.ruc	
10	Batches: 1 Memory Usage: 32kB	
11	Buffers: shared hit=342 read=3	
12	-> Nested Loop (cost=0.56..125.88 rows=84 width=26) (actual time=0.302..0.303 rows=0.00 loops=1)	
13	Buffers: shared hit=342 read=3	
14	-> Nested Loop (cost=0.42..118.61 rows=84 width=12) (actual time=0.302..0.302 rows=0.00 loops=1)	
15	Buffers: shared hit=342 read=3	
16	-> Nested Loop (cost=0.27..101.60 rows=84 width=8) (actual time=0.301..0.302 rows=0.00 loops=1)	
17	Buffers: shared hit=342 read=3	
18	-> Seq Scan on incidencia i (cost=0.00..23.50 rows=167 width=8) (actual time=0.016..0.107 rows=167.00 loops=1)	
19	Filter: (tipo_incidencia = 'Falta_Combustible'::tipo_incidencia_t)	
20	Rows Removed by Filter: 833	
21	Buffers: shared hit=11	

Figura 18: Query Plan de la consulta 3 de 1k datos sin índices

22	-> Index Only Scan using retrasa_pk on retrasa rt (cost=0.27..0.46 rows=1 width=16) (actual time=0.001..0.001 rows=0.00 loop...
23	Index Cond: (id_incidencia = i.id)
24	Heap Fetches: 0
25	Index Searches: 167
26	Buffers: shared hit=331 read=3
27	-> Index Scan using vuelo_pk on vuelo v (cost=0.14..0.20 rows=1 width=20) (never executed)
28	Index Cond: (id = rt.id_vuelo)
29	Index Searches: 0
30	-> Memoize (cost=0.15..0.23 rows=1 width=26) (never executed)
31	Cache Key: v.ruc_aerolinea
32	Cache Mode: logical
33	-> Index Scan using aerolinea_pk on aerolinea al (cost=0.14..0.21 rows=1 width=26) (never executed)
34	Index Cond: ((ruc)::text = (v.ruc_aerolinea)::text)
35	Index Searches: 0
36	Planning:
37	Buffers: shared hit=253 read=14
38	Planning Time: 10.372 ms
39	Execution Time: 0.389 ms

Figura 19: Elemento gráfico extraído de la página 55 del documento original.

5.4.3.2. Ejecución con índices para 1k datos

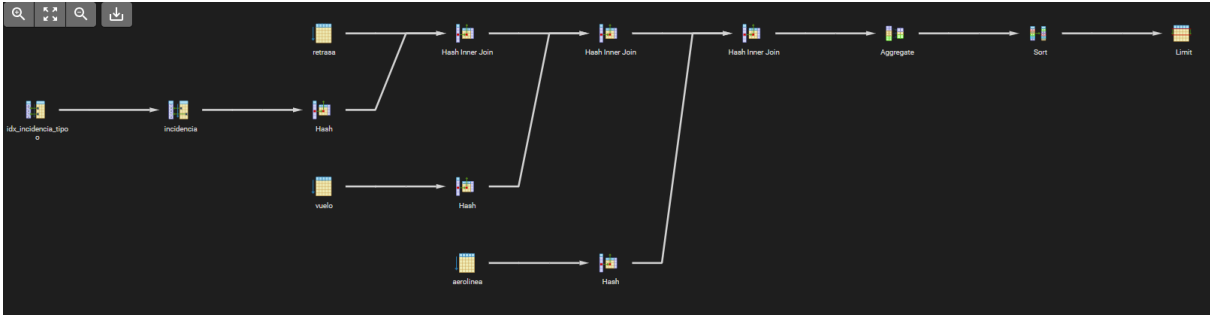


Figura 20: Query Plan Tree de la consulta 3 de 1k datos con índices

	QUERY PLAN
	text
1	Limit (cost=44.09..44.09 rows=1 width=34) (actual time=0.250..0.252 rows=0.00 loops=1)
2	Buffers: shared hit=20
3	-> Sort (cost=44.09..44.15 rows=24 width=34) (actual time=0.249..0.251 rows=0.00 loops=1)
4	Sort Key: (count(*)) DESC
5	Sort Method: quicksort Memory: 25kB
6	Buffers: shared hit=20
7	-> HashAggregate (cost=43.73..43.97 rows=24 width=34) (actual time=0.245..0.246 rows=0.00 loops=1)
8	Group Key: al.ruc
9	Batches: 1 Memory Usage: 32kB
10	Buffers: shared hit=20
11	-> Hash Join (cost=33.51..43.31 rows=84 width=26) (actual time=0.243..0.244 rows=0.00 loops=1)
12	Hash Cond: ((v.ruc_aerolinea)::text = (al.ruc)::text)
13	Buffers: shared hit=20
14	-> Hash Join (cost=31.97..41.50 rows=84 width=12) (actual time=0.217..0.219 rows=0.00 loops=1)
15	Hash Cond: (rt.id_vuelo = v.id)
16	Buffers: shared hit=19
17	-> Hash Join (cost=24.47..33.78 rows=84 width=8) (actual time=0.161..0.162 rows=0.00 loops=1)
18	Hash Cond: (rt.id_incidencia = i.id)
19	Buffers: shared hit=16
20	-> Seq Scan on retrasa rt (cost=0.00..8.00 rows=500 width=16) (actual time=0.010..0.037 rows=500.00 loops=1)
21	Buffers: shared hit=3

Figura 21: Elemento gráfico extraído de la página 56 del documento original.

Análisis de los planes de ejecución — Escenario 1 000 registros

En el escenario de 1 000 registros, los planes confirman el comportamiento esperado para volúmenes pequeños. Sin índices, la Consulta 1 se resuelve con un Seq Scan sobre Incidencia (el filtro fecha_reporte >= now() - 7 days descarta 980 de las 1 000 filas), encadenado con dos Nested Loop para las juntas con Afecto y Recurso, seguidos de un HashAggregate y un Sort con método top-N heapsort para el Limit final. Con índices, el plan cambia de estrategia: la junta principal pasa a Hash Join (condición a.id_incidencia = i.id) y la agregación se realiza con GroupAggregate. Pese al cambio, la diferencia de tiempos es reducida (0.623 ms a 0.229 ms), pues a este volumen ambos enfoques son muy rápidos. Las Consultas 2 y 3 siguen el mismo patrón: recorridos secuenciales y juntas por nested loop sin índices, que el optimizador sustituye parcialmente por hash joins al disponer de los índices, con mejoras marginales.

	QUERY PLAN	
	text	
22	-> Hash (cost=22.38..22.38 rows=167 width=8) (actual time=0.087..0.087 rows=167.00 loops=1)	
23	Buckets: 1024 Batches: 1 Memory Usage: 15kB	
24	Buffers: shared hit=13	
25	-> Bitmap Heap Scan on incidencia i (cost=9.29..22.38 rows=167 width=8) (actual time=0.025..0.069 rows=167.00 loops=1)	
26	Recheck Cond: (tipo_incidencia = 'Falta_Combustible':tipo_incidencia_t)	
27	Heap Blocks: exact=11	
28	Buffers: shared hit=13	
29	-> Bitmap Index Scan on idx_incidencia_tipo (cost=0.00..9.25 rows=167 width=0) (actual time=0.015..0.015 rows=167.00 lo...	
30	Index Cond: (tipo_incidencia = 'Falta_Combustible':tipo_incidencia_t)	
31	Index Searches: 1	
32	Buffers: shared hit=2	
33	-> Hash (cost=5.00..5.00 rows=200 width=20) (actual time=0.055..0.055 rows=200.00 loops=1)	
34	Buckets: 1024 Batches: 1 Memory Usage: 19kB	
35	Buffers: shared hit=3	
36	-> Seq Scan on vuelo v (cost=0.00..5.00 rows=200 width=20) (actual time=0.007..0.032 rows=200.00 loops=1)	
37	Buffers: shared hit=3	
38	-> Hash (cost=1.24..1.24 rows=24 width=26) (actual time=0.022..0.022 rows=24.00 loops=1)	
39	Buckets: 1024 Batches: 1 Memory Usage: 10kB	
40	Buffers: shared hit=1	
41	-> Seq Scan on aerolinea al (cost=0.00..1.24 rows=24 width=26) (actual time=0.013..0.016 rows=24.00 loops=1)	
42	Buffers: shared hit=1	

Figura 22: Query Plan de la consulta 3 de 1k datos con índices

42	Buffers: shared hit=1
43	Planning:
44	Buffers: shared hit=297 read=22
45	Planning Time: 14.946 ms
46	Execution Time: 0.312 ms

Figura 23: Elemento gráfico extraído de la página 57 del documento original.

EDICIÓN 10K REGISTROS

5.4.4. Ejecución de consulta 1

5.4.4.1. Ejecución sin índices para 10k datos

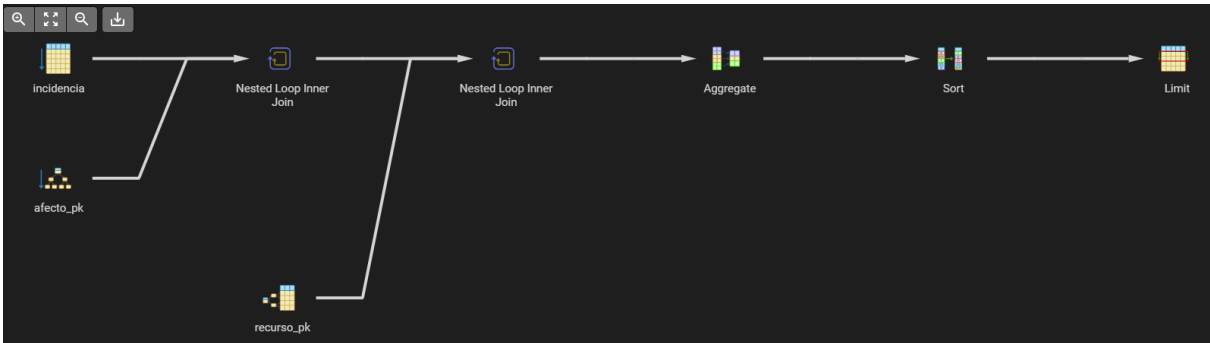


Figura 24: Query Plan Tree de la consulta 1 de 10k datos sin índices

	QUERY PLAN
	text
1	Limit (cost=560.96..560.96 rows=1 width=21) (actual time=2.763..2.764 rows=1.00 loops=1)
2	Buffers: shared hit=767 read=21
3	-> Sort (cost=560.96..561.20 rows=98 width=21) (actual time=2.762..2.763 rows=1.00 loops=1)
4	Disabled: true
5	Sort Key: (count(*)) DESC
6	Sort Method: top-N heapsort Memory: 25kB
7	Buffers: shared hit=767 read=21
8	-> HashAggregate (cost=559.49..560.47 rows=98 width=21) (actual time=2.734..2.746 rows=82.00 loops=1)
9	Group Key: r.id
10	Batches: 1 Memory Usage: 32kB
11	Buffers: shared hit=767 read=21
12	-> Nested Loop (cost=0.43..559.00 rows=98 width=13) (actual time=0.141..2.698 rows=97.00 loops=1)
13	Buffers: shared hit=767 read=21
14	-> Nested Loop (cost=0.28..542.76 rows=98 width=4) (actual time=0.121..2.562 rows=97.00 loops=1)
15	Buffers: shared hit=574 read=20
16	-> Seq Scan on incidencia i (cost=0.00..282.00 rows=196 width=8) (actual time=0.017..2.031 rows=195.00 loops=1)
17	Filter: (fecha_reporte >= (now() - '7 days'::interval))
18	Rows Removed by Filter: 9805
19	Buffers: shared hit=107
20	-> Index Only Scan using afecto_pk on afecto a (cost=0.28..1.32 rows=1 width=12) (actual time=0.002..0.002 rows=0.50 loops=1)
21	Index Cond: (id_incidencia = i.id)

Figura 25: Elemento gráfico extraído de la página 58 del documento original.

5.4.4.2. Ejecución con índices para 10k datos

22	Heap Fetches: 97
23	Index Searches: 195
24	Buffers: shared hit=467 read=20
25	-> Index Scan using recurso_pk on recurso r (cost=0.14..0.17 rows=1 width=13) (actual time=0.001..0.001 rows=1.00 loops=97)
26	Index Cond: (id = a.id_recurso)
27	Index Searches: 97
28	Buffers: shared hit=193 read=1
29	Planning:
30	Buffers: shared hit=139 read=9
31	Planning Time: 6.134 ms
32	Execution Time: 2.816 ms

Figura 26: Query Plan de la consulta 1 de 10k datos sin índices

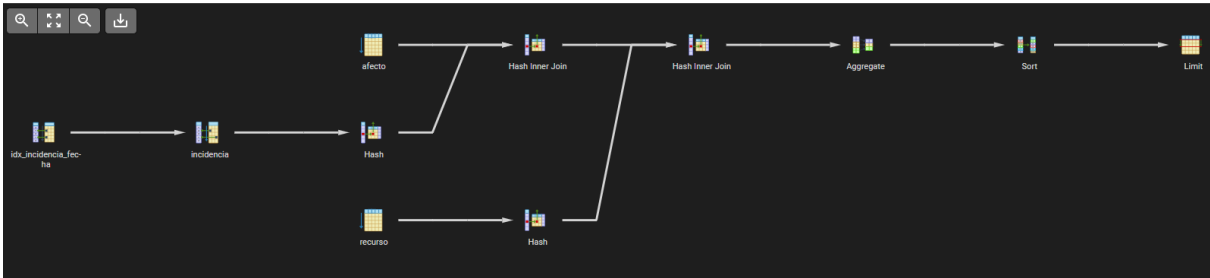


Figura 27: Query Plan Tree de la consulta 1 de 10k datos con índices

	QUERY PLAN	
	text	🔒
1	Limit (cost=220.36..220.36 rows=1 width=21) (actual time=1.270..1.273 rows=1.00 loops=1)	
2	Buffers: shared hit=62	
3	-> Sort (cost=220.36..220.61 rows=98 width=21) (actual time=1.268..1.272 rows=1.00 loops=1)	
4	Sort Key: (count(*)) DESC	
5	Sort Method: top-N heapsort Memory: 25kB	
6	Buffers: shared hit=62	
7	-> HashAggregate (cost=218.89..219.87 rows=98 width=21) (actual time=1.232..1.248 rows=82.00 loops=1)	
8	Group Key: r.id	
9	Batches: 1 Memory Usage: 32kB	
10	Buffers: shared hit=62	
11	-> Hash Join (cost=127.02..218.40 rows=98 width=13) (actual time=0.296..1.193 rows=97.00 loops=1)	
12	Hash Cond: (a.id_recurso = r.id)	
13	Buffers: shared hit=62	
14	-> Hash Join (cost=120.52..211.64 rows=98 width=4) (actual time=0.221..1.086 rows=97.00 loops=1)	
15	Hash Cond: (a.id_incidencia = i.id)	
16	Buffers: shared hit=60	
17	-> Seq Scan on afecto a (cost=0.00..78.00 rows=5000 width=12) (actual time=0.014..0.374 rows=5000.00 loops=1)	
18	Buffers: shared hit=28	
19	-> Hash (cost=118.07..118.07 rows=196 width=8) (actual time=0.201..0.202 rows=195.00 loops=1)	
20	Buckets: 1024 Batches: 1 Memory Usage: 16kB	
21	Buffers: shared hit=32	

Figura 28: Query Plan de la consulta 1 de 10k datos con índices

21	Buffers: shared hit=32
22	-> Bitmap Heap Scan on incidencia i (cost=5.81..118.07 rows=196 width=8) (actual time=0.046..0.149 rows=195.00 loops=1)
23	Recheck Cond: (fecha_reporte >= (now() - '7 days'::interval))
24	Heap Blocks: exact=30
25	Buffers: shared hit=32
26	-> Bitmap Index Scan on idx_incidencia_fecha (cost=0.00..5.76 rows=196 width=0) (actual time=0.028..0.028 rows=195.00 lo...
27	Index Cond: (fecha_reporte >= (now() - '7 days'::interval))
28	Index Searches: 1
29	Buffers: shared hit=2
30	-> Hash (cost=4.00..4.00 rows=200 width=13) (actual time=0.070..0.070 rows=200.00 loops=1)
31	Buckets: 1024 Batches: 1 Memory Usage: 17kB
32	Buffers: shared hit=2
33	-> Seq Scan on recurso r (cost=0.00..4.00 rows=200 width=13) (actual time=0.025..0.042 rows=200.00 loops=1)
34	Buffers: shared hit=2
35	Planning:
36	Buffers: shared hit=206 read=20
37	Planning Time: 7.305 ms
38	Execution Time: 1.321 ms

Figura 29: Elemento gráfico extraído de la página 60 del documento original.

5.4.5. Ejecución de consulta 2

5.4.5.1. Ejecución sin índices para 10k datos

5.4.5.2. Ejecución con índices para 10k datos



Figura 30: Query Plan Tree de la consulta 2 de 10k datos sin índices

	QUERY PLAN	
	text	
1	Aggregate (cost=57.00..57.01 rows=1 width=32) (actual time=0.431..0.432 rows=1.00 loops=1)	
2	Buffers: shared hit=25	
3	-> Seq Scan on vuelo v (cost=0.00..50.00 rows=700 width=16) (actual time=0.013..0.232 rows=600.00 loo...	
4	Filter: ((hora_real IS NOT NULL) AND (tipo = 'Internacional'::tipo_vuelo_t))	
5	Rows Removed by Filter: 1400	
6	Buffers: shared hit=25	
7	Planning:	
8	Buffers: shared hit=55 read=5	
9	Planning Time: 1.850 ms	
10	Execution Time: 0.459 ms	

Figura 31: Query Plan de la consulta 2 de 10k datos sin índices



Figura 32: Query Plan Tree de la consulta 2 de 10k datos con índices

5.4.6. Ejecución de consulta 3

5.4.6.1. Ejecución sin índices para 10k datos

	QUERY PLAN
	text
1	Aggregate (cost=56.20..56.22 rows=1 width=32) (actual time=0.591..0.592 rows=1.00 loops=1)
2	Buffers: shared hit=25 read=3
3	-> Bitmap Heap Scan on vuelo v (cost=15.45..49.20 rows=700 width=16) (actual time=0.111..0.235 rows=600.00 loops=1)
4	Recheck Cond: ((tipo = 'Internacional'::tipo_vuelo_t) AND (hora_real IS NOT NULL))
5	Heap Blocks: exact=25
6	Buffers: shared hit=25 read=3
7	-> Bitmap Index Scan on idx_vuelo_tipo_hora (cost=0.00..15.28 rows=700 width=0) (actual time=0.080..0.080 rows=600.00 loo...
8	Index Cond: ((tipo = 'Internacional'::tipo_vuelo_t) AND (hora_real IS NOT NULL))
9	Index Searches: 1
10	Buffers: shared read=3
11	Planning:
12	Buffers: shared hit=95 read=9
13	Planning Time: 6.027 ms
14	Execution Time: 0.674 ms

Figura 33: Query Plan de la consulta 2 de 10k datos con índices

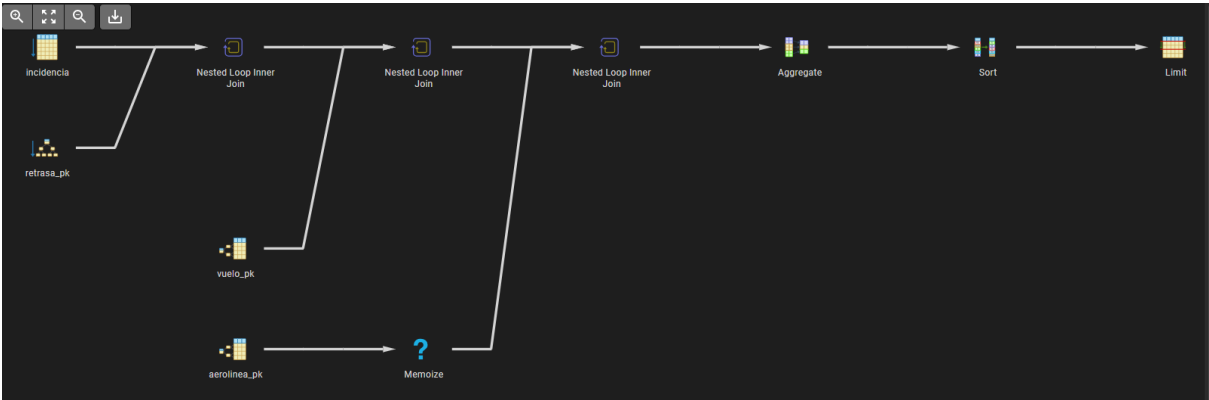


Figura 34: Query Plan Tree de la consulta 3 de 10k datos sin índices

	QUERY PLAN	
	text	
1	Limit (cost=1245.71..1245.71 rows=1 width=34) (actual time=4.693..4.696 rows=0.00 loops=1)	
2	Buffers: shared hit=3420 read=21	
3	-> Sort (cost=1245.71..1245.77 rows=24 width=34) (actual time=4.691..4.694 rows=0.00 loops=1)	
4	Disabled: true	
5	Sort Key: (count(*)) DESC	
6	Sort Method: quicksort Memory: 25kB	
7	Buffers: shared hit=3420 read=21	
8	-> HashAggregate (cost=1245.35..1245.59 rows=24 width=34) (actual time=4.685..4.688 rows=0.00 loops=1)	
9	Group Key: al.ruc	
10	Batches: 1 Memory Usage: 32kB	
11	Buffers: shared hit=3420 read=21	
12	-> Nested Loop (cost=0.71..1241.18 rows=834 width=26) (actual time=4.681..4.683 rows=0.00 loops=1)	
13	Buffers: shared hit=3420 read=21	
14	-> Nested Loop (cost=0.56..1216.82 rows=834 width=12) (actual time=4.680..4.681 rows=0.00 loops=1)	
15	Buffers: shared hit=3420 read=21	
16	-> Nested Loop (cost=0.28..948.77 rows=834 width=8) (actual time=4.679..4.680 rows=0.00 loops=1)	
17	Buffers: shared hit=3420 read=21	
18	-> Seq Scan on incidencia i (cost=0.00..232.00 rows=1667 width=8) (actual time=0.026..1.540 rows=1667.00 loops=1)	
19	Filter: (tipo_incidencia = 'Falta_Combustible'::tipo_incidencia_t)	
20	Rows Removed by Filter: 8333	
21	Buffers: shared hit=107	

Figura 35: Query Plan de la consulta 3 de 10k datos sin índices

22	-> Index Only Scan using retrasa_pk on retrasa rt (cost=0.28..0.42 rows=1 width=16) (actual time=0.002..0.002 rows=0.00 loops=1)
23	Index Cond: (id_incidencia = i.id)
24	Heap Fetches: 0
25	Index Searches: 1667
26	Buffers: shared hit=3313 read=21
27	-> Index Scan using vuelo_pk on vuelo v (cost=0.28..0.32 rows=1 width=20) (never executed)
28	Index Cond: (id = rt.id_vuelo)
29	Index Searches: 0
30	-> Memoize (cost=0.15..0.17 rows=1 width=26) (never executed)
31	Cache Key: v.ruc_aerolinea
32	Cache Mode: logical
33	-> Index Scan using aerolinea_pk on aerolinea al (cost=0.14..0.16 rows=1 width=26) (never executed)
34	Index Cond: ((ruc)::text = (v.ruc_aerolinea)::text)
35	Index Searches: 0
36	Planning:
37	Buffers: shared hit=233 read=12
38	Planning Time: 8.775 ms
39	Execution Time: 4.797 ms

Figura 36: Elemento gráfico extraído de la página 63 del documento original.

5.4.6.2. Ejecución con índices para 10k datos

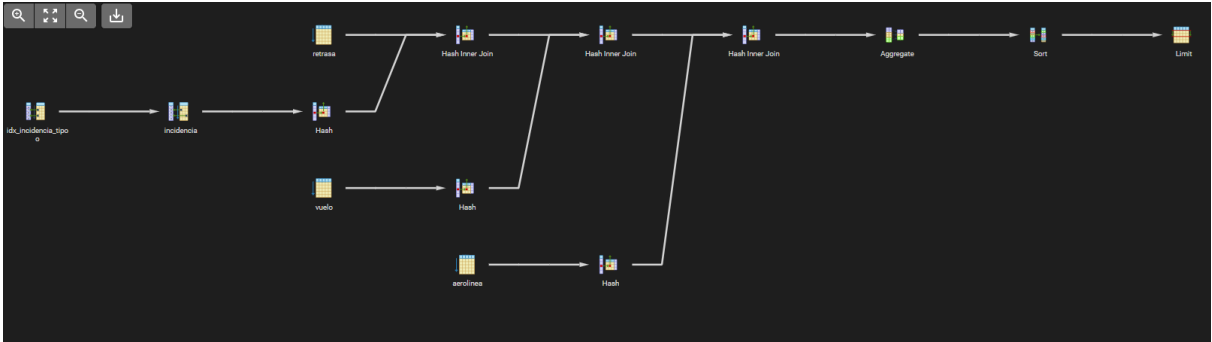


Figura 37: Query Plan Tree de la consulta 3 de 10k datos con índices

	QUERY PLAN
	text
1	Limit (cost=397.56..397.56 rows=1 width=34) (actual time=3.569..3.574 rows=0.00 loops=1)
2	Buffers: shared hit=167
3	-> Sort (cost=397.56..397.62 rows=24 width=34) (actual time=3.567..3.572 rows=0.00 loops=1)
4	Sort Key: (count(*)) DESC
5	Sort Method: quicksort Memory: 25kB
6	Buffers: shared hit=167
7	-> HashAggregate (cost=397.20..397.44 rows=24 width=34) (actual time=3.561..3.565 rows=0.00 loops=1)
8	Group Key: al.ruc
9	Batches: 1 Memory Usage: 32kB
10	Buffers: shared hit=167
11	-> Hash Join (cost=297.13..393.03 rows=834 width=26) (actual time=3.556..3.561 rows=0.00 loops=1)
12	Hash Cond: ((v.ruc_aerolinea)::text = (al.ruc)::text)
13	Buffers: shared hit=167
14	-> Hash Join (cost=295.59..388.91 rows=834 width=12) (actual time=3.514..3.517 rows=0.00 loops=1)
15	Hash Cond: (rt.id_vuelo = v.id)
16	Buffers: shared hit=166
17	-> Hash Join (cost=225.59..316.72 rows=834 width=8) (actual time=2.622..2.624 rows=0.00 loops=1)
18	Hash Cond: (rt.id_incidencia = i.id)
19	Buffers: shared hit=141
20	-> Seq Scan on retrasa rt (cost=0.00..78.00 rows=5000 width=16) (actual time=0.015..0.472 rows=5000.00 loops=1)
21	Buffers: shared hit=28

Figura 38: Elemento gráfico extraído de la página 64 del documento original.

Análisis de los planes de ejecución — Escenario 10 000 registros

Con 10 000 registros, los planes mantienen una estructura similar a la del escenario anterior, pero el efecto de los índices empieza a ser más perceptible. Sin índices, persisten los Seq Scan y los Nested Loop, ahora sobre conjuntos diez veces mayores, lo que incrementa el tiempo de las juntas. Con índices, el optimizador resuelve las juntas mediante Hash Join y aprovecha los Index Scan sobre las columnas de filtro (la fecha en la Consulta 1 y el tipo de incidencia en la Consulta 3), reduciendo el número de filas procesadas en cada etapa. La mejora de tiempos es ya consistente en las tres consultas (por ejemplo, la Consulta 1 baja de 2.826 ms a 1.264 ms), aunque el tamaño aún moderado de las tablas impide que la diferencia sea drástica.

EDICIÓN 100K REGISTROS

5.4.7. Ejecución de consulta 1

5.4.7.1. Ejecución sin índices para 100k datos

22	-> Hash (cost=204.76..204.76 rows=1667 width=8) (actual time=1.407..1.408 rows=1667.00 loops=1)
23	Buckets: 2048 Batches: 1 Memory Usage: 82kB
24	Buffers: shared hit=113
25	-> Bitmap Heap Scan on incidencia i (cost=76.92..204.76 rows=1667 width=8) (actual time=0.291..1.057 rows=1667.00 loops=1)
26	Recheck Cond: (tipo_incidencia = 'Falta_Combustible':tipo_incidencia_t)
27	Heap Blocks: exact=107
28	Buffers: shared hit=113
29	-> Bitmap Index Scan on idx_incidencia_tipo (cost=0.00..76.50 rows=1667 width=0) (actual time=0.237..0.237 rows=1667.00 loops=1)
30	Index Cond: (tipo_incidencia = 'Falta_Combustible':tipo_incidencia_t)
31	Index Searches: 1
32	Buffers: shared hit=6
33	-> Hash (cost=45.00..45.00 rows=2000 width=20) (actual time=0.867..0.868 rows=2000.00 loops=1)
34	Buckets: 2048 Batches: 1 Memory Usage: 118kB
35	Buffers: shared hit=25
36	-> Seq Scan on vuelo v (cost=0.00..45.00 rows=2000 width=20) (actual time=0.011..0.467 rows=2000.00 loops=1)
37	Buffers: shared hit=25
38	-> Hash (cost=1.24..1.24 rows=24 width=26) (actual time=0.037..0.038 rows=24.00 loops=1)
39	Buckets: 1024 Batches: 1 Memory Usage: 10kB
40	Buffers: shared hit=1

Figura 39: Query Plan de la consulta 3 de 10k datos con índices

41	-> Seq Scan on aerolinea al (cost=0.00..1.24 rows=24 width=26) (actual time=0.022..0.026 rows=24.00 loops=1)
42	Buffers: shared hit=1
43	Planning:
44	Buffers: shared hit=302 read=27
45	Planning Time: 17.570 ms
46	Execution Time: 3.755 ms

Figura 40: Elemento gráfico extraído de la página 65 del documento original.



Figura 41: Query Plan Tree de la consulta 1 de 100k datos sin índices

	QUERY PLAN	
	text	🔒
1	Limit (cost=4606.70..4606.70 rows=1 width=21) (actual time=69.828..78.582 rows=1.00 loops=1)	
2	Buffers: shared hit=5863 read=193	
3	-> Sort (cost=4606.70..4607.20 rows=200 width=21) (actual time=69.826..78.578 rows=1.00 loops=1)	
4	Disabled: true	
5	Sort Key: (count(*)) DESC	
6	Sort Method: top-N heapsort Memory: 25kB	
7	Buffers: shared hit=5863 read=193	
8	-> Finalize HashAggregate (cost=4603.70..4605.70 rows=200 width=21) (actual time=69.736..78.526 rows=100.00 loops=1)	
9	Group Key: r.id	
10	Batches: 1 Memory Usage: 32kB	
11	Buffers: shared hit=5863 read=193	
12	-> Gather (cost=4566.00..4602.00 rows=340 width=21) (actual time=56.795..78.397 rows=100.00 loops=1)	
13	Workers Planned: 1	
14	Workers Launched: 1	
15	Buffers: shared hit=5863 read=193	
16	-> Partial HashAggregate (cost=3566.00..3568.00 rows=200 width=21) (actual time=27.952..27.978 rows=50.00 loops=2)	
17	Group Key: r.id	
18	Batches: 1 Memory Usage: 32kB	
19	Buffers: shared hit=5863 read=193	
20	Worker 0: Batches: 1 Memory Usage: 32kB	
21	-> Nested Loop (cost=0.44..3563.28 rows=544 width=13) (actual time=0.153..27.717 rows=479.00 loops=2)	

Figura 42: Elemento gráfico extraído de la página 66 del documento original.

5.4.7.2. Ejecución con índices para 100k datos

22	Buffers: shared hit=5863 read=193
23	-> Nested Loop (cost=0.29..3533.32 rows=544 width=4) (actual time=0.134..27.071 rows=479.00 loops=2)
24	Buffers: shared hit=5664 read=192
25	-> Parallel Seq Scan on incidencia i (cost=0.00..2093.41 rows=1088 width=8) (actual time=0.045..21.949 rows=958.50 loops=2)
26	Filter: (fecha_reporte >= (now() - '7 days'::interval))
27	Rows Removed by Filter: 49042
28	Buffers: shared hit=1064
29	-> Index Only Scan using afecto_pk on afecto a (cost=0.29..1.31 rows=1 width=12) (actual time=0.005..0.005 rows=0.50 loops=...
30	Index Cond: (id_incidencia = i.id)
31	Heap Fetches: 958
32	Index Searches: 1917
33	Buffers: shared hit=4600 read=192
34	-> Memoize (cost=0.15..0.17 rows=1 width=13) (actual time=0.001..0.001 rows=1.00 loops=958)
35	Cache Key: a.id_recurso
36	Cache Mode: logical
37	Hits: 858 Misses: 100 Evictions: 0 Overflows: 0 Memory Usage: 12kB
38	Buffers: shared hit=199 read=1
39	-> Index Scan using recurso_pk on recurso r (cost=0.14..0.16 rows=1 width=13) (actual time=0.002..0.002 rows=1.00 loops=10...
40	Index Cond: (id = a.id_recurso)
41	Index Searches: 100
42	Buffers: shared hit=199 read=1

Figura 43: Query Plan de la consulta 1 de 100k datos sin índices

43	Planning:
44	Buffers: shared hit=141 read=9
45	Planning Time: 12.658 ms
46	Execution Time: 78.762 ms

Figura 44: Query Plan Tree de la consulta 1 de 100k datos con índices

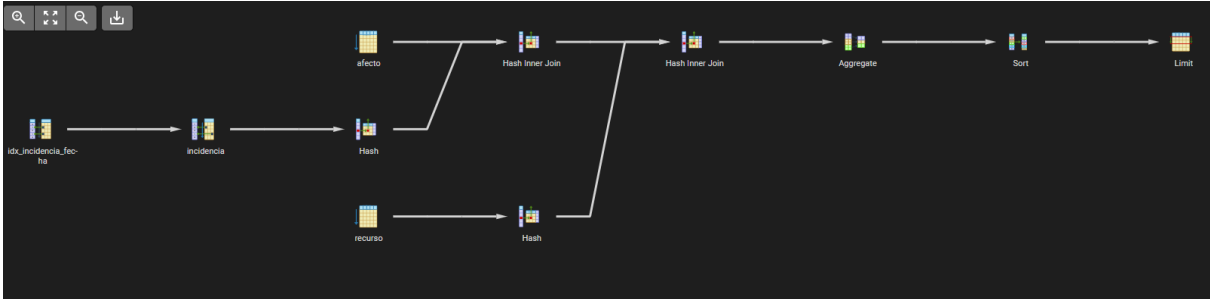


Figura 45: Elemento gráfico extraído de la página 67 del documento original.

5.4.8. Ejecución de consulta 2

5.4.8.1. Ejecución sin índices para 100k datos

	QUERY PLAN	
	text	🔒
1	Limit (cost=2096.05..2096.05 rows=1 width=21) (actual time=4.424..4.426 rows=1.00 loops=1)	
2	Buffers: shared hit=569 read=2	
3	-> Sort (cost=2096.05..2096.55 rows=200 width=21) (actual time=4.423..4.425 rows=1.00 loops=1)	
4	Sort Key: (count(*)) DESC	
5	Sort Method: top-N heapsort Memory: 25kB	
6	Buffers: shared hit=569 read=2	
7	-> HashAggregate (cost=2093.05..2095.05 rows=200 width=21) (actual time=4.399..4.409 rows=100.00 loops=1)	
8	Group Key: r.id	
9	Batches: 1 Memory Usage: 32kB	
10	Buffers: shared hit=566 read=2	
11	-> Hash Join (cost=1183.70..2088.43 rows=924 width=13) (actual time=0.519..4.316 rows=958.00 loops=1)	
12	Hash Cond: (a.id_recurso = r.id)	
13	Buffers: shared hit=566 read=2	
14	-> Hash Join (cost=1177.20..2079.45 rows=924 width=4) (actual time=0.495..4.200 rows=958.00 loops=1)	
15	Hash Cond: (a.id_incidencia = i.id)	
16	Buffers: shared hit=564 read=2	
17	-> Seq Scan on afecto a (cost=0.00..771.00 rows=50000 width=12) (actual time=0.004..1.463 rows=50000.00 loops=1)	
18	Buffers: shared hit=271	
19	-> Hash (cost=1154.10..1154.10 rows=1848 width=8) (actual time=0.486..0.486 rows=1917.00 loops=1)	
20	Buckets: 2048 Batches: 1 Memory Usage: 91kB	
21	Buffers: shared hit=293 read=2	

Figura 46: Query Plan de la consulta 1 de 100k datos con índices

22	-> Bitmap Heap Scan on incidencia i (cost=26.62..1154.10 rows=1848 width=8) (actual time=0.078..0.347 rows=1917.00 loops=1)
23	Recheck Cond: (fecha_reporte >= (now() - '7 days'::interval))
24	Heap Blocks: exact=291
25	Buffers: shared hit=293 read=2
26	-> Bitmap Index Scan on idx_incidencia_fecha (cost=0.00..26.16 rows=1848 width=0) (actual time=0.056..0.056 rows=1917.00 lo...
27	Index Cond: (fecha_reporte >= (now() - '7 days'::interval))
28	Index Searches: 1
29	Buffers: shared hit=2 read=2
30	-> Hash (cost=4.00..4.00 rows=200 width=13) (actual time=0.022..0.022 rows=200.00 loops=1)
31	Buckets: 1024 Batches: 1 Memory Usage: 17kB
32	Buffers: shared hit=2
33	-> Seq Scan on recurso r (cost=0.00..4.00 rows=200 width=13) (actual time=0.007..0.012 rows=200.00 loops=1)
34	Buffers: shared hit=2
35	Planning:
36	Buffers: shared hit=204 read=20
37	Planning Time: 1.780 ms
38	Execution Time: 4.454 ms

Figura 47: Elemento gráfico extraído de la página 68 del documento original.

5.4.8.2. Ejecución con índices para 100k datos



Figura 48: Query Plan Tree de la consulta 2 de 100k datos sin índices

	QUERY PLAN text	
1	Aggregate (cost=567.00..567.02 rows=1 width=32) (actual time=2.047..2.047 rows=1.00 loops=1)	
2	Buffers: shared hit=247	
3	-> Seq Scan on vuelo v (cost=0.00..497.00 rows=7000 width=16) (actual time=0.008..1.105 rows=6000.00 loo...	
4	Filter: ((hora_real IS NOT NULL) AND (tipo = 'Internacional'::tipo_vuelo_t))	
5	Rows Removed by Filter: 14000	
6	Buffers: shared hit=247	
7	Planning:	
8	Buffers: shared hit=53 read=5	
9	Planning Time: 0.799 ms	
10	Execution Time: 2.066 ms	

Figura 49: Query Plan de la consulta 2 de 100k datos sin índices

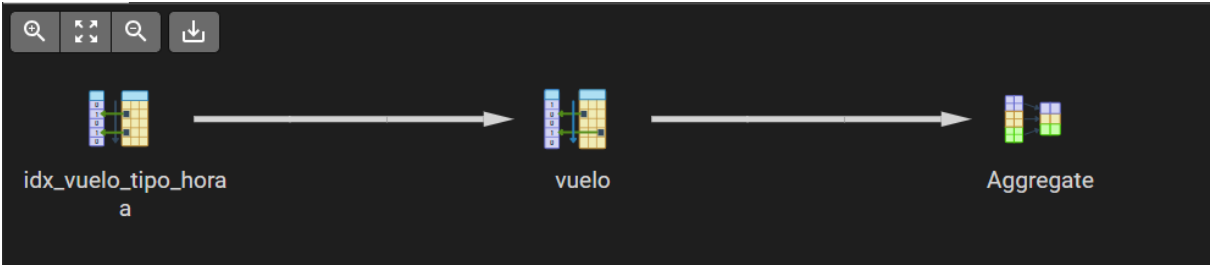


Figura 50: Query Plan Tree de la consulta 2 de 100k datos con índices

5.4.9. Ejecución de consulta 3

5.4.9.1. Ejecución sin índices para 100k datos

	QUERY PLAN
	text
1	Aggregate (cost=504.54..504.55 rows=1 width=32) (actual time=1.571..1.571 rows=1.00 loops=1)
2	Buffers: shared hit=247 read=7
3	-> Bitmap Heap Scan on vuelo v (cost=100.04..434.54 rows=7000 width=16) (actual time=0.149..0.564 rows=6000.00 loops=1)
4	Recheck Cond: ((tipo = 'Internacional'::tipo_vuelo_t) AND (hora_real IS NOT NULL))
5	Heap Blocks: exact=247
6	Buffers: shared hit=247 read=7
7	-> Bitmap Index Scan on idx_vuelo_tipo_hora (cost=0.00..98.29 rows=7000 width=0) (actual time=0.125..0.125 rows=6000.00 loo...
8	Index Cond: ((tipo = 'Internacional'::tipo_vuelo_t) AND (hora_real IS NOT NULL))
9	Index Searches: 1
10	Buffers: shared read=7
11	Planning:
12	Buffers: shared hit=89 read=9
13	Planning Time: 1.066 ms
14	Execution Time: 1.587 ms

Figura 51: Query Plan de la consulta 2 de 100k datos con índices

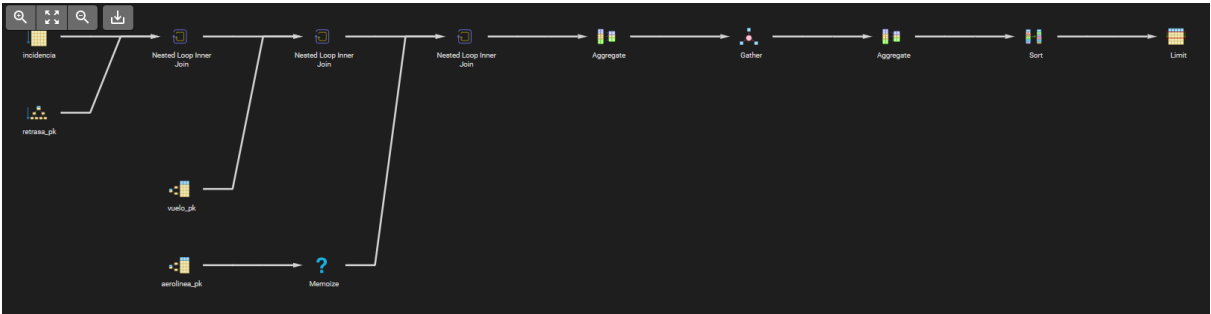


Figura 52: Query Plan Tree de la consulta 3 de 100k datos sin índices

	QUERY PLAN text	
1	Limit (cost=8707.89..8707.89 rows=1 width=34) (actual time=28.051..30.835 rows=0.00 loops=1)	
2	Buffers: shared hit=34205 read=193	
3	-> Sort (cost=8707.89..8707.95 rows=24 width=34) (actual time=28.050..30.834 rows=0.00 loops=1)	
4	Disabled: true	
5	Sort Key: (count(*)) DESC	
6	Sort Method: quicksort Memory: 25kB	
7	Buffers: shared hit=34205 read=193	
8	-> Finalize HashAggregate (cost=8707.53..8707.77 rows=24 width=34) (actual time=28.047..30.830 rows=0.00 loops=1)	
9	Group Key: al.ruc	
10	Batches: 1 Memory Usage: 32kB	
11	Buffers: shared hit=34205 read=193	
12	-> Gather (cost=8702.98..8707.32 rows=41 width=34) (actual time=28.046..30.829 rows=0.00 loops=1)	
13	Workers Planned: 1	
14	Workers Launched: 1	
15	Buffers: shared hit=34205 read=193	
16	-> Partial HashAggregate (cost=7702.98..7703.22 rows=24 width=34) (actual time=7.226..7.227 rows=0.00 loops=2)	
17	Group Key: al.ruc	
18	Batches: 1 Memory Usage: 32kB	
19	Buffers: shared hit=34205 read=193	
20	Worker 0: Batches: 1 Memory Usage: 32kB	
21	-> Nested Loop (cost=0.72..7678.80 rows=4836 width=26) (actual time=7.224..7.225 rows=0.00 loops=2)	

Figura 53: Elemento gráfico extraído de la página 71 del documento original.

22	Buffers: shared hit=34205 read=193
23	-> Nested Loop (cost=0.58..7556.82 rows=4836 width=12) (actual time=7.224..7.224 rows=0.00 loops=2)
24	Buffers: shared hit=34205 read=193
25	-> Nested Loop (cost=0.29..5964.23 rows=4836 width=8) (actual time=7.223..7.224 rows=0.00 loops=2)
26	Buffers: shared hit=34205 read=193
27	-> Parallel Seq Scan on incidencia i (cost=0.00..1799.29 rows=9672 width=8) (actual time=0.003..2.352 rows=8333.50 loops=2)
28	Filter: (tipo_incidencia = 'Falta_Combustible'::tipo_incidencia_t)
29	Rows Removed by Filter: 41666
30	Buffers: shared hit=1064
31	-> Index Only Scan using retrasa_pk on retrasa rt (cost=0.29..0.42 rows=1 width=16) (actual time=0.000..0.000 rows=0.00 loops=...)
32	Index Cond: (id_incidencia = i.id)
33	Heap Fetches: 0
34	Index Searches: 16667
35	Buffers: shared hit=33141 read=193
36	-> Index Scan using vuelo_pk on vuelo v (cost=0.29..0.33 rows=1 width=20) (never executed)
37	Index Cond: (id = rt.id_vuelo)
38	Index Searches: 0
39	-> Memoize (cost=0.15..0.17 rows=1 width=26) (never executed)
40	Cache Key: v.ruc_aerolinea
41	Cache Mode: logical

Figura 54: Elemento gráfico extraído de la página 71 del documento original.

5.4.9.2. Ejecución con índices para 100k datos

42	-> Index Scan using aerolinea_pk on aerolinea al (cost=0.14..0.16 rows=1 width=26) (never executed)
43	Index Cond: ((ruc)::text = (v.ruc_aerolinea)::text)
44	Index Searches: 0
45	Planning:
46	Buffers: shared hit=238 read=12
47	Planning Time: 2.335 ms
48	Execution Time: 30.880 ms

Figura 55: Query Plan de la consulta 3 de 100k datos sin índices

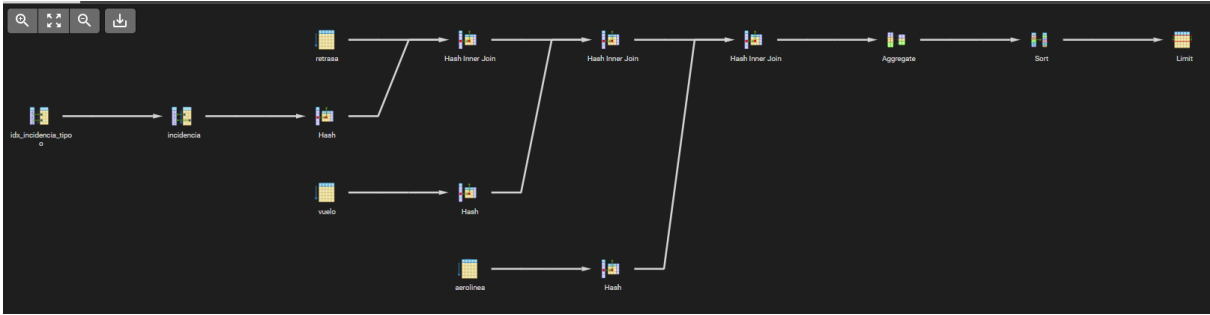


Figura 56: Query Plan Tree de la consulta 3 de 100k datos con índices

	QUERY PLAN
	text
1	Limit (cost=3787.75..3787.75 rows=1 width=34) (actual time=10.639..10.647 rows=0.00 loops=1)
2	Buffers: shared hit=1625
3	-> Sort (cost=3787.75..3787.81 rows=24 width=34) (actual time=10.639..10.645 rows=0.00 loops=1)
4	Sort Key: (count(*)) DESC
5	Sort Method: quicksort Memory: 25kB
6	Buffers: shared hit=1625
7	-> HashAggregate (cost=3787.39..3787.63 rows=24 width=34) (actual time=10.633..10.640 rows=0.00 loops=1)
8	Group Key: al.ruc
9	Batches: 1 Memory Usage: 32kB
10	Buffers: shared hit=1625
11	-> Hash Join (cost=2797.05..3746.28 rows=8222 width=26) (actual time=10.629..10.635 rows=0.00 loops=1)
12	Hash Cond: ((v.ruc_aerolinea)::text = (al.ruc)::text)
13	Buffers: shared hit=1625
14	-> Hash Join (cost=2795.51..3719.34 rows=8222 width=12) (actual time=10.613..10.618 rows=0.00 loops=1)
15	Hash Cond: (rt.id_vuelo = v.id)
16	Buffers: shared hit=1624
17	-> Hash Join (cost=2098.51..3000.76 rows=8222 width=8) (actual time=7.770..7.773 rows=0.00 loops=1)
18	Hash Cond: (rt.id_incidencia = i.id)
19	Buffers: shared hit=1377
20	-> Seq Scan on retrasa rt (cost=0.00..771.00 rows=50000 width=16) (actual time=0.005..1.636 rows=50000.00 loops=1)
21	Buffers: shared hit=271

Figura 57: Elemento gráfico extraído de la página 72 del documento original.

Análisis de los planes de ejecución — Escenario 100 000 registros

A partir de los 100 000 registros, la diferencia entre ambos escenarios se vuelve notoria. Sin índices, la Consulta 1 debe recorrer secuencialmente las 100 000 incidencias y filtrarlas una a una, lo que dispara su tiempo a unos 90 ms; lo mismo ocurre con la Consulta 3, cuyas cuatro juntas por nested loop sobre tablas grandes elevan su costo. Con índices, los planes cambian sustancialmente: el filtro por

rango de fechas se resuelve mediante un Index Scan sobre idx_incidencia_fecha, el índice hash sobre tipo_incidencia permite localizar directamente las incidencias de combustible, y las juntas se realizan con Hash Join. El resultado es una reducción superior al 90 % en la Consulta 1 (de 90 ms a 5 ms) y de cerca del 67 % en la Consulta 3. La Consulta 2, sobre una tabla mediana, presenta una mejora más contenida pero igualmente positiva.

EDICIÓN 1M REGISTROS

5.4.10. Ejecución de consulta 1

5.4.10.1. Ejecución sin índices para 1M datos

22	-> Hash (cost=1892.97..1892.97 rows=16443 width=8) (actual time=3.273..3.276 rows=16667.00 loops=1)
23	Buckets: 32768 Batches: 1 Memory Usage: 908kB
24	Buffers: shared hit=1106
25	-> Bitmap Heap Scan on incidencia i (cost=623.43..1892.97 rows=16443 width=8) (actual time=0.399..2.306 rows=16667.00 loops=1)
26	Recheck Cond: (tipo_incidencia = 'Falta_Combustible'::tipo_incidencia_t)
27	Heap Blocks: exact=1064
28	Buffers: shared hit=1106
29	-> Bitmap Index Scan on idx_incidencia_tipo (cost=0.00..619.32 rows=16443 width=0) (actual time=0.317..0.317 rows=16667.00 lo...
30	Index Cond: (tipo_incidencia = 'Falta_Combustible'::tipo_incidencia_t)
31	Index Searches: 1
32	Buffers: shared hit=42
33	-> Hash (cost=447.00..447.00 rows=20000 width=20) (actual time=2.778..2.779 rows=20000.00 loops=1)
34	Buckets: 32768 Batches: 1 Memory Usage: 1272kB
35	Buffers: shared hit=247
36	-> Seq Scan on vuelo v (cost=0.00..447.00 rows=20000 width=20) (actual time=0.005..1.445 rows=20000.00 loops=1)
37	Buffers: shared hit=247
38	-> Hash (cost=1.24..1.24 rows=24 width=26) (actual time=0.013..0.014 rows=24.00 loops=1)
39	Buckets: 1024 Batches: 1 Memory Usage: 10kB
40	Buffers: shared hit=1
41	-> Seq Scan on aerolinea al (cost=0.00..1.24 rows=24 width=26) (actual time=0.006..0.008 rows=24.00 loops=1)

Figura 58: Query Plan de la consulta 3 de 100k datos con índices

42	Buffers: shared hit=1
43	Planning:
44	Buffers: shared hit=321 read=29
45	Planning Time: 2.742 ms
46	Execution Time: 10.891 ms

Figura 59: Elemento gráfico extraído de la página 73 del documento original.

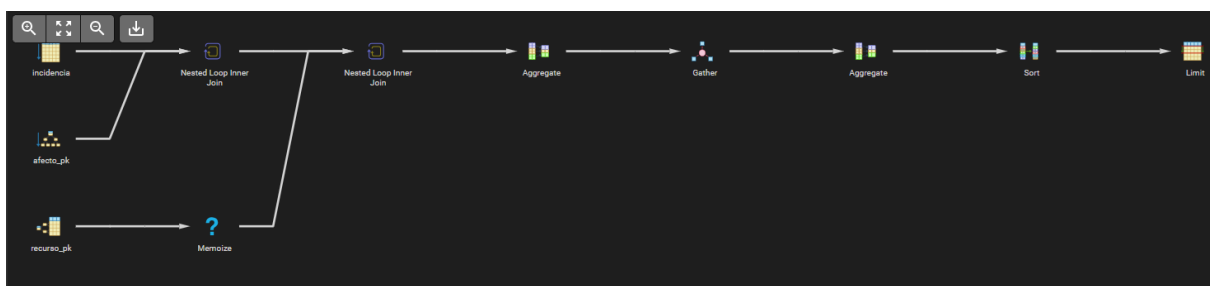


Figura 60: Query Plan Tree de la consulta 1 de 1M datos sin índices

	QUERY PLAN text	
1	Limit (cost=30579.25..30579.25 rows=1 width=21) (actual time=212.810..219.088 rows=1.00 loops=1)	
2	Buffers: shared hit=65651 read=12376	
3	-> Sort (cost=30579.25..30579.75 rows=200 width=21) (actual time=212.806..219.084 rows=1.00 loops=1)	
4	Disabled: true	
5	Sort Key: (count(*)) DESC	
6	Sort Method: top-N heapsort Memory: 25kB	
7	Buffers: shared hit=65651 read=12376	
8	-> Finalize HashAggregate (cost=30576.25..30578.25 rows=200 width=21) (actual time=212.737..219.035 rows=100.00 loops=1)	
9	Group Key: r.id	
10	Batches: 1 Memory Usage: 32kB	
11	Buffers: shared hit=65651 read=12376	
12	-> Gather (cost=30523.85..30573.85 rows=480 width=21) (actual time=212.601..218.926 rows=300.00 loops=1)	
13	Workers Planned: 2	
14	Workers Launched: 2	
15	Buffers: shared hit=65651 read=12376	
16	-> Partial HashAggregate (cost=29523.85..29525.85 rows=200 width=21) (actual time=166.958..166.975 rows=100.00 loops=3)	
17	Group Key: r.id	
18	Batches: 1 Memory Usage: 32kB	
19	Buffers: shared hit=65651 read=12376	
20	Worker 0: Batches: 1 Memory Usage: 32kB	
21	Worker 1: Batches: 1 Memory Usage: 32kB	

Figura 61: Elemento gráfico extraído de la página 74 del documento original.

5.4.10.2. Ejecución con índices para 1M datos

22	-> Nested Loop (cost=0.58..29503.07 rows=4155 width=13) (actual time=1.316..165.576 rows=3196.33 loops=3)
23	Buffers: shared hit=65651 read=12376
24	-> Nested Loop (cost=0.42..29383.11 rows=4155 width=4) (actual time=1.273..162.546 rows=3196.33 loops=3)
25	Buffers: shared hit=65050 read=12375
26	-> Parallel Seq Scan on incidencia i (cost=0.00..17930.67 rows=8310 width=8) (actual time=0.776..125.663 rows=6355.00 loops=...
27	Filter: (fecha_reporte >= (now() - '7 days'::interval))
28	Rows Removed by Filter: 326978
29	Buffers: shared hit=188 read=10451
30	-> Index Only Scan using afecto_pk on afecto a (cost=0.42..1.37 rows=1 width=12) (actual time=0.005..0.005 rows=0.50 loops=...
31	Index Cond: (id_incidencia = i.id)
32	Heap Fetches: 9589
33	Index Searches: 19065
34	Buffers: shared hit=64862 read=1924
35	-> Memoize (cost=0.15..0.17 rows=1 width=13) (actual time=0.000..0.000 rows=1.00 loops=9589)
36	Cache Key: a.id_recurso
37	Cache Mode: logical
38	Hits: 3548 Misses: 100 Evictions: 0 Overflows: 0 Memory Usage: 12kB
39	Buffers: shared hit=601 read=1
40	Worker 0: Hits: 2809 Misses: 100 Evictions: 0 Overflows: 0 Memory Usage: 12kB
41	Worker 1: Hits: 2932 Misses: 100 Evictions: 0 Overflows: 0 Memory Usage: 12kB

Figura 62: Query Plan de la consulta 1 de 1M datos sin índices

42	-> Index Scan using recurso_pk on recurso r (cost=0.14..0.16 rows=1 width=13) (actual time=0.002..0.002 rows=1.00 loops=300)
43	Index Cond: (id = a.id_recurso)
44	Index Searches: 300
45	Buffers: shared hit=601 read=1
46	Planning:
47	Buffers: shared hit=148 read=7
48	Planning Time: 7.933 ms
49	Execution Time: 219.286 ms

Figura 63: Query Plan Tree de la consulta 1 de 1M datos con índices



Figura 64: Elemento gráfico extraído de la página 75 del documento original.

	QUERY PLAN	
	text	
1	Limit (cost=18797.30..18797.31 rows=1 width=21) (actual time=130.680..138.058 rows=1.00 loops=1)	
2	Buffers: shared hit=2815 read=2833	
3	-> Sort (cost=18797.30..18797.80 rows=200 width=21) (actual time=130.677..138.054 rows=1.00 loops=1)	
4	Sort Key: (count(*)) DESC	
5	Sort Method: top-N heapsort Memory: 25kB	
6	Buffers: shared hit=2815 read=2833	
7	-> Finalize GroupAggregate (cost=18753.85..18796.30 rows=200 width=21) (actual time=130.468..138.006 rows=100.00 loops=1)	
8	Group Key: r.id	
9	Buffers: shared hit=2812 read=2833	
10	-> Gather Merge (cost=18753.85..18792.60 rows=340 width=21) (actual time=130.460..137.934 rows=200.00 loops=1)	
11	Workers Planned: 1	
12	Workers Launched: 1	
13	Buffers: shared hit=2812 read=2833	
14	-> Sort (cost=17753.84..17754.34 rows=200 width=21) (actual time=94.237..94.251 rows=100.00 loops=2)	
15	Sort Key: r.id	
16	Sort Method: quicksort Memory: 28kB	
17	Buffers: shared hit=2812 read=2833	
18	Worker 0: Sort Method: quicksort Memory: 28kB	
19	-> Partial HashAggregate (cost=17744.20..17746.20 rows=200 width=21) (actual time=94.160..94.188 rows=100.00 loops=2)	
20	Group Key: r.id	
21	Batches: 1 Memory Usage: 32kB	

Figura 65: Elemento gráfico extraído de la página 76 del documento original.

22	Buffers: shared hit=2805 read=2833
23	Worker 0: Batches: 1 Memory Usage: 32kB
24	-> Hash Join (cost=11282.96..17714.90 rows=5860 width=13) (actual time=12.579..92.768 rows=4737.50 loops=2)
25	Hash Cond: (a.id_recurso = r.id)
26	Buffers: shared hit=2805 read=2833
27	-> Parallel Hash Join (cost=11276.46..17692.70 rows=5860 width=4) (actual time=12.190..90.686 rows=4737.50 loops=2)
28	Hash Cond: (a.id_incidencia = i.id)
29	Buffers: shared hit=2801 read=2833
30	-> Parallel Seq Scan on afecto a (cost=0.00..5644.18 rows=294118 width=12) (actual time=0.010..24.633 rows=250000.00 loops=2)
31	Buffers: shared hit=2696 read=7
32	-> Parallel Hash (cost=11172.68..11172.68 rows=8302 width=8) (actual time=12.009..12.011 rows=9475.50 loops=2)
33	Buckets: 32768 Batches: 1 Memory Usage: 1024kB
34	Buffers: shared hit=105 read=2826
35	-> Parallel Bitmap Heap Scan on incidencia i (cost=226.85..11172.68 rows=8302 width=8) (actual time=2.060..18.187 rows=18951.00 lo...
36	Recheck Cond: (fecha_reporte >= (now() - '7 days'::interval))
37	Heap Blocks: exact=2912
38	Buffers: shared hit=105 read=2826
39	-> Bitmap Index Scan on idx_incidencia_fecha (cost=0.00..221.87 rows=19925 width=0) (actual time=1.586..1.586 rows=18951.00 lo...
40	Index Cond: (fecha_reporte >= (now() - '7 days'::interval))
41	Index Searches: 1

Figura 66: Elemento gráfico extraído de la página 76 del documento original.

5.4.11. Ejecución de consulta 2

5.4.11.1. Ejecución sin índices para 1M datos

42	Buffers: shared hit=3 read=16
43	-> Hash (cost=4.00..4.00 rows=200 width=13) (actual time=0.374..0.375 rows=200.00 loops=2)
44	Buckets: 1024 Batches: 1 Memory Usage: 17kB
45	Buffers: shared hit=4
46	-> Seq Scan on recurso r (cost=0.00..4.00 rows=200 width=13) (actual time=0.326..0.340 rows=200.00 loops=2)
47	Buffers: shared hit=4
48	Planning:
49	Buffers: shared hit=205 read=27
50	Planning Time: 9.985 ms
51	Execution Time: 138.521 ms

Figura 67: Query Plan de la consulta 1 de 1M datos con índices



Figura 68: Query Plan Tree de la consulta 2 de 1M datos sin índices

	QUERY PLAN
	text
1	Aggregate (cost=2834.65..2834.67 rows=1 width=32) (actual time=27.012..27.013 rows=1.00 loops=1)
2	Buffers: shared hit=1235
3	-> Seq Scan on vuelo v (cost=0.00..2485.00 rows=34965 width=16) (actual time=0.024..14.340 rows=30000.00 loo...
4	Filter: ((hora_real IS NOT NULL) AND (tipo = 'Internacional'::tipo_vuelo_t))
5	Rows Removed by Filter: 70000
6	Buffers: shared hit=1235
7	Planning:
8	Buffers: shared hit=62 read=6
9	Planning Time: 6.386 ms
10	Execution Time: 27.434 ms

Figura 69: Query Plan de la consulta 2 de 1M datos sin índices

5.4.11.2. Ejecución con índices para 1M datos

5.4.12. Ejecución de consulta 3

5.4.12.1. Ejecución sin índices para 1M datos

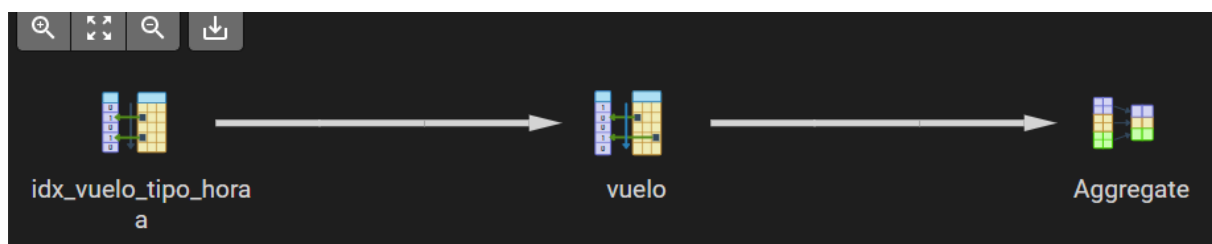


Figura 70: Query Plan Tree de la consulta 2 de 1M datos con índices

	QUERY PLAN
	text
1	Aggregate (cost=2508.40..2508.41 rows=1 width=32) (actual time=21.410..21.412 rows=1.00 loops=1)
2	Buffers: shared hit=1235 read=29
3	-> Bitmap Heap Scan on vuelo v (cost=486.68..2158.75 rows=34965 width=16) (actual time=2.103..8.634 rows=30000.00 loops=1)
4	Recheck Cond: ((tipo = 'Internacional'::tipo_vuelo_t) AND (hora_real IS NOT NULL))
5	Heap Blocks: exact=1235
6	Buffers: shared hit=1235 read=29
7	-> Bitmap Index Scan on idx_vuelo_tipo_hora (cost=0.00..477.94 rows=34965 width=0) (actual time=1.886..1.887 rows=30000.00 loo...
8	Index Cond: ((tipo = 'Internacional'::tipo_vuelo_t) AND (hora_real IS NOT NULL))
9	Index Searches: 1
10	Buffers: shared read=29
11	Planning:
12	Buffers: shared hit=91 read=10
13	Planning Time: 5.293 ms
14	Execution Time: 21.450 ms

Figura 71: Query Plan de la consulta 2 de 1M datos con índices

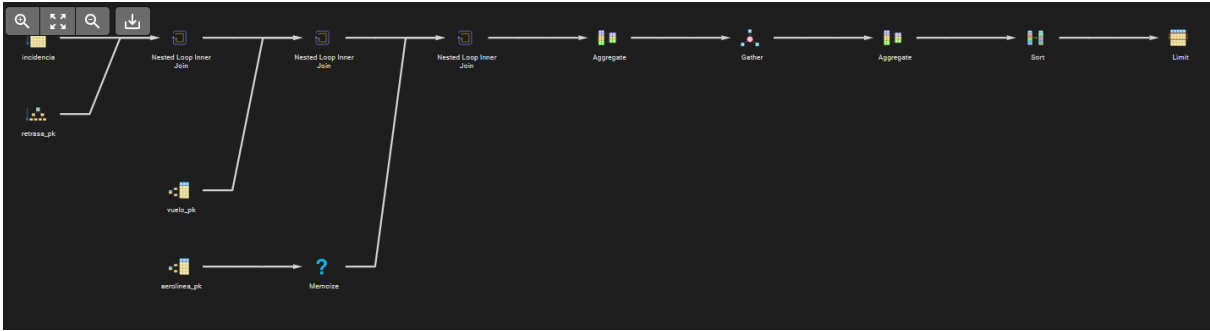


Figura 72: Query Plan Tree de la consulta 3 de 1M datos sin índices

	QUERY PLAN	
	text	
1	Limit (cost=68043.76..68043.77 rows=1 width=34) (actual time=268.117..274.294 rows=0.00 loops=1)	
2	Buffers: shared hit=498265 read=12377	
3	-> Sort (cost=68043.76..68043.82 rows=24 width=34) (actual time=268.115..274.292 rows=0.00 loops=1)	
4	Disabled: true	
5	Sort Key: (count(*)) DESC	
6	Sort Method: quicksort Memory: 25kB	
7	Buffers: shared hit=498265 read=12377	
8	-> Finalize HashAggregate (cost=68043.40..68043.64 rows=24 width=34) (actual time=268.108..274.284 rows=0.00 loops=1)	
9	Group Key: al.ruc	
10	Batches: 1 Memory Usage: 32kB	
11	Buffers: shared hit=498265 read=12377	
12	-> Gather (cost=68037.07..68043.11 rows=58 width=34) (actual time=268.105..274.279 rows=0.00 loops=1)	
13	Workers Planned: 2	
14	Workers Launched: 2	
15	Buffers: shared hit=498265 read=12377	
16	-> Partial HashAggregate (cost=67037.07..67037.31 rows=24 width=34) (actual time=216.085..216.087 rows=0.00 loops=3)	
17	Group Key: al.ruc	
18	Batches: 1 Memory Usage: 32kB	
19	Buffers: shared hit=498265 read=12377	
20	Worker 0: Batches: 1 Memory Usage: 32kB	
21	Worker 1: Batches: 1 Memory Usage: 32kB	

Figura 73: Elemento gráfico extraído de la página 79 del documento original.

5.4.12.2. Ejecución con índices para 1M datos

22	-> Nested Loop (cost=0.86..66863.39 rows=34736 width=26) (actual time=216.083..216.084 rows=0.00 loops=3)
23	Buffers: shared hit=498265 read=12377
24	-> Nested Loop (cost=0.71..66011.11 rows=34736 width=12) (actual time=216.082..216.083 rows=0.00 loops=3)
25	Buffers: shared hit=498265 read=12377
26	-> Nested Loop (cost=0.42..54823.06 rows=34736 width=8) (actual time=216.081..216.082 rows=0.00 loops=3)
27	Buffers: shared hit=498265 read=12377
28	-> Parallel Seq Scan on incidencia i (cost=0.00..15847.33 rows=69472 width=8) (actual time=0.744..61.915 rows=55555.67 loops=...
29	Filter: (tipo_incidencia = 'Falta_Combustible'::tipo_incidencia_t)
30	Rows Removed by Filter: 277778
31	Buffers: shared hit=186 read=10453
32	-> Index Only Scan using retrasa_pk on retrasa rt (cost=0.42..0.55 rows=1 width=16) (actual time=0.002..0.002 rows=0.00 loops=1...
33	Index Cond: (id_incidencia = i.id)
34	Heap Fetches: 0
35	Index Searches: 166667
36	Buffers: shared hit=498079 read=1924
37	-> Index Scan using vuelo_pk on vuelo v (cost=0.29..0.32 rows=1 width=20) (never executed)
38	Index Cond: (id = rt.id_vuelo)
39	Index Searches: 0
40	-> Memoize (cost=0.15..0.17 rows=1 width=26) (never executed)
41	Cache Key: v.ruc_aerolinea

Figura 74: Query Plan de la consulta 3 de 1M datos sin índices

42	Cache Mode: logical
43	-> Index Scan using aerolinea_pk on aerolinea al (cost=0.14..0.16 rows=1 width=26) (never executed)
44	Index Cond: ((ruc)::text = (v.ruc_aerolinea)::text)
45	Index Searches: 0
46	Planning:
47	Buffers: shared hit=266 read=12
48	Planning Time: 10.158 ms
49	Execution Time: 274.518 ms

Figura 75: Query Plan Tree de la consulta 3 de 1M datos con índices

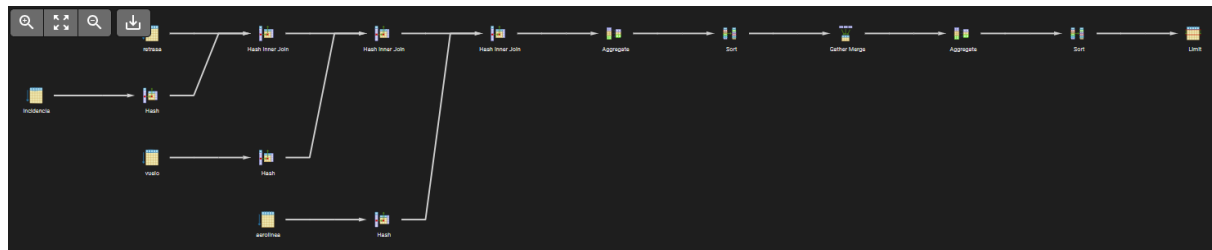


Figura 76: Elemento gráfico extraído de la página 80 del documento original.

	QUERY PLAN text	
1	Limit (cost=27223.46..27223.46 rows=1 width=34) (actual time=153.167..157.883 rows=0.00 loops=1)	
2	Buffers: shared hit=4231 read=10356	
3	-> Sort (cost=27223.46..27223.52 rows=24 width=34) (actual time=153.166..157.881 rows=0.00 loops=1)	
4	Sort Key: (count(*)) DESC	
5	Sort Method: quicksort Memory: 25kB	
6	Buffers: shared hit=4231 read=10356	
7	-> Finalize GroupAggregate (cost=27218.22..27223.34 rows=24 width=34) (actual time=153.163..157.878 rows=0.00 loops=1)	
8	Group Key: al.ruc	
9	Buffers: shared hit=4231 read=10356	
10	-> Gather Merge (cost=27218.22..27222.90 rows=41 width=34) (actual time=153.162..157.875 rows=0.00 loops=1)	
11	Workers Planned: 1	
12	Workers Launched: 1	
13	Buffers: shared hit=4231 read=10356	
14	-> Sort (cost=26218.21..26218.27 rows=24 width=34) (actual time=131.631..131.636 rows=0.00 loops=2)	
15	Sort Key: al.ruc	
16	Sort Method: quicksort Memory: 25kB	
17	Buffers: shared hit=4231 read=10356	
18	Worker 0: Sort Method: quicksort Memory: 25kB	
19	-> Partial HashAggregate (cost=26217.42..26217.66 rows=24 width=34) (actual time=131.611..131.616 rows=0.00 loops=2)	
20	Group Key: al.ruc	
21	Batches: 1 Memory Usage: 32kB	

Figura 77: Elemento gráfico extraído de la página 81 del documento original.

21	Batches: 1 Memory Usage: 32kB
22	Buffers: shared hit=4223 read=10356
23	Worker 0: Batches: 1 Memory Usage: 32kB
24	-> Hash Join (cost=19275.81..25972.23 rows=49039 width=26) (actual time=131.608..131.613 rows=0.00 loops=2)
25	Hash Cond: ((v.ruc_aerolinea)::text = (al.ruc)::text)
26	Buffers: shared hit=4223 read=10356
27	-> Parallel Hash Join (cost=19274.27..25819.23 rows=49039 width=12) (actual time=131.403..131.406 rows=0.00 loops=2)
28	Hash Cond: (rt.id_vuelo = v.id)
29	Buffers: shared hit=4221 read=10356
30	-> Parallel Hash Join (cost=16715.73..23131.97 rows=49039 width=8) (actual time=122.323..122.326 rows=0.00 loops=2)
31	Hash Cond: (rt.id_incidencia = i.id)
32	Buffers: shared hit=2986 read=10356
33	-> Parallel Seq Scan on retrasa rt (cost=0.00..5644.18 rows=294118 width=16) (actual time=0.008..14.117 rows=250000.00 loop...
34	Buffers: shared hit=2703
35	-> Parallel Hash (cost=15847.33..15847.33 rows=69472 width=8) (actual time=61.885..61.887 rows=83333.50 loops=2)
36	Buckets: 262144 Batches: 1 Memory Usage: 8608kB
37	Buffers: shared hit=283 read=10356
38	-> Parallel Seq Scan on incidencia i (cost=0.00..15847.33 rows=69472 width=8) (actual time=0.157..43.428 rows=83333.50 lo...
39	Filter: (tipo_incidencia = 'Falta_Combustible'::tipo_incidencia_t)
40	Rows Removed by Filter: 416666
41	Buffers: shared hit=283 read=10356

Figura 78: Elemento gráfico extraído de la página 81 del documento original.

Análisis de los planes de ejecución — Escenario 1 000 000 registros

En el volumen de un millón de registros, los planes incorporan paralelización para afrontar el tamaño de los datos. Sin índices, las consultas recurren a Parallel Seq Scan sobre Incidencia, repartiendo el trabajo entre varios procesos (se observan Worker 0 y Worker 1 en los planes), junto con operadores Memoize que cachean resultados de subconsultas repetidas para reducir accesos. Aun así, los tiempos

son elevados por la magnitud de las tablas (la Consulta 3 alcanza ~404 ms). Con índices, la Consulta 1 y la Consulta 3 aprovechan los Index Scan y los Index Only Scan sobre las claves foráneas (por ejemplo, Index Only Scan using afecto_pk) y resuelven las juntas con Hash Join, reduciendo de forma significativa el tiempo (la Consulta 3 baja a ~156 ms). Cabe destacar que, a este volumen, incluso con índices persiste un costo apreciable derivado del procesamiento y la agregación de grandes conjuntos de filas: la indexación, aunque indispensable, no elimina el impacto del crecimiento de los datos, sino que lo atenúa drásticamente.

5.5. Medición de Tiempos

Para la medición de tiempos se ejecutó cada consulta cinco veces en cada volumen de datos (1 000, 10 000, 100 000 y 1 000 000), tanto sin índices como con índices. Se descartó la primera ejecución en frío por reflejar el costo de carga inicial desde disco, y se promediaron cinco ejecuciones estables en caché. A partir de ellas se calcularon el promedio y la desviación estándar. Todos los tiempos están expresados en milisegundos (ms).

42	-> Parallel Hash (cost=1823.24..1823.24 rows=58824 width=20) (actual time=8.893..8.893 rows=50000.00 loops=2)
43	Buckets: 131072 Batches: 1 Memory Usage: 6528kB
44	Buffers: shared hit=1235
45	-> Parallel Seq Scan on vuelo v (cost=0.00..1823.24 rows=58824 width=20) (actual time=0.003..7.533 rows=100000.00 loops=1)
46	Buffers: shared hit=1235
47	-> Hash (cost=1.24..1.24 rows=24 width=26) (actual time=0.188..0.189 rows=24.00 loops=2)
48	Buckets: 1024 Batches: 1 Memory Usage: 10kB
49	Buffers: shared hit=2
50	-> Seq Scan on aerolinea al (cost=0.00..1.24 rows=24 width=26) (actual time=0.179..0.180 rows=24.00 loops=2)
51	Buffers: shared hit=2
52	Planning:
53	Buffers: shared hit=302 read=34
54	Planning Time: 2.889 ms
55	Execution Time: 157.934 ms

Figura 79: Query Plan de la consulta 3 de 1M datos con índices

5.5.1. Sin Índices

Tiempos de la Consulta 1 sin índices				
Ejecución	1k	10k	100k	1M
1	0.446	2.816	78.762	219.286
2	0.755	2.776	93.276	259.134
3	0.544	2.771	96.705	204.228
4	0.654	3.068	92.244	286.923
5	0.715	2.700	89.609	323.084
Promedio	0.623	2.826	90.119	258.531
Desv. Est.	0.127	0.141	6.839	48.638

Tabla 22: Tiempos de la Consulta 1 sin índices (ms)

Tiempos de la Consulta 2 sin índices				
Ejecución	1k	10k	100k	1M
1	0.160	0.459	2.066	27.434
2	0.129	0.797	2.195	22.284
3	0.089	0.934	1.950	25.317
4	0.090	1.062	1.983	37.553
5	0.229	0.757	2.210	27.630
Promedio	0.139	0.802	2.081	28.044
Desv. Est.	0.058	0.226	0.119	5.735

Tabla 23: Tiempos de la Consulta 2 sin índices (ms)

Tiempos de la Consulta 3 sin índices				
Ejecución	1k	10k	100k	1M
1	0.389	4.797	30.880	274.518
2	0.447	4.795	27.408	321.219
3	0.430	5.541	37.633	512.346
4	0.406	5.208	32.300	442.067
5	0.399	5.223	34.309	468.313
Promedio	0.414	5.113	32.506	403.693
Desv. Est.	0.024	0.318	3.815	101.171

Tabla 24: Tiempos de la Consulta 3 sin índices (ms)

5.5.2. Con Índices

Tiempos de la Consulta 1 con índices				
Ejecución	1k	10k	100k	1M
1	0.229	1.321	4.454	138.521
2	0.259	1.006	4.477	182.724
3	0.210	1.171	7.253	133.137
4	0.218	1.316	4.435	128.913
5	0.227	1.506	4.362	131.138
Promedio	0.229	1.264	4.996	142.887
Desv. Est.	0.019	0.187	1.262	22.552

Tabla 25: Tiempos de la Consulta 1 con índices (ms)

Tiempos de la Consulta 2 con índices				
Ejecución	1k	10k	100k	1M
1	0.164	0.654	1.573	21.450
2	0.092	0.741	1.587	17.135
3	0.096	0.697	1.583	15.823
4	0.184	0.674	1.591	22.394
5	0.097	0.722	1.597	19.266
Promedio	0.127	0.698	1.586	19.214
Desv. Est.	0.044	0.035	0.009	2.781

Tabla 26: Tiempos de la Consulta 2 con índices (ms)

Tiempos de la Consulta 3 con índices				
Ejecución	1k	10k	100k	1M
1	0.312	3.755	10.891	154.378
2	0.323	3.532	10.597	143.676
3	0.342	3.392	11.163	157.934
4	0.394	3.882	10.079	162.062
5	0.310	3.764	10.737	162.641
Promedio	0.336	3.665	10.693	156.138
Desv. Est.	0.035	0.198	0.403	7.730

Tabla 27: Tiempos de la Consulta 3 con índices (ms)

5.6. Resultados

Los resultados de la experimentación se presentan a continuación mediante gráficas que comparan, para cada consulta, el tiempo de ejecución promedio sin índices (línea roja) y con índices (línea azul) frente al volumen de datos. Por la amplitud del rango de tiempos medidos (desde fracciones de milisegundo hasta cientos de milisegundos), se emplea una escala logarítmica en ambos ejes, lo que permite visualizar con claridad la separación entre ambas curvas en todos los volúmenes.

5.6.1. Consulta 1

La Consulta 1 muestra el contraste más marcado. Mientras que sin índices el tiempo crece de 0.62 ms a 258.53 ms, con índices se mantiene mucho más contenido, alcanzando solo 142.89 ms en el millón de registros. El punto más notable se da en 100 000 registros, donde el índice reduce el tiempo de 90.12 ms a 4.99 ms, una mejora del 94 %.

5.6.2. Consulta 2

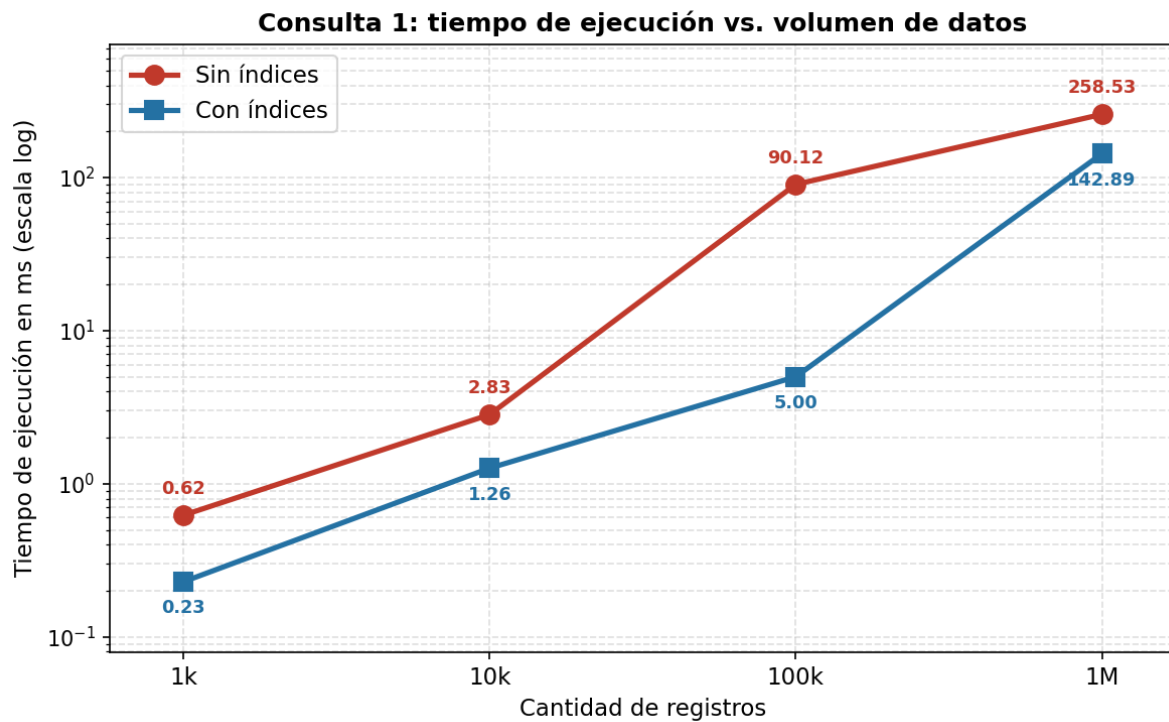


Figura 80: Tiempo de ejecución de la Consulta 1 según volumen de datos.

La Consulta 2, por operar sobre una sola tabla con una agregación simple, presenta los tiempos más bajos de las tres. La mejora del índice es modesta en volúmenes pequeños pero crece de forma sostenida, llegando a una reducción del 31 % en el millón de registros (de 28.04 ms a 19.21 ms).

5.6.3. Consulta 3

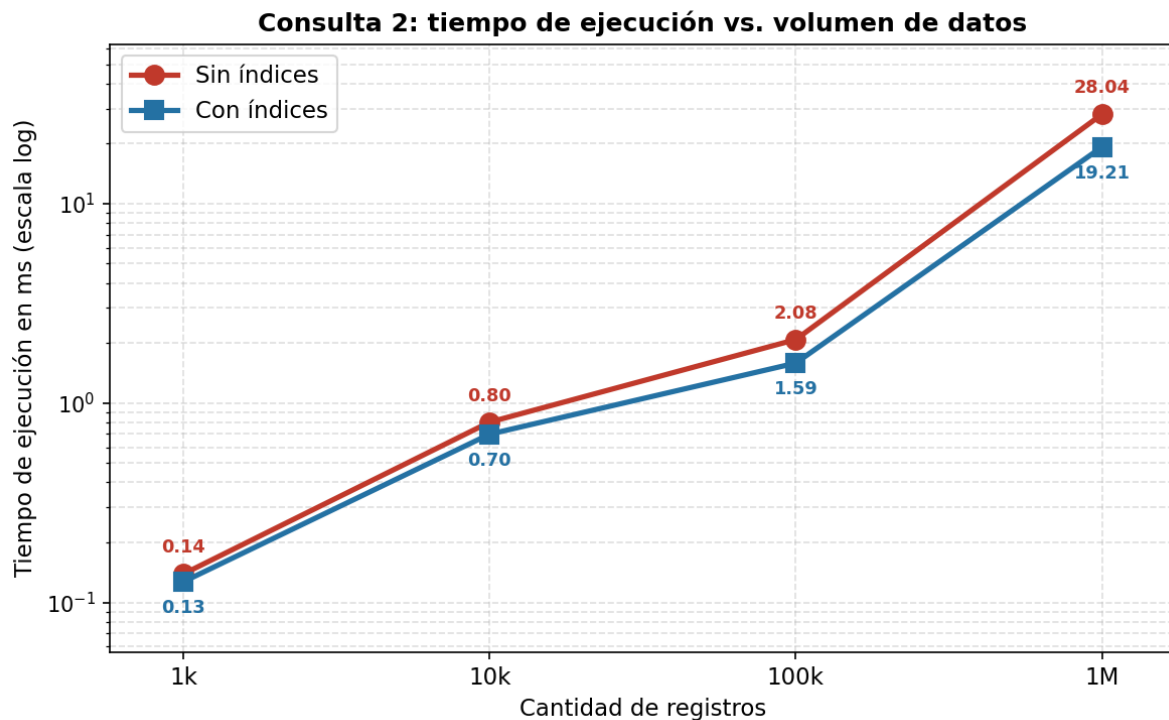


Figura 81: Tiempo de ejecución de la Consulta 2 según volumen de datos.

La Consulta 3, la más compleja por encadenar cuatro tablas, registra los tiempos más altos sin índices (hasta 403.69 ms en 1M). Con índices, el tiempo se reduce a 156.14 ms, una mejora del 61 %. La brecha entre ambas curvas se ensancha progresivamente con el volumen.

5.7. Análisis y Discusión

Los resultados obtenidos permiten extraer varias conclusiones sobre el impacto de los índices en el rendimiento de las consultas.

Impacto creciente de los índices con el volumen. El hallazgo más consistente es que el beneficio de la indexación es prácticamente nulo en volúmenes pequeños y se vuelve determinante a medida que crecen los datos. En 1 000 registros, las diferencias entre ejecutar con o sin índices son de fracciones de milisegundo, e incluso el planificador puede optar por no usar el índice, ya que recorrer secuencialmente una tabla pequeña es más económico que acceder mediante una estructura auxiliar. Sin embargo, a partir de 100 000 registros la diferencia se vuelve drástica: la Consulta 1 pasa de 90 ms a 5 ms. Esto confirma que los índices son una herramienta indispensable para bases de datos de gran escala, pero de utilidad marginal en conjuntos reducidos.

Relación entre complejidad de la consulta y beneficio. Las consultas que involucran más juntas y filtros selectivos (Consultas 1 y 3) se benefician notablemente más de los índices que la consulta de agregación simple sobre una sola tabla (Consulta 2). Esto se debe a que los índices optimizan tanto los filtros (mediante Index Scan) como las juntas (permitiendo Hash Joins eficientes en lugar de Nested Loops

costosos).

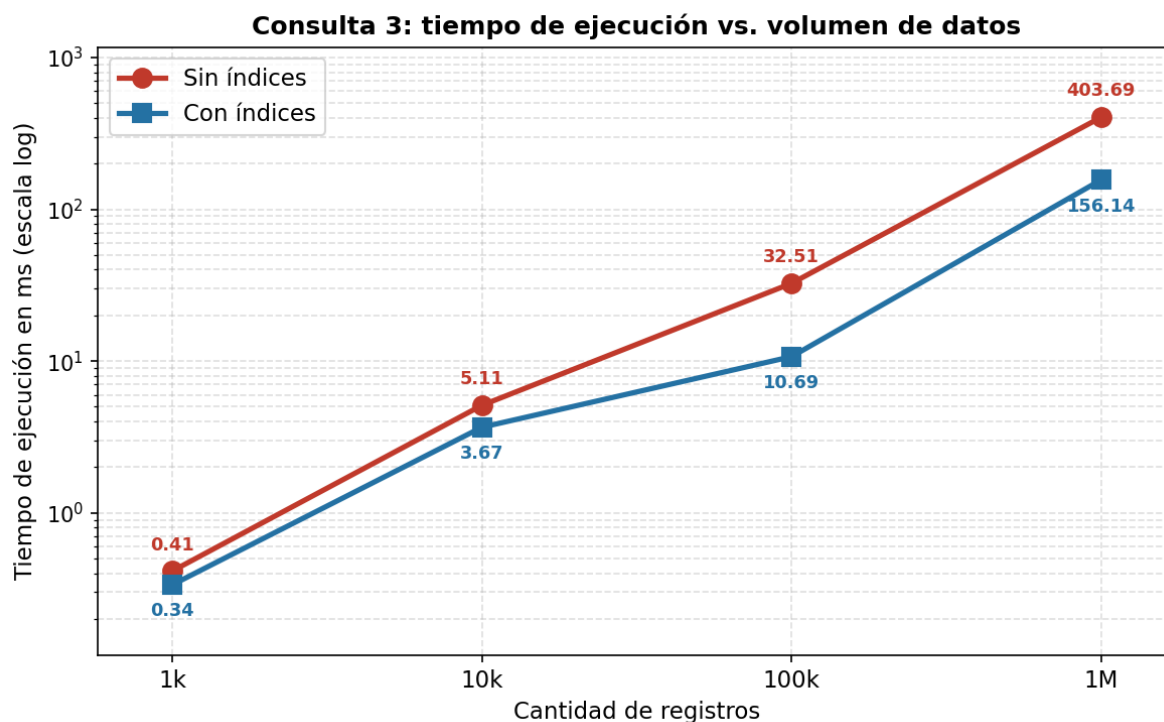


Figura 82: Tiempo de ejecución de la Consulta 3 según volumen de datos.

Estabilidad de los tiempos. Además de reducir el tiempo promedio, los índices disminuyen la desviación estándar de las mediciones, produciendo tiempos de respuesta más predecibles, una propiedad deseable en sistemas en producción.

Cambio en los planes de ejecución. El análisis de los planes muestra que, al escalar a un millón de registros, PostgreSQL recurre automáticamente a la paralelización (Parallel Seq Scan, operadores Gather y múltiples workers) y a mecanismos de caché como Memoize. Aun así, los índices siguen aportando una mejora sustancial sobre estos mecanismos automáticos.

Costo de los índices. Es importante señalar que los índices no son gratuitos: ocupan espacio adicional en disco y penalizan las operaciones de escritura (INSERT, UPDATE, DELETE), pues cada modificación obliga a actualizar también las estructuras de índice. Por ello, la indexación debe aplicarse selectivamente sobre las columnas efectivamente usadas en filtros y juntas frecuentes, y no de manera indiscriminada.

Complejidad operacional al escalar más allá del millón

Si se escalara la base de datos por encima del millón de registros, el comportamiento de las consultas dependería críticamente de la presencia de índices. Sin índices, una búsqueda requiere un recorrido secuencial completo de la tabla, con complejidad $O(n)$: al duplicar los datos, se duplica el tiempo. Con un índice de tipo B-tree, la búsqueda tiene complejidad $O(\log n)$, de modo que incluso multiplicando los datos por diez, el tiempo de localización crece de forma casi imperceptible. Esta diferencia, ya visible en nuestros resultados, se acentuaría aún más a mayores volúmenes: la curva sin índices crecería de forma lineal (o peor, en consultas con juntas múltiples, de forma cercana a $O(n \cdot m)$), mientras que la curva con índices permanecería casi plana en los filtros.

No obstante, los índices no resuelven todos los problemas de escala. A volúmenes de decenas o cientos de millones de registros, las operaciones de agregación y ordenamiento sobre grandes conjuntos de filas (como las de nuestras Consultas 1 y 3) siguen teniendo un costo considerable, pues deben procesar todas las filas que cumplen el filtro, independientemente del índice.

¿Es suficiente la arquitectura cliente-servidor? Para volúmenes en el orden de millones, una arquitectura cliente-servidor tradicional con una única instancia de PostgreSQL resulta adecuada, especialmente con índices apropiados, particionamiento de tablas (partitioning) y ajuste de parámetros de memoria. Sin embargo, al escalar a cientos de millones o miles de millones de registros, o ante una alta concurrencia de usuarios, una sola instancia se convierte en cuello de botella, limitada por la memoria, el disco y la capacidad de un único servidor. En ese punto sería necesario recurrir a estrategias de escalado horizontal: réplicas de lectura para distribuir las consultas, particionamiento distribuido o sharding (repartir los datos entre varios servidores), y eventualmente soluciones de bases de datos distribuidas. En conclusión, la arquitectura cliente-servidor es suficiente para el alcance de este proyecto y para volúmenes de hasta algunos millones de registros bien indexados, pero requeriría evolucionar hacia arquitecturas distribuidas para sostener cargas significativamente mayores.

6. Stored Procedures

El modelo implementado soporta diversas operaciones de manipulación de datos propias de la operativa del aeropuerto, como el registro de pasajeros, la emisión de tickets con su respectivo check-in y el reporte de incidencias sobre los recursos del terminal. Para abstraer estas operaciones y garantizar la integridad de los datos, se implementaron procedimientos almacenados (stored procedures) que encapsulan la lógica de inserción coordinada en múltiples tablas, de modo que el

usuario final pueda ejecutar estas acciones sin conocer los detalles internos del esquema. El uso de procedimientos almacenados asegura además que las operaciones que afectan a varias tablas se realicen de forma coherente, preservando las restricciones de integridad referencial del modelo.

6.1. Registro de un nuevo pasajero

Este procedimiento crea de forma coordinada una nueva persona y su correspondiente registro de pasajero, respetando la relación de herencia entre ambas entidades. Recibe los datos personales y migratorios, inserta primero la fila en Persona, obtiene el identificador generado automáticamente y lo utiliza para crear la fila asociada en Pasajero. Devuelve el identificador del nuevo pasajero.

```
1 CREATE OR REPLACE FUNCTION registrar_pasajero(  
2  
3 p_nombre VARCHAR, p_apellido VARCHAR, p_fecha_nac DATE,  
4  
5 p_tipo_doc tipo_doc_t, p_num_doc VARCHAR, p_id_categoria INT  
6  
7 ) RETURNS INT AS $$  
8  
9 DECLARE
```

```

10
11  nuevo_id INT;
12
13  BEGIN
14
15  INSERT INTO Persona (nombre, apellido, fecha_nacimiento)
16
17  VALUES (p_nombre, p_apellido, p_fecha_nac)
18
19  RETURNING id_persona INTO nuevo_id;
20
21
22
23  INSERT INTO Pasajero (id_persona, tipo_documento, numero_documento, id_categoria)
24
25  VALUES (nuevo_id, p_tipo_doc, p_num_doc, p_id_categoria);
26
27
28
29  RETURN nuevo_id;
30
31  END;
32
33  $$ LANGUAGE plpgsql;

```

6.2. Emisión de un ticket con su check-in

Este procedimiento modela el proceso de facturación de un pasajero: emite un nuevo ticket para un vuelo dado, le asigna directamente el estado de boarding. Check-in genera de forma automática el registro de check-in asociado, con el mostrador (counter) y la indicación de si el pasajero registró equipaje. De este modo, la emisión del ticket y el check-in quedan vinculados en una sola operación atómica. Devuelve el identificador del ticket emitido.

```

1  CREATE OR REPLACE FUNCTION emitir_ticket_con_checkin(
2
3  p_id_vuelo BIGINT, p_id_persona INT, p_precio NUMERIC,
4
5  p_counter VARCHAR, p_con_equipaje BOOLEAN
6
7  ) RETURNS BIGINT AS $$
8
9  DECLARE
10
11  nuevo_ticket BIGINT;
12
13  BEGIN
14

```

```

15 INSERT INTO Ticket (precio, fecha_emision, estado_boarding, id_vuelo, id_persona)
16
17 VALUES (p_precio, CURRENT_DATE, 'Check-in', p_id_vuelo, p_id_persona)
18
19 RETURNING id_ticket INTO nuevo_ticket;
20
21
22
23 INSERT INTO CheckIn (id_ticket, fecha_hora, counter, con equipaje)
24
25 VALUES (nuevo_ticket, NOW(), p_counter, p_con equipaje);
26
27
28
29 RETURN nuevo_ticket;
30
31 END;
32
33 $$ LANGUAGE plpgsql;

```

6.3. Registro de una incidencia sobre un recurso

Este procedimiento permite reportar una nueva incidencia en la infraestructura del terminal y vincularla simultáneamente al recurso afectado. Inserta la incidencia con su gravedad, descripción, tipo y fecha de reporte, obtiene su identificador y crea la relación correspondiente en la tabla Afecto. Refleja la dependencia existencial de la entidad débil Incidencia respecto del recurso al que afecta. Devuelve el identificador de la incidencia registrada.

```

1 CREATE OR REPLACE FUNCTION registrar_incidencia_recurso(
2
3 p_gravedad gravedad_t, p_descripcion TEXT,
4
5 p_tipo tipo_incidencia_t, p_id_recurso INT
6
7 ) RETURNS BIGINT AS $$
8
9 DECLARE
10
11 nueva_inc BIGINT;
12
13
14 BEGIN
15
16 INSERT INTO Incidencia (gravedad, descripcion, tipo_incidencia, fecha_reporte)
17
18 VALUES (p_gravedad, p_descripcion, p_tipo, NOW())
19

```

```
20 RETURNING id INTO nueva_inc;
21
22
23
24 INSERT INTO Afecto (id_incidencia, id_recurso)
25
26 VALUES (nueva_inc, p_id_recurso);
27
28
29
30 RETURN nueva_inc;
31
32 END;
33
34 $$ LANGUAGE plpgsql;
```

7. Conclusiones

1. El modelo diseñado representa fielmente la operativa del Aeropuerto Internacional Jorge Chávez. El modelo entidad-relación y su transformación al modelo relacional, compuesto por veintiuna relaciones, lograron capturar los procesos centrales del terminal: la gestión de vuelos, pasajeros, tripulación, equipaje, check-in e incidencias de infraestructura. La incorporación de entidades débiles (Asiento, Incidencia y CheckIn), la jerarquía de especialización entre Persona, Pasajero y Personal, y las relaciones entre recursos y vuelos permitieron normalizar la información y preservar la integridad referencial mediante claves foráneas y restricciones de dominio.
 2. La combinación de datos reales y sintéticos resultó adecuada para un dominio confidencial. Dado que la información operativa real del aeropuerto no es de acceso público, se optó por poblar los catálogos (aerolíneas, aeronaves, rutas y categorías migratorias) con información verificable de fuentes públicas, y generar sintéticamente la información masiva mediante `generate_series`. Esta estrategia permitió construir cuatro escenarios de volumen (1 000, 10 000, 100 000 y 1 000 000 de registros) reproducibles y coherentes con las restricciones del modelo.
 3. Los índices producen mejoras de rendimiento marginales en volúmenes pequeños pero determinantes a gran escala. La experimentación demostró que en 1 000 registros la diferencia entre ejecutar con o sin índices es de fracciones de milisegundo, mientras que a partir de 100 000 registros la mejora se vuelve drástica: la Consulta 1 redujo su tiempo en un 94 % (de 90 ms a 5 ms) y la Consulta 3 en un 61 % al alcanzar el millón de registros (de 404 ms a 156 ms). Esto confirma que la indexación es una herramienta indispensable para bases de datos de gran escala.
1. El beneficio del índice depende de la complejidad de la consulta. Las consultas que encadenan múltiples juntas y filtros selectivos (Consultas 1 y 3) se beneficiaron notablemente más de los índices que la consulta de agregación simple sobre una sola tabla (Consulta 2), pues los índices optimizan tanto los filtros, mediante Index Scan, como las juntas, sustituyendo los costosos Nested Loops por Hash Joins más eficientes.

2. La elección del tipo de índice debe responder a la naturaleza de la operación. El uso de índices btree para filtros por rango (como las fechas de incidencias) y de índices hash para búsquedas por igualdad (como el tipo de incidencia) demostró ser una decisión acertada, alineada con el funcionamiento interno del planificador de PostgreSQL.
3. Los índices conllevan un costo que obliga a un uso selectivo. Si bien aceleran las consultas de lectura, los índices ocupan espacio adicional en disco y penalizan las operaciones de escritura, ya que cada inserción o actualización debe propagarse a las estructuras de índice. Por ello, la indexación debe aplicarse únicamente sobre las columnas efectivamente utilizadas en filtros y juntas frecuentes.
4. La arquitectura cliente-servidor es suficiente para el alcance del proyecto, pero tiene límites de escalabilidad. Para volúmenes de hasta algunos millones de registros bien indexados, una única instancia de PostgreSQL ofrece un rendimiento adecuado. Sin embargo, ante volúmenes significativamente mayores o alta concurrencia, sería necesario evolucionar hacia estrategias de escalado horizontal como réplicas de lectura, particionamiento o sharding.
5. Los procedimientos almacenados y las vistas de usuario facilitan la manipulación segura de la base de datos. La implementación de stored procedures para operaciones coordinadas (registro de pasajeros, emisión de tickets con check-in y reporte de incidencias) y de vistas orientadas a cada tipo de usuario permitió abstraer la complejidad del esquema y garantizar la coherencia de las operaciones, sentando las bases para una eventual aplicación de gestión sobre la base de datos.

8. Anexos

En esta sección se incluyen los materiales complementarios que respaldan la implementación y la experimentación del proyecto. Todos los archivos se encuentran disponibles en el siguiente enlace de Google Drive:

[AEROPUERTO_JORGE_CHÁVEZ](#)

El repositorio contiene los siguientes elementos:

- Dumps de la base de datos: cuatro respaldos independientes en formato SQL, correspondientes a los cuatro escenarios de volumen utilizados en la experimentación (1 000, 10 000, 100 000 y 1 000 000 de registros), generados mediante la utilidad pg_dump de PostgreSQL.
- Video de la experimentación: grabación de la ejecución de las tres consultas sobre el escenario de un millón de registros, mostrando la comparación de los tiempos de respuesta con y sin índices mediante el comando EXPLAIN ANALYZE.
- Scripts SQL: los archivos de creación de la estructura de la base de datos, carga de catálogos, generación de datos sintéticos, vistas de usuario y procedimientos almacenados.